

เอกสารประกอบการอบรม
โครงการอบรมเชิงปฏิบัติการ

การพัฒนาทักษะการเขียนโปรแกรมด้านเทคโนโลยี Internet of
Things (IoT) ในระดับเบื้องต้น

โดย

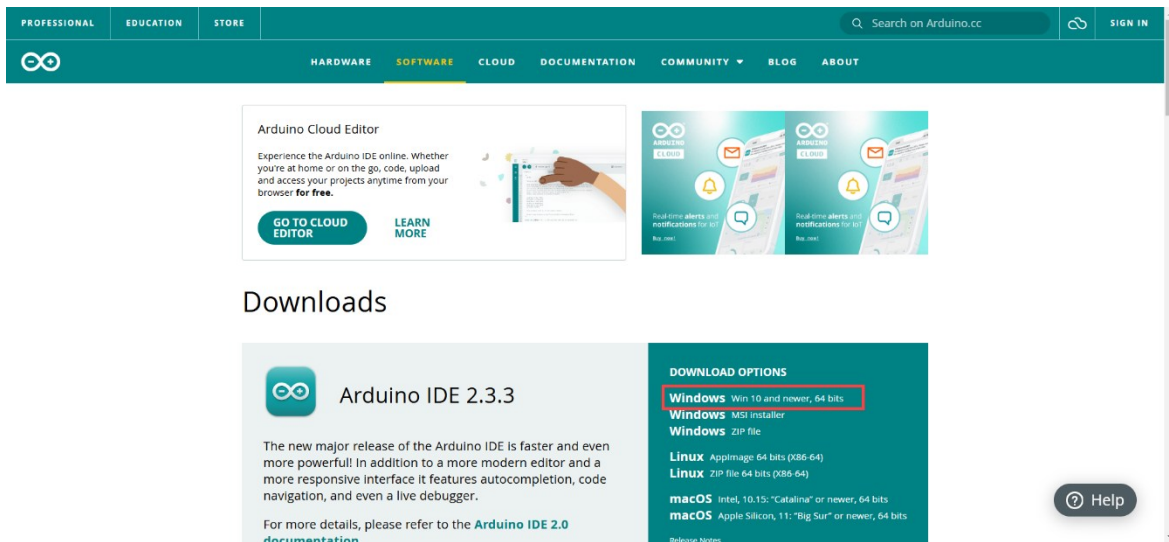
คณะเทคโนโลยีอุตสาหกรรม มหาวิทยาลัยราชภัฏอุบลราชธานี

ตอนที่ 1

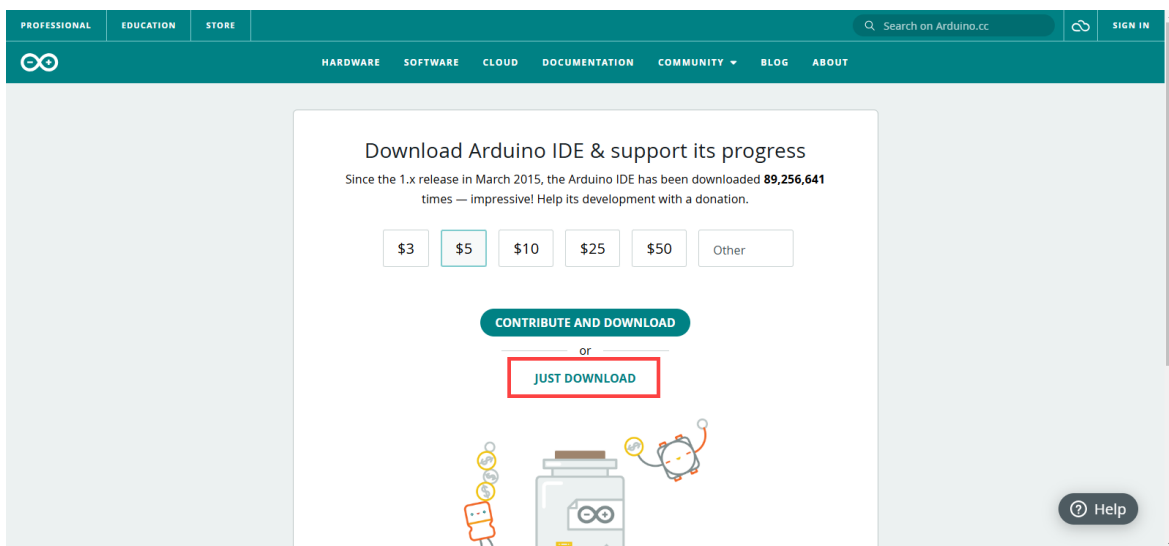
การติดตั้งโปรแกรม Arduino IDE และไลบรารี (Library)

การติดตั้ง Arduino IDE สำหรับการเขียนโปรแกรมเพื่อควบคุมการทำงานของไมโครคอนโทรลเลอร์ ESP32 มีขั้นตอนดังต่อไปนี้

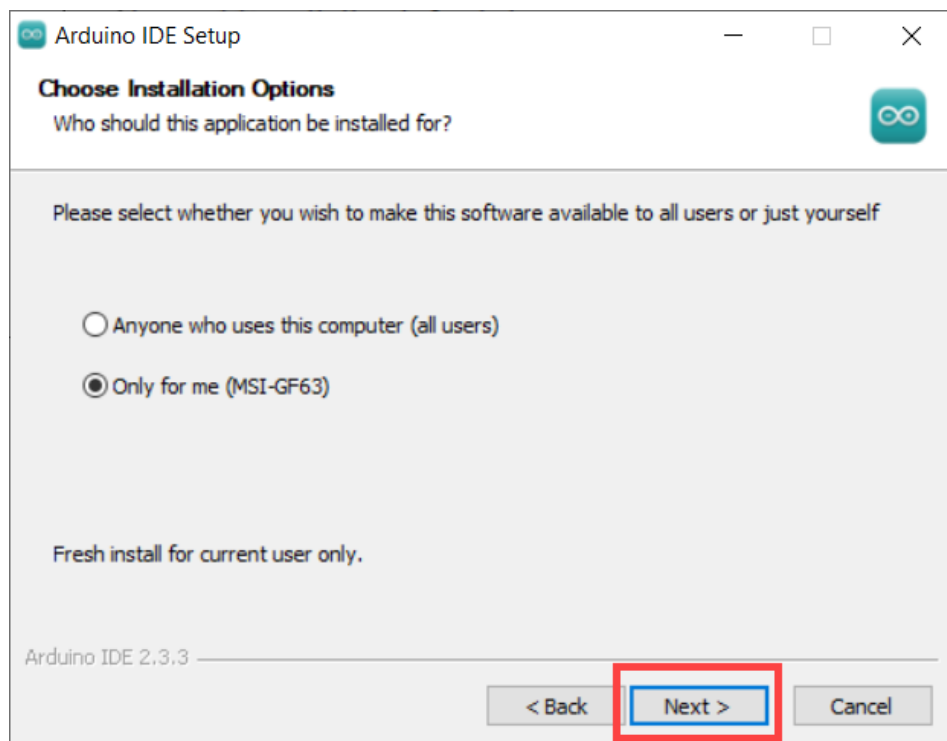
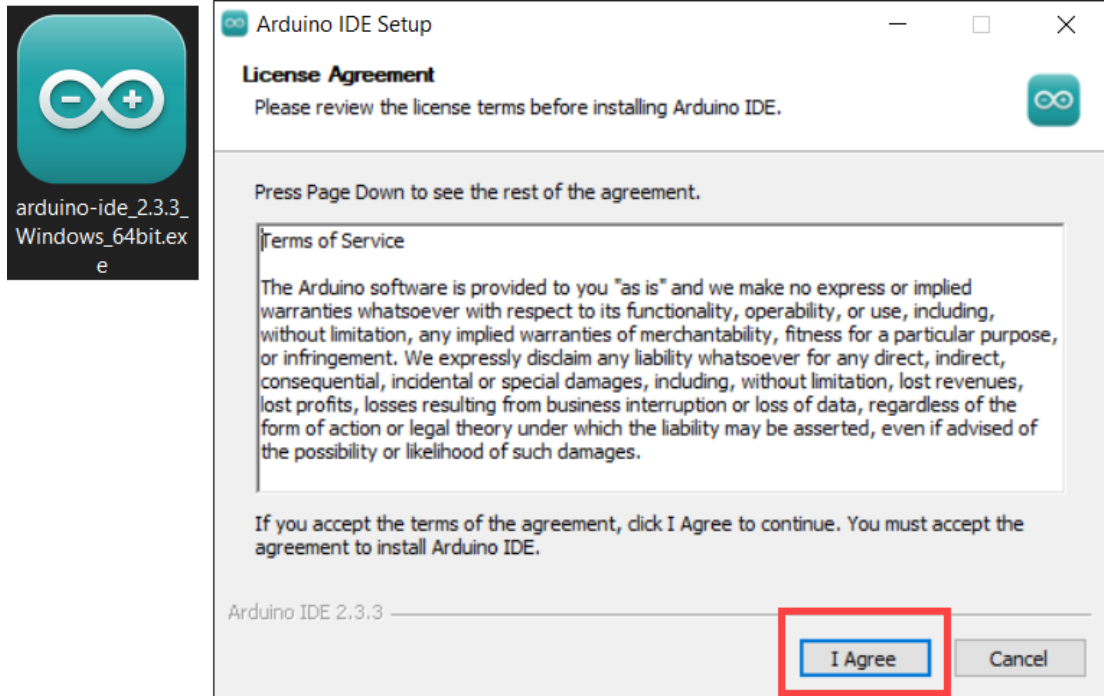
- 1) ดาวน์โหลดโปรแกรม Arduino IDE ได้ที่ <https://www.arduino.cc/en/Main/Software>
- 2) เลือกรายการ Windows installer, for Windows 10 and newer, 64 bits

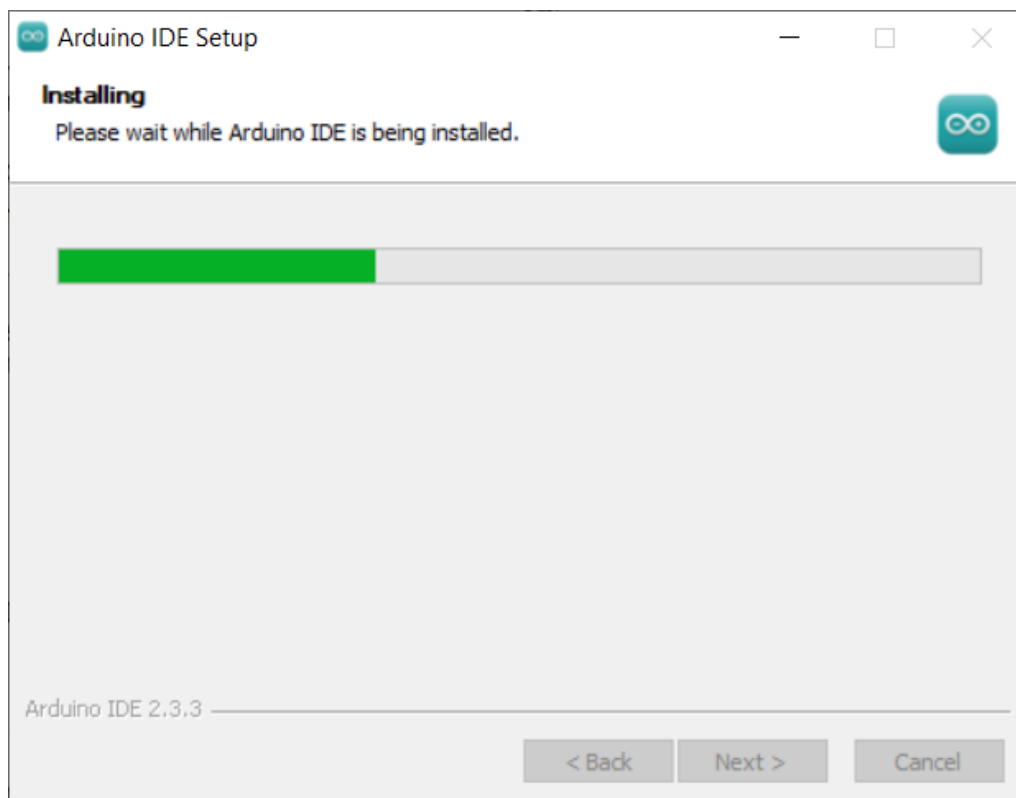
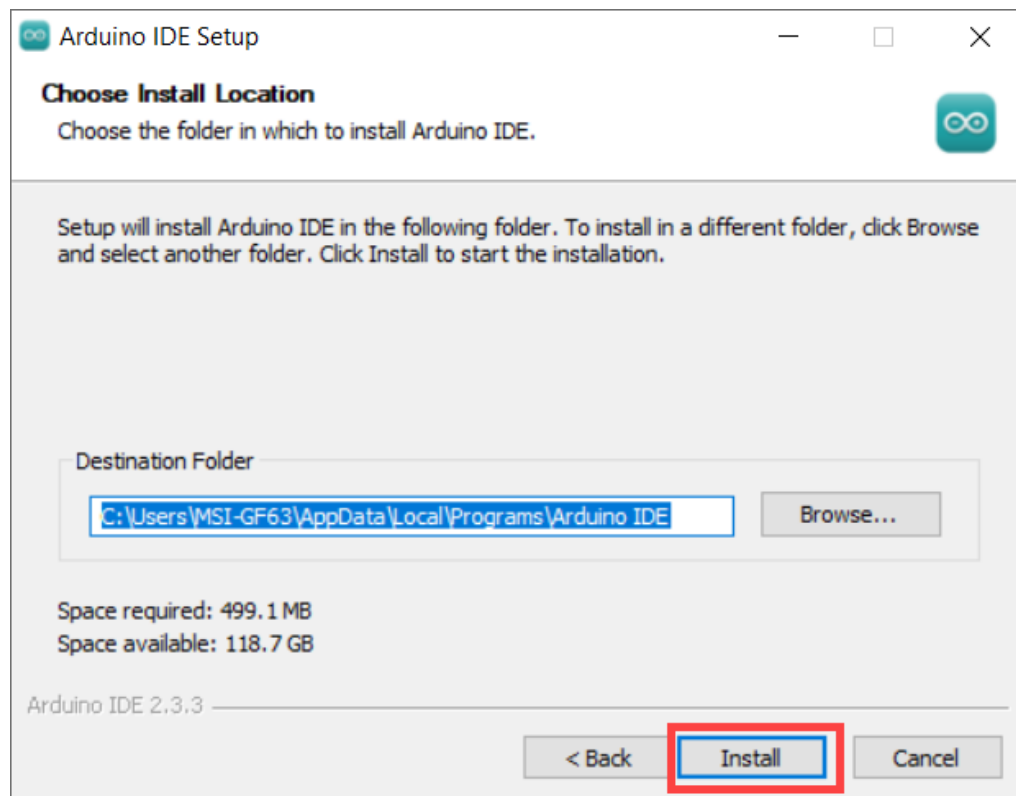


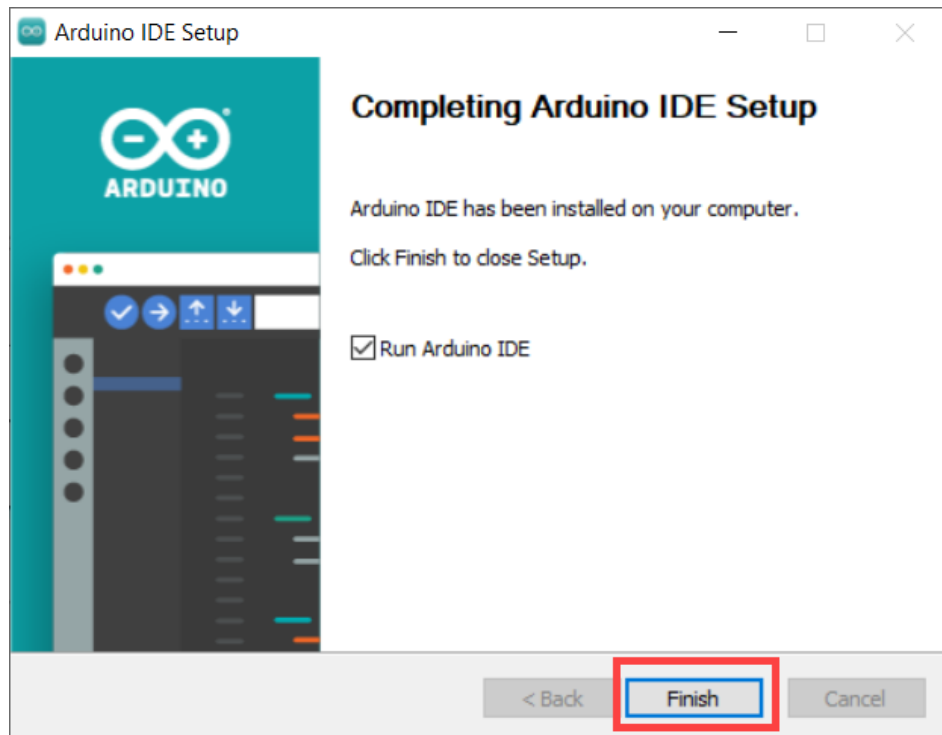
- 3) เลือกรายการ JUST DOWNLOAD



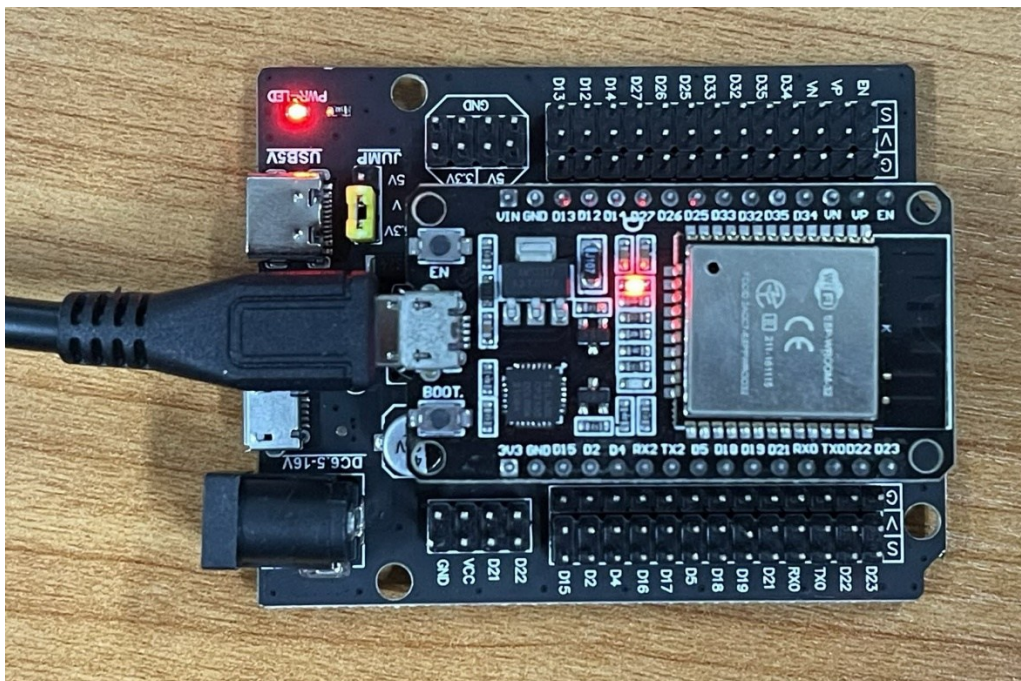
4) เมื่อดาวน์โหลดโปรแกรม Arduino IDE เสร็จ ให้ทำการดับเบิลคลิกเพื่อทำการติดตั้งโปรแกรม จากนั้นให้คลิกเลือกปุ่ม I Agree เพื่อยอมรับการติดตั้ง ทาการคลิกปุ่ม Next จากนั้นรอนจนกว่าโปรแกรมจะทำการติดตั้งเสร็จสิ้น แล้วทำการคลิกปุ่ม Close



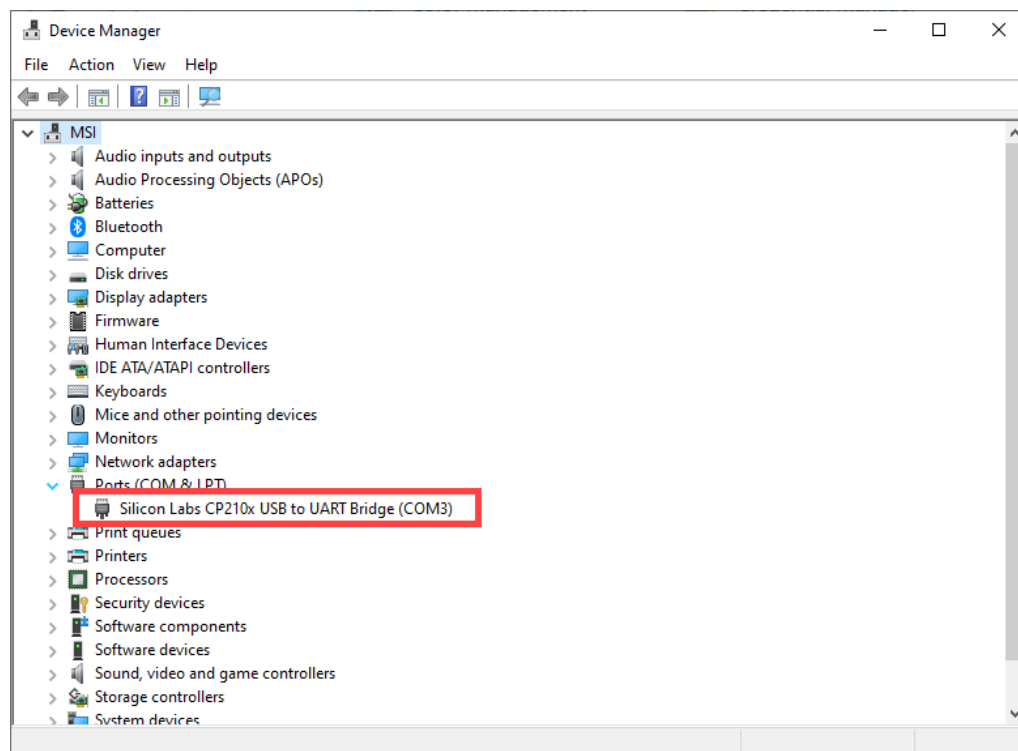
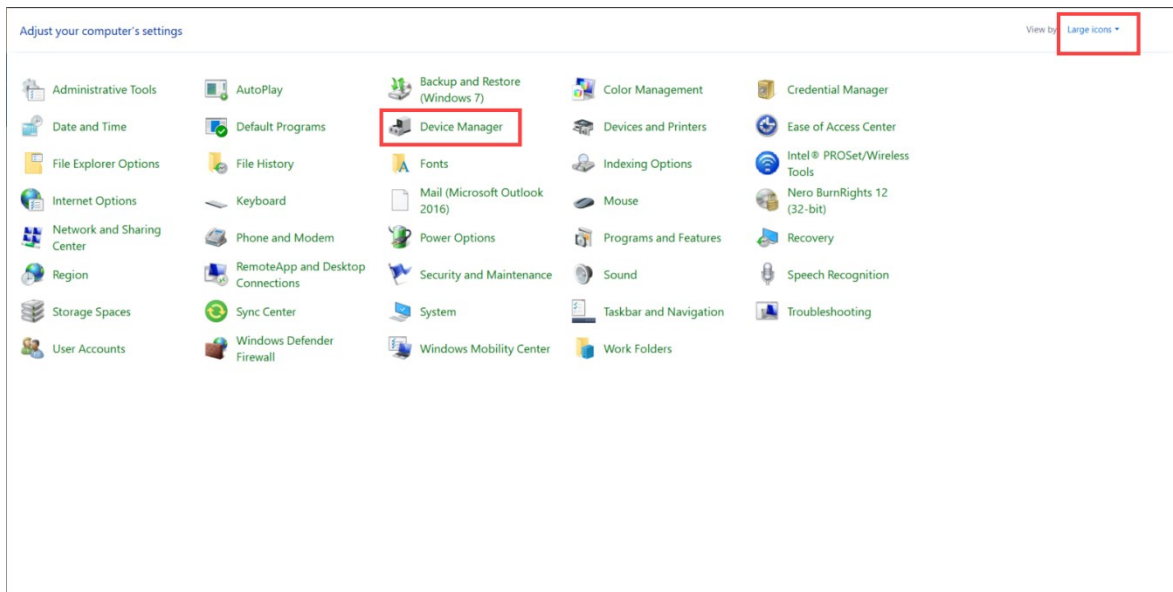




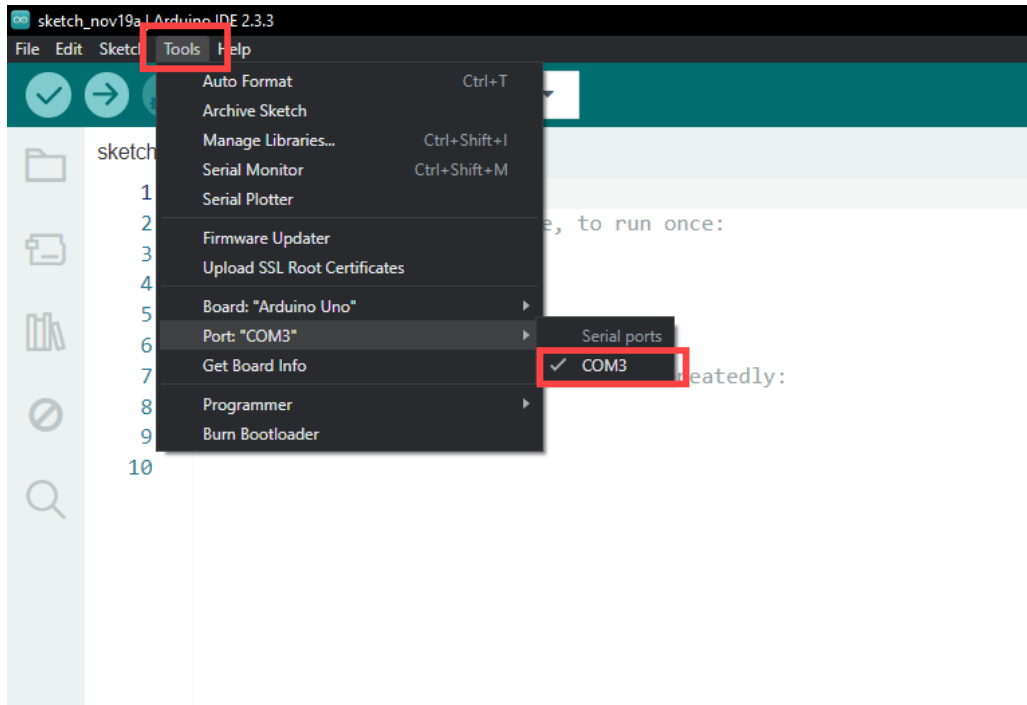
5) เสียบสาย USB เข้าคอมพิวเตอร์และบอร์ด ESP32



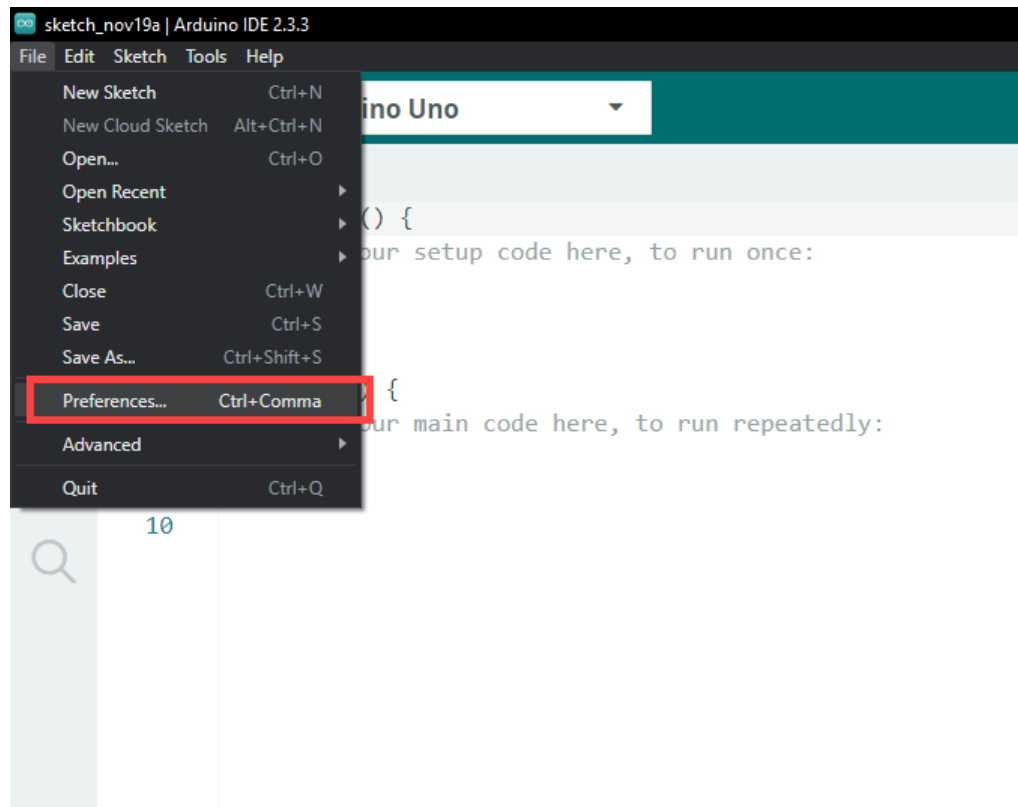
6) ทำการเช็ค Ports COM โดยเข้าไปที่ Control Panel > Device Manager ในขั้นตอนนี้อาจจะต้องเสียบสาย Micro USB เข้ากับอุปกรณ์ระบบถึงจะแสดง Ports COM



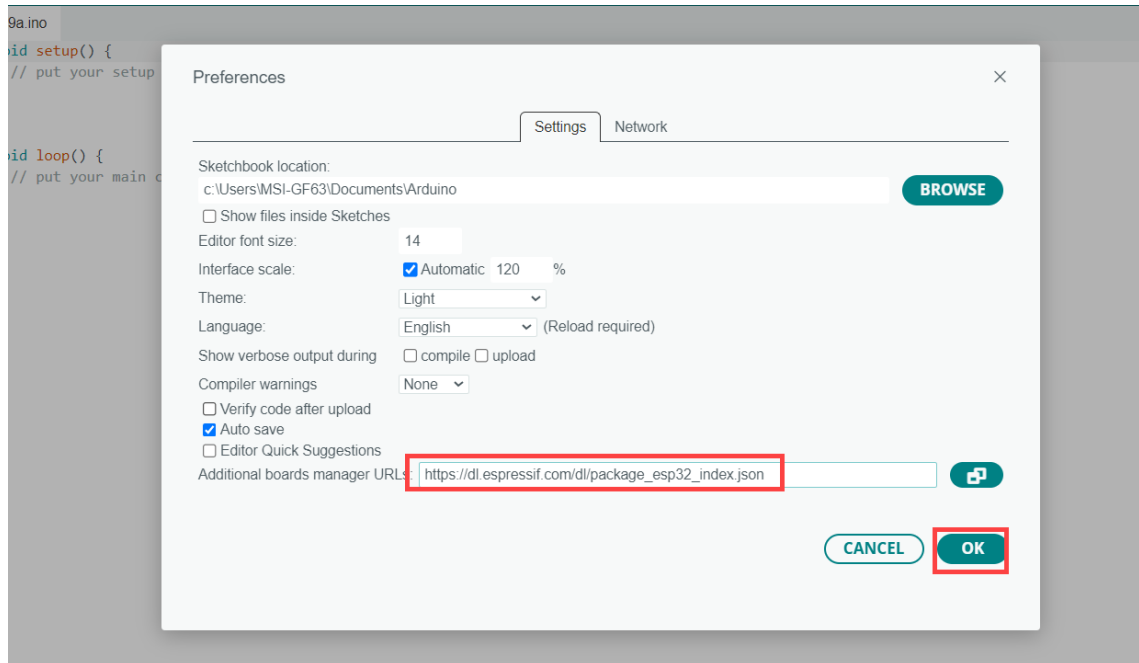
7) เชื่อมการเชื่อมต่อระหว่าง Arduino กับ Ports COM โดยเข้าไปที่เมนู **Tools > Port**



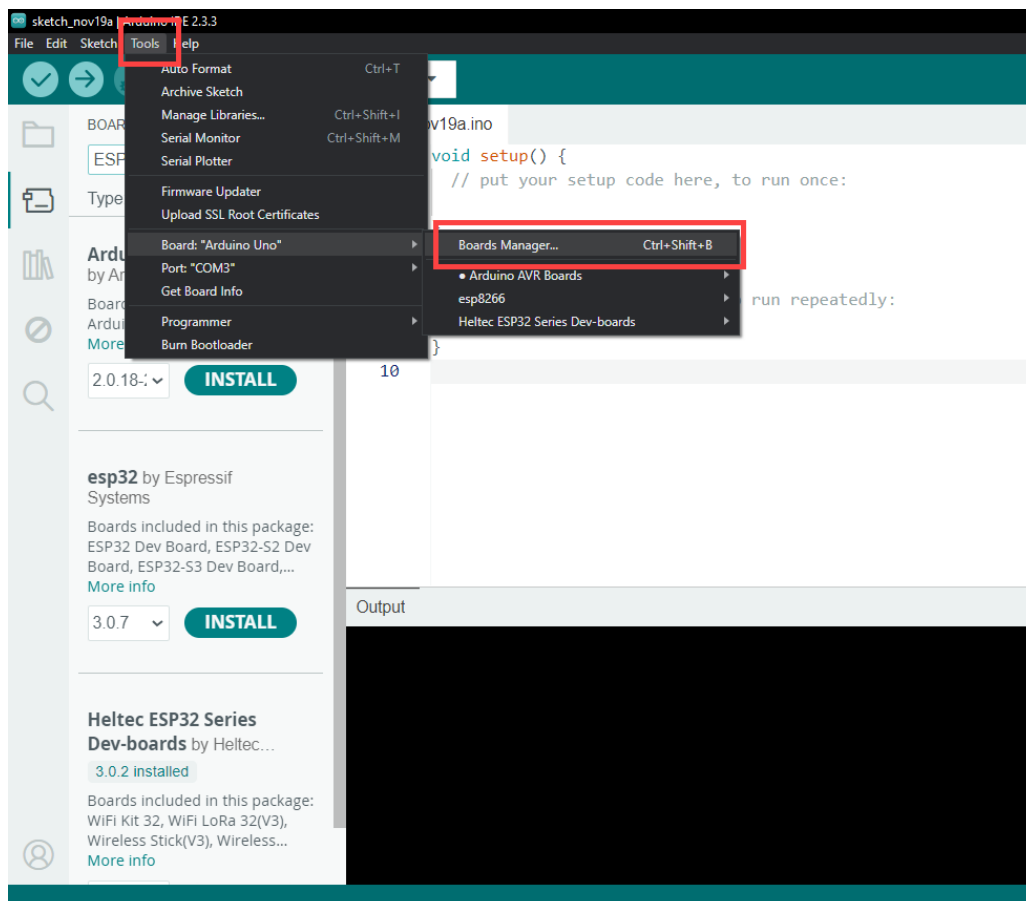
8) ติดตั้ง ESP32 Package โดยเข้าไป Arduino IDE แล้วเลือกเมนู **File > Preferences**



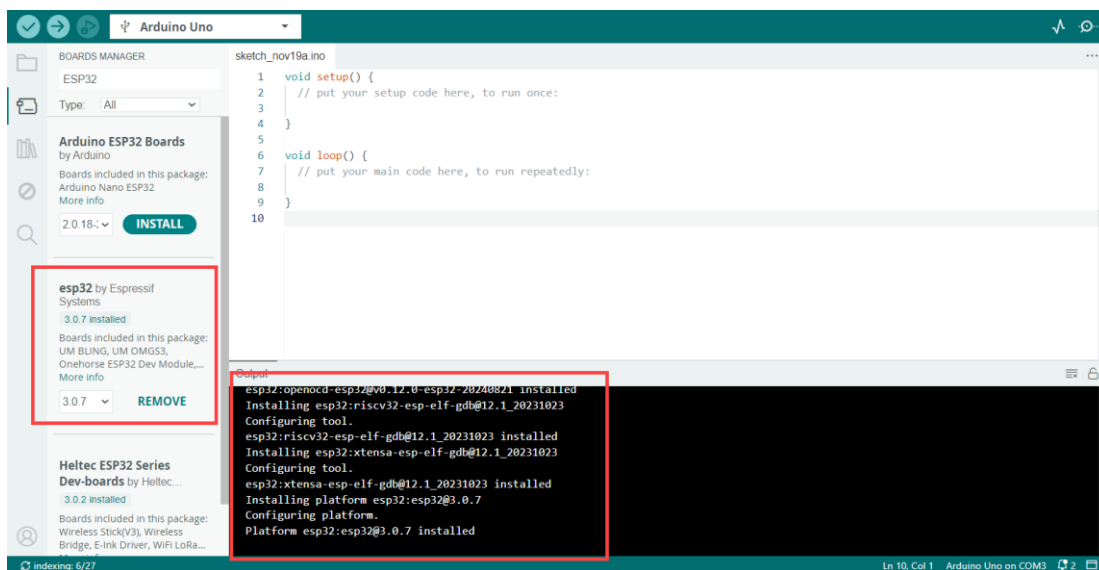
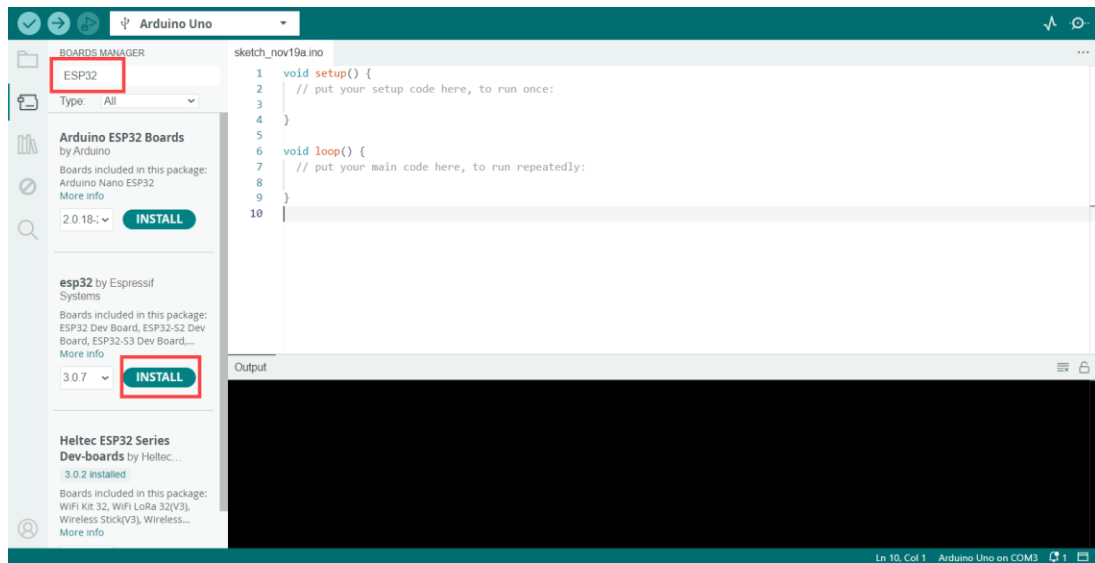
9) เพิ่มลิงค์ https://dl.espressif.com/dl/package_esp32_index.json เข้าไปในช่อง Additional Boards Manager URLs



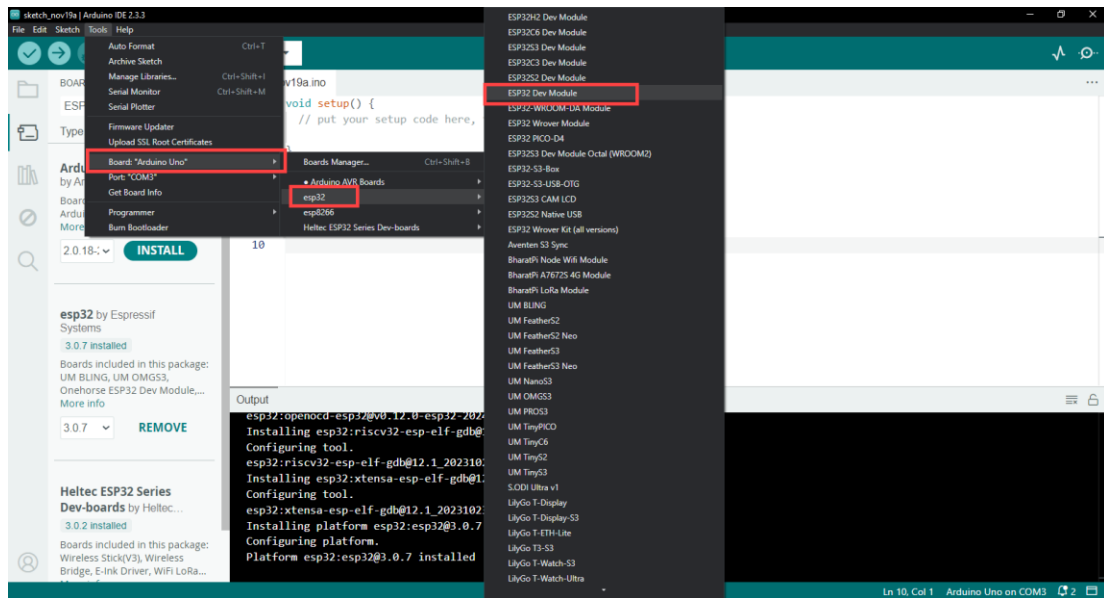
10) เข้าไปติดตั้ง ESP32 โดยเลือกเมนู Tools > Board > Boards Manager



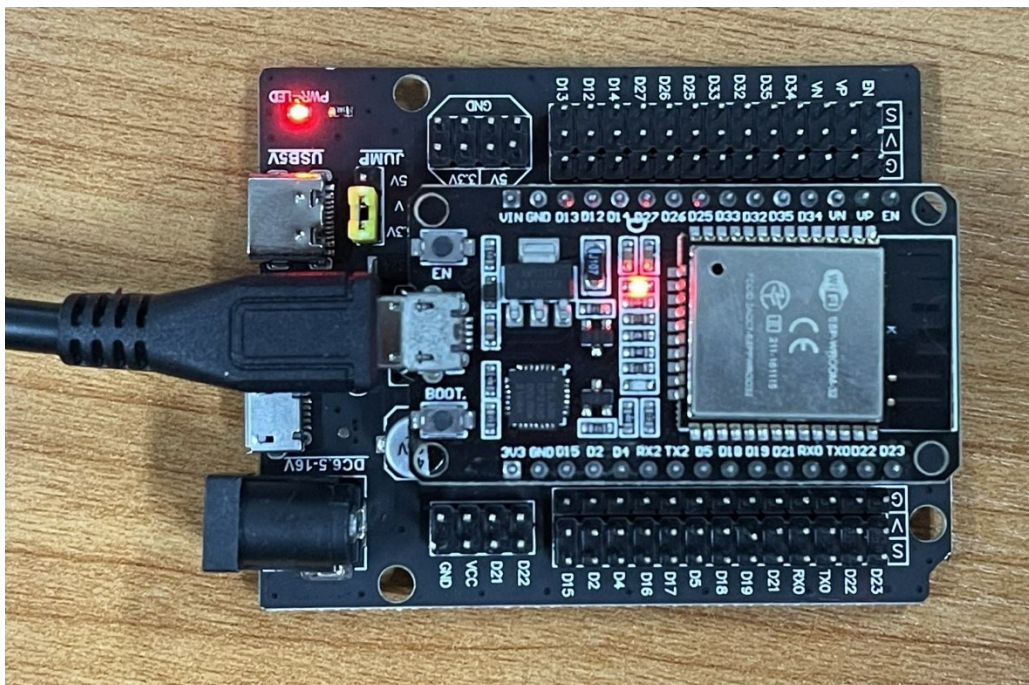
11) กรอก **ESP32** ในช่องค้นหาและทำการติดตั้ง (คลิกปุ่ม Install) รอจนกว่าโปรแกรมจะทำการติดตั้งเสร็จสมบูรณ์

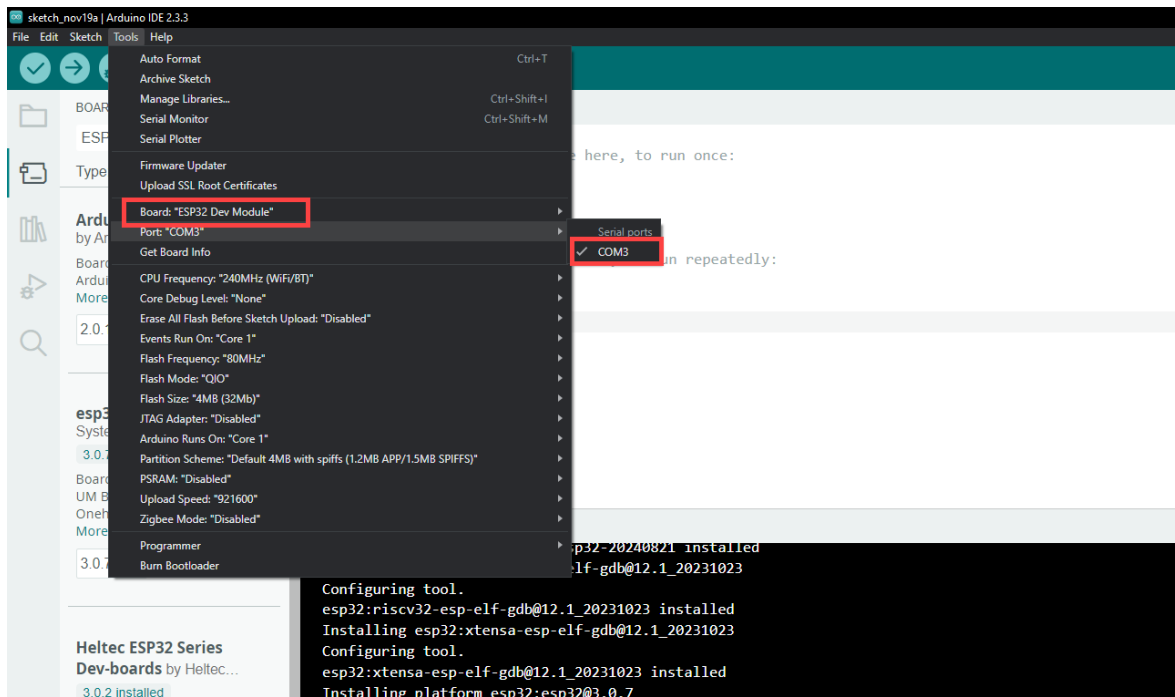


12) เลือกการเชื่อมต่อ ESP32 โดยเลือกเมนู Tools > Board: > ESP32 Arduino > ESP32 Dev Module



13) เสียบสาย USB เข้าคอมพิวเตอร์และบอร์ด ESP32 ตรวจสอบการเลือกบอร์ด และเลือก Port COM เพื่อให้พร้อมเขียนโปรแกรม



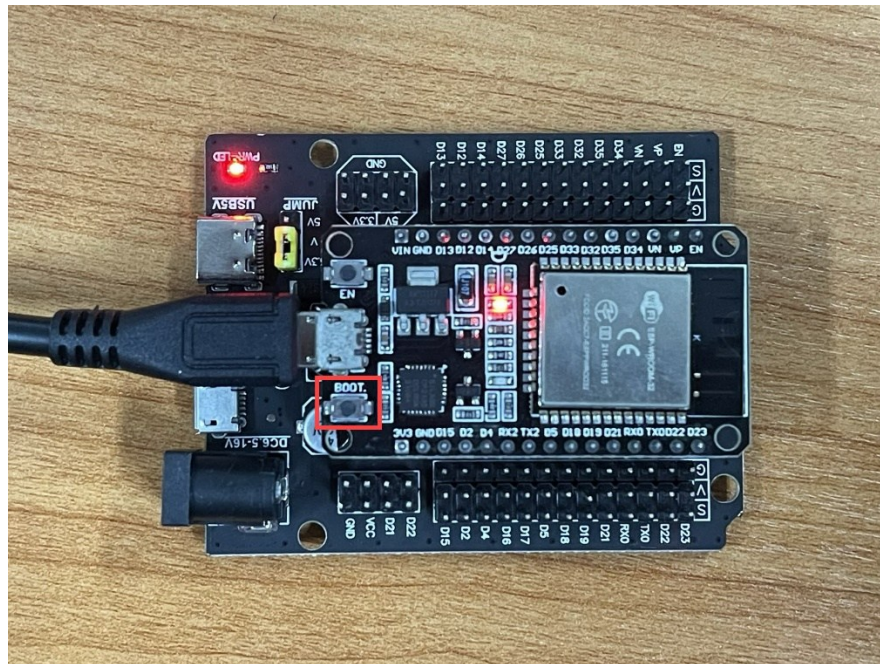
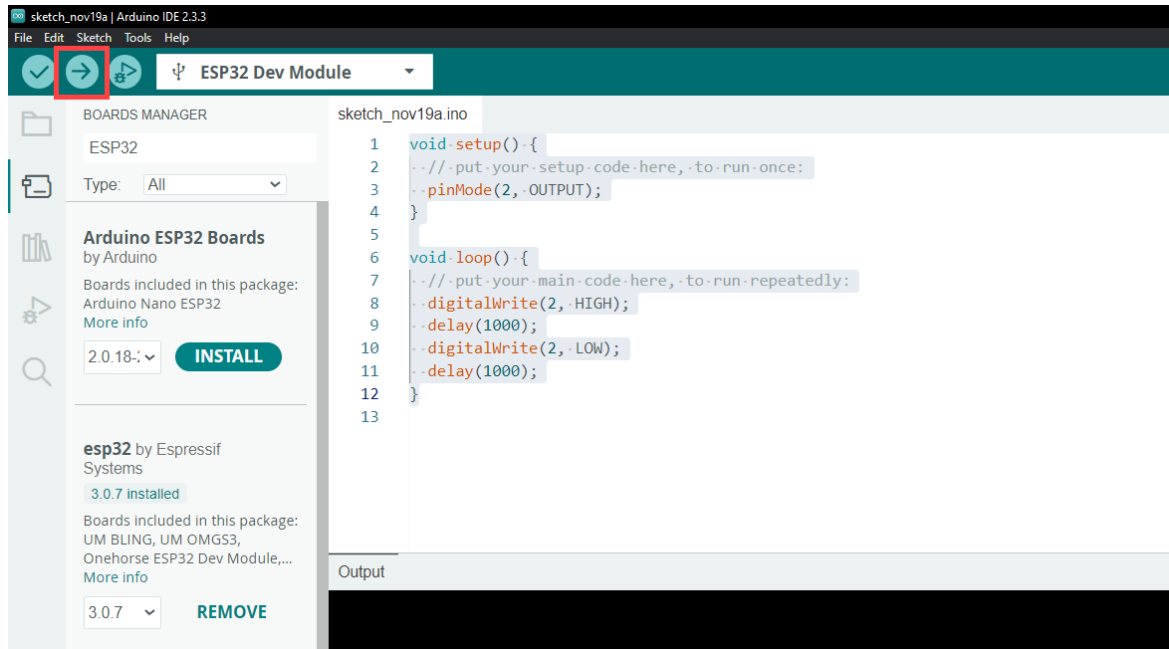


14) ทดสอบการเขียนโปรแกรมแสดงสถานะไฟ LED

```
void setup() {
  pinMode(2, OUTPUT);
}

void loop() {
  digitalWrite(2, HIGH);
  delay(1000);
  digitalWrite(2, LOW);
  delay(1000);
}
```

15) คลิกปุ่มลูกศรเพื่อทำการเขียนคำสั่ง (Upload) ลงในบอร์ดไมโครคอนโทรลเลอร์ ESP32 โดยให้ทำการกดปุ่ม BOOT บน ESP32 ค้างไว้ จนกว่าจะสถานะ Upload ขึ้น จากนั้นรอสถานะ Upload จนเสร็จสิ้น



ตอนที่ 2

ทำความรู้จักกับบอร์ด ESP32

ESP32 เป็นชื่อของไอซีไมโครคอนโทรลเลอร์ที่รองรับการเชื่อมต่อ WiFi และ Bluetooth 4.2 BLE ในตัว ผลิตโดยบริษัท Espressif จากประเทศจีน โดยราคาไม่เกิน 500 บาท (บอร์ดพัฒนาสำเร็จรูป) โดยตัวไอซี ESP32 มีสเปคโดยละเอียด ดังนี้

- ซีพียูใช้สถาปัตยกรรม Tensilica LX6 แบบ 2 แกนสมอง สัญญาณนาฬิกา 240MHz มีแรมในตัว 512KB
- รองรับการเชื่อมต่อรวมภายนอกสูงสุด 16MB
- มาพร้อมกับ WiFi มาตรฐาน 802.11 b/g/n รองรับการใช้งานทั้งในโหมด Station softAP และ Wi-Fi direct
- มีบลูทูธในตัว รองรับการใช้งานในโหมด 2.0 และโหมด 4.0 BLE
- ใช้แรงดันไฟฟ้าในการทำงาน 2.6V ถึง 3V
- ทำงานได้ที่อุณหภูมิ -40°C ถึง 125°C

นอกจากนี้ ESP32 ยังมีเซ็นเซอร์ต่าง ๆ มาในตัวด้วย ดังนี้

- วงจรรองสัญญาณรบกวนในวงจรขยายสัญญาณ
- เซ็นเซอร์แม่เหล็ก
- เซ็นเซอร์สัมผัส (Capacitive touch) รองรับ 10 ช่อง
- รองรับการเชื่อมต่อคลิสตอล 32.768kHz สำหรับใช้กับส่วนวงจรนับเวลาโดยเฉพาะ

การใช้งานต่าง ๆ ของ ESP32 รองรับการทำงานต่อบัสต่าง ๆ ดังนี้

- มี GPIO จำนวน 32 ช่อง
- รองรับ UART จำนวน 3 ช่อง
- รองรับ SPI จำนวน 3 ช่อง
- รองรับ I2C จำนวน 2 ช่อง
- รองรับ ADC จำนวน 12 ช่อง
- รองรับ DAC จำนวน 2 ช่อง
- รองรับ I2S จำนวน 2 ช่อง
- รองรับ PWM / Timer ทุกช่อง
- รองรับการทำงานต่อกับ SD-Card

นอกจากนี้ ESP32 ยังรองรับฟังก์ชันเกี่ยวกับความปลอดภัยต่าง ๆ ดังนี้

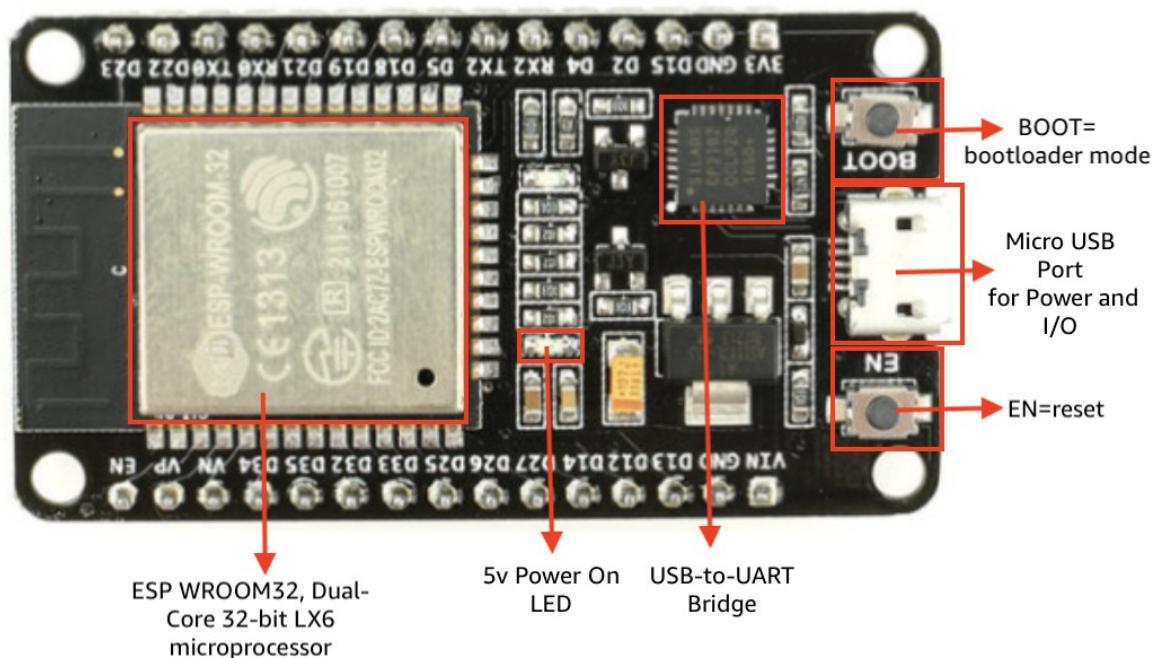
- รองรับการทำงานเข้ารหัส WiFi แบบ WEP และ WPA/WPA2 PSK/Enterprise
- มีวงจรเข้ารหัส AES / SHA2 / Elliptical Curve Cryptography / RSA-4096 ในตัว

ในด้านประสิทธิภาพการใช้งาน ตัว ESP32 สามารถทำงานได้ดี โดย

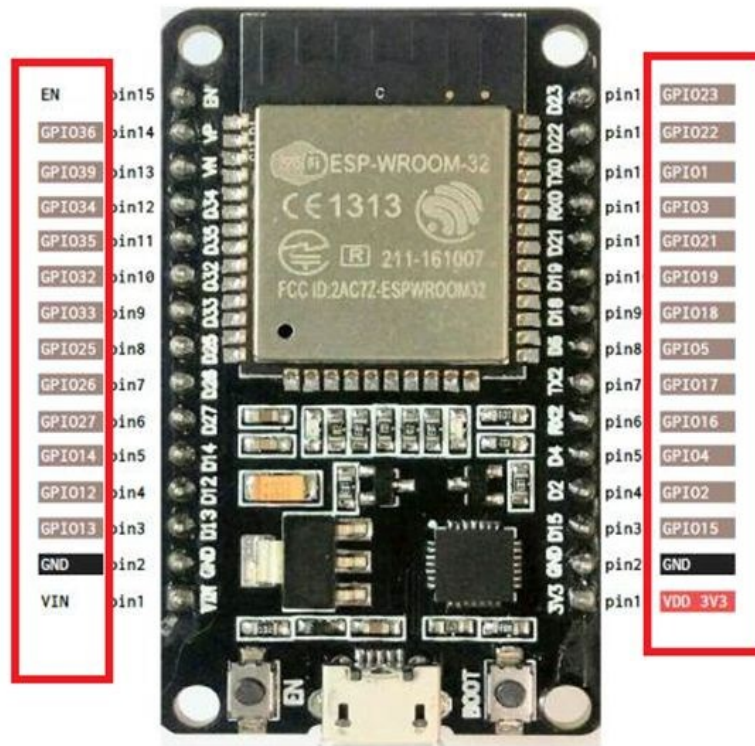
- รับ - ส่ง ข้อมูลได้ความเร็วสูงสุดที่ 150Mbps เมื่อเชื่อมต่อแบบ 11n HT40 ได้ความเร็วสูงสุด 72Mbps เมื่อเชื่อมต่อแบบ 11n HT20 ได้ความเร็วสูงสุดที่ 54Mbps เมื่อเชื่อมต่อแบบ 11g และได้ความเร็วสูงสุดที่ 11Mbps เมื่อเชื่อมต่อแบบ 11b
- เมื่อใช้การเชื่อมต่อผ่านโปรโตคอล UDP จะสามารถรับ - ส่งข้อมูลได้ด้วยความเร็ว 135Mbps
- ในโหมด Sleep ใช้กระแสไฟฟ้าเพียง 2.5uA

จะเห็นว่า ในราคาไม่ถึง 500 บาท (บอร์ดพัฒนาสำเร็จรูป) และโมดูลเปล่าราคาไม่ถึง 400 บาท สามารถให้ประสิทธิภาพได้เกินราคา ด้วยเหตุนี้ ESP32 จึงเหมาะสำหรับนำมาใช้งานมาก ด้วยเหตุผลทางด้านราคา และประสิทธิภาพที่ได้

แสดงประกอบต่างในบอร์ด ESP32



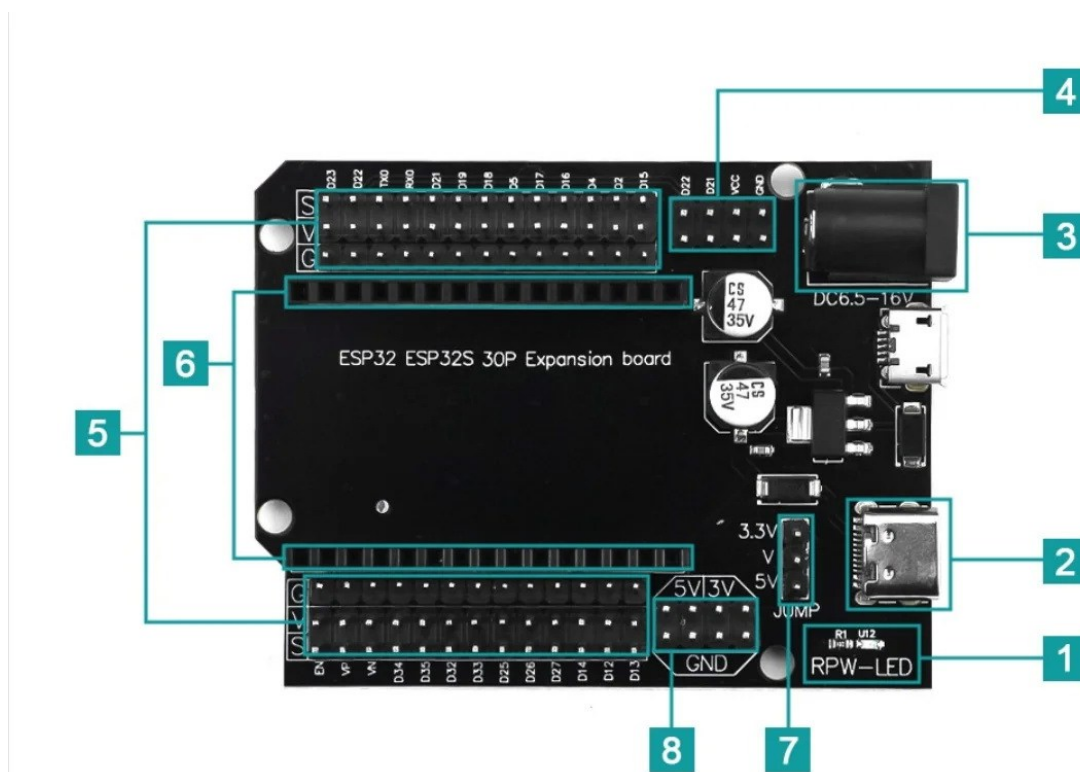
แสดงขาสัญญาณ I/O เป็นส่วนควบคุมการทำงานอุปกรณ์ต่าง ๆ



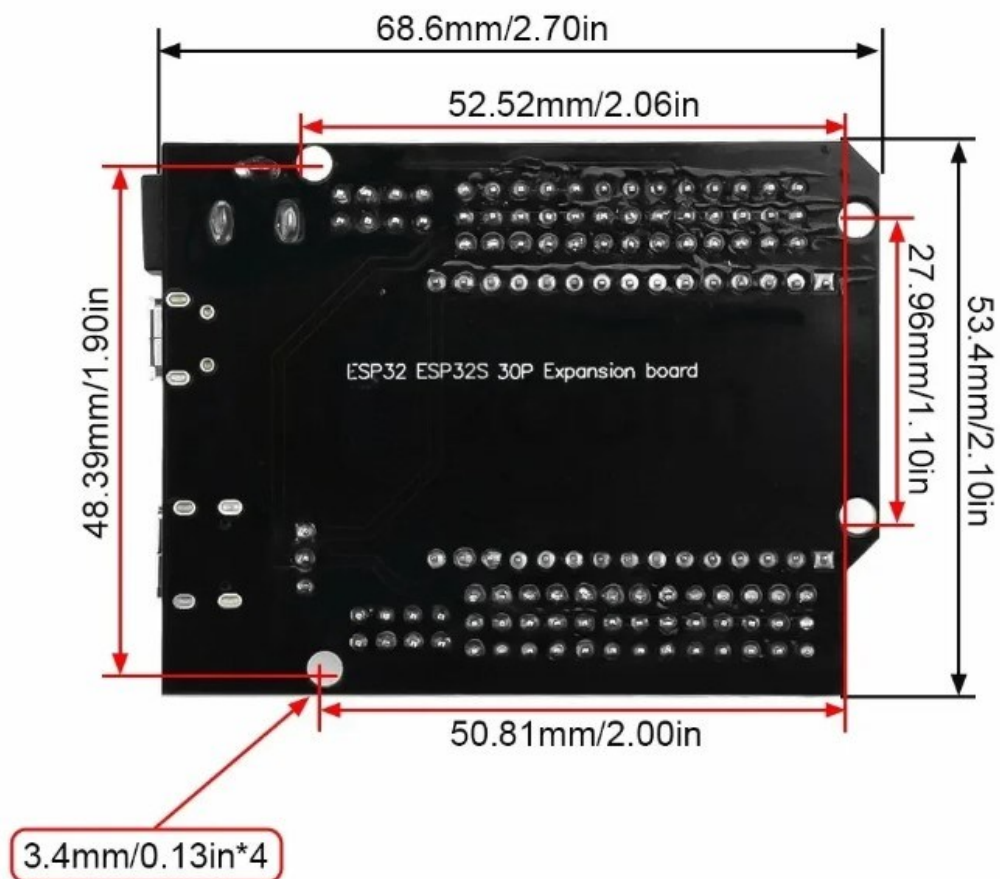
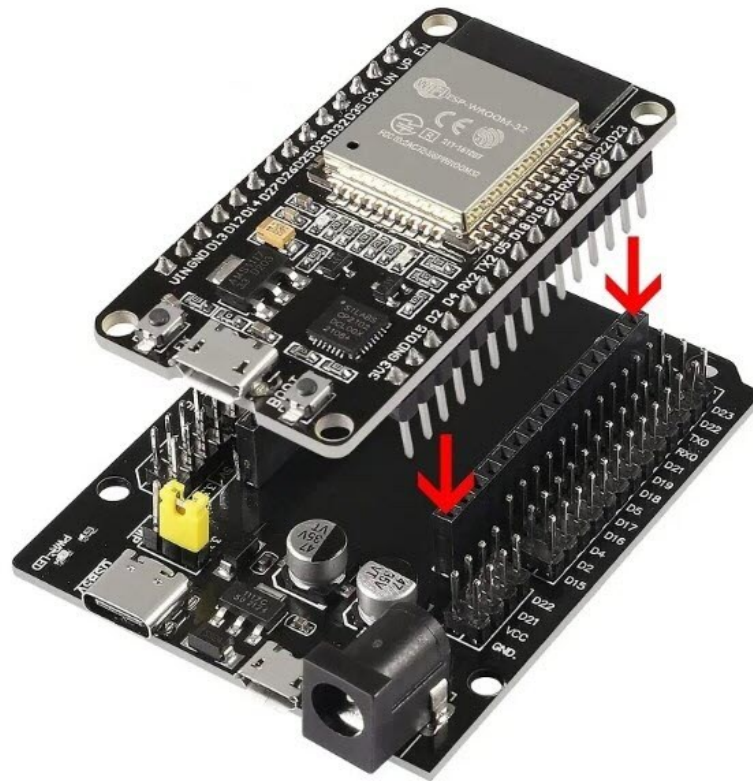
ตอนที่ 3

การใช้งานบอร์ดขยายขา ESP32

บอร์ดขยายขา ESP32 (ESP32 Expansion Board) เป็นอุปกรณ์เสริมที่ออกแบบมาเพื่อเพิ่มความสะดวกในการใช้งานไมโครคอนโทรลเลอร์ ESP32 ในตัว Expansion Board ทำหน้าที่เสริมความสามารถและช่วยให้การพัฒนาโปรเจกต์เป็นไปได้อย่างง่ายและรวดเร็วยิ่งขึ้น



1. ไฟ LED Power 1 ดวง
2. ช่องต่อไฟเลี้ยง 5V แบบ USB Type-C และ Micro USB
3. ช่องต่อไฟเลี้ยง 6.5-16V แบบ DC Jack 5.5x2.1mm
4. ช่องต่อสัญญาณแบบ I2C 2 ช่อง
5. อินเตอร์เฟซขยายขาของบอร์ด ESP32 แบบ GVS
6. ช่องต่อกับบอร์ด ESP32
7. ช่องไฟออก 5V / 3.3V และกราวด์
8. ช่องไฟออก 5V / 3.3V และกราวด์



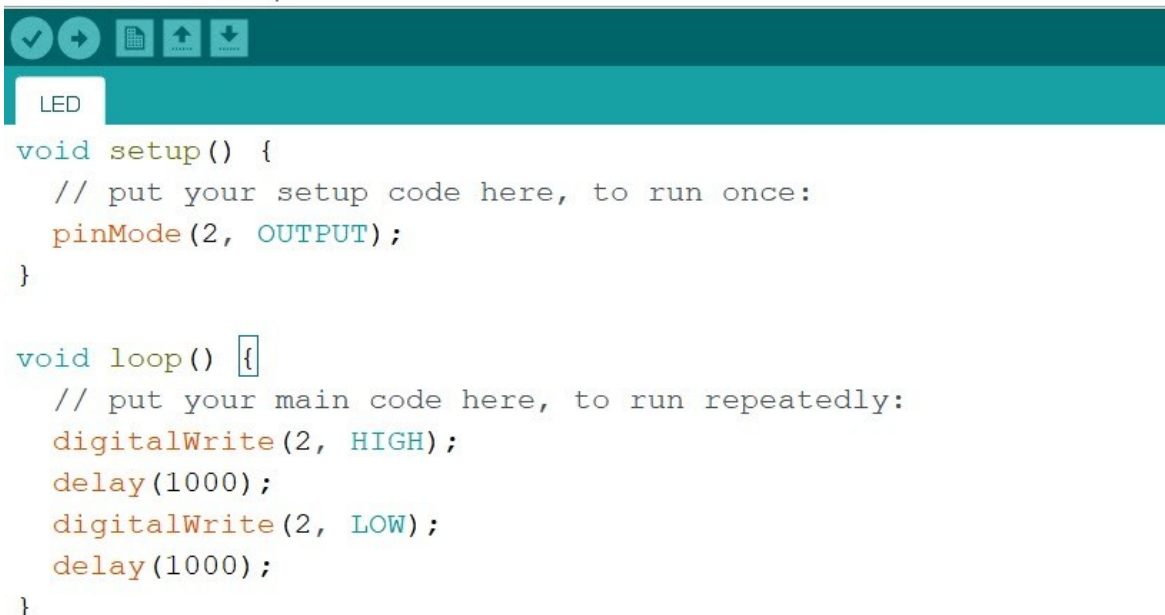
ตอนที่ 4

ทำความเข้าใจเริ่มต้นเกี่ยวกับการเขียนโปรแกรม Arduino

โปรแกรมของ Arduino แบ่งได้ เป็นสองส่วนคือ ภาษาซีของ Arduino จะจัดรูปแบบโครงสร้างของการเขียนโปรแกรมออกเป็นส่วนย่อยๆ หลายๆ ส่วน โดยเรียกแต่ละส่วนว่า ฟังก์ชัน และ เมื่อนำฟังก์ชัน มารวมเข้าด้วยกัน ก็จะเรียกว่าโปรแกรม โดยโครงสร้างการเขียนโปรแกรมของ Arduino นั้น ทุกๆโปรแกรมจะต้องประกอบไปด้วยฟังก์ชันจำนวนเท่าใดก็ได้ แต่อย่างน้อยที่สุดต้องมีฟังก์ชัน จำนวน 2 ฟังก์ชัน คือ setup() และ loop()

LED | Arduino 1.8.15

File Edit Sketch Tools Help



```
void setup() {
  // put your setup code here, to run once:
  pinMode(2, OUTPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(2, HIGH);
  delay(1000);
  digitalWrite(2, LOW);
  delay(1000);
}
```

- ฟังก์ชัน **Setup** คือฟังก์ชันหลักสำหรับการทำงาน จะทำงานเพียง 1 ครั้งหลังจากได้รับโปรแกรมและจะทำการไปทำงานในฟังก์ชันถัดไป คือ ฟังก์ชัน Loop
- ฟังก์ชัน Loop คือฟังก์ชันการทำงานของฟังก์ชัน Loop จะทำงานวนซ้ำไปเรื่อยๆ เรียกว่า infinite loop การทำงานที่ไม่มีสิ้นสุด

วิธีใช้งาน Arduino สื่อสารแบบอนุกรม เพื่อรับและแสดงผลข้อมูลระหว่าง Arduino กับเครื่องคอมพิวเตอร์

การติดต่อบอร์ด Arduino กับเครื่องคอมพิวเตอร์ สามารถทำได้ทางมาตรฐานสื่อสารแบบ Serial ใน Arduino จะใช้ 2 ขา คือ rx,tx ผ่านวงจร usb ttl เพื่อสื่อสารกับเครื่องคอมพิวเตอร์ ทำให้เราสามารถส่งข้อความจากบอร์ด ESP32 ออกมาแสดงผลทางหน้าจอ ที่เมนู Serial Monitor ใน Arduino IDE และสามารถรับค่าจาก keyboard หรือจากในเครื่องคอมพิวเตอร์ส่งมาเป็นอินพุตให้กับบอร์ด ESP32 ได้ด้วยเช่นกัน

โดยการสื่อสารแบบนี้จะมีการกำหนดความเร็วในการรับส่ง ซึ่งจะต้องมีความเร็วที่ตรงกันทั้ง 2 ฝ่ายจึงจะสามารถติดต่อกันได้ถูกต้อง การกำหนดความเร็วในการส่งข้อมูลเราเรียกว่า Baudrate

โดยทั่วไปจะกำหนดความเร็วในการติดต่อดัง เช่น

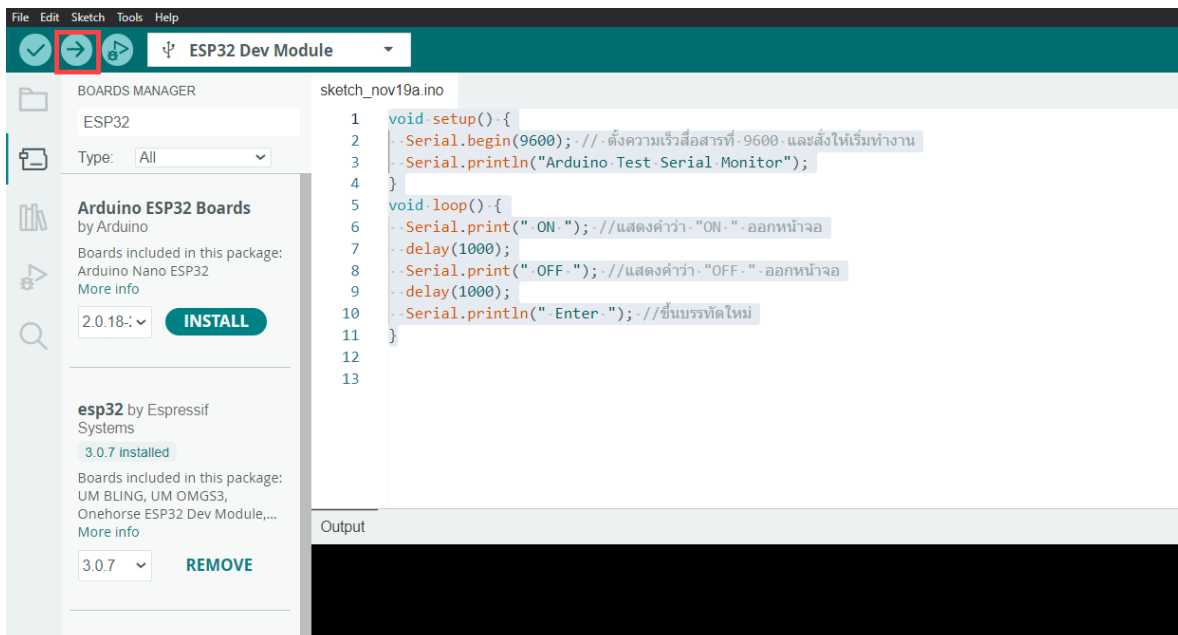
300 , 1200 , 2400 , 4800 , 9600 , 14400 , 38400 , 57600 , 115200 , 230400 , 460800 , 921600 ทั้งนี้ขึ้นอยู่กับอุปกรณ์ว่ารองรับการสื่อสารได้ที่ความเร็วไหนได้บ้าง

การเริ่มติดต่อทำได้โดยคำสั่ง Serial.begin(9600);

ตัวเลข 9600 คือการกำหนดว่าจะใช้ความเร็วที่ 9600 ซึ่งสามารถเปลี่ยนเป็นค่าอื่นได้ตามค่ามาตรฐานด้านบน ยิ่งค่าสูงก็จะส่งข้อมูลได้รวดเร็วขึ้น

```
void setup() {
  Serial.begin(9600);
  Serial.println("Arduino Test Serial Monitor");
}
void loop() {
  Serial.print(" ON ");
  delay(1000);
  Serial.print(" OFF ");
  delay(1000);
  Serial.println(" Enter ");
}
```

คลิกปุ่มลูกศรเพื่อทำการเขียนคำสั่ง (Upload) และให้เปิด Serial Monitor ขึ้นมาแล้วกำหนดอัตราการส่งข้อมูลให้ตรงกับโปรแกรมที่อัปโหลดลง ESP32



The screenshot shows the Arduino IDE interface with the following components:

- Boards Manager:** The 'ESP32' board is selected. The 'Arduino ESP32 Boards' package is installed (version 2.0.18). The 'esp32' package by Espressif Systems is also installed (version 3.0.7).
- Sketch Editor:** The sketch 'sketch_nov19a.ino' is open. It contains the following code:

```
1 void setup() {  
2   Serial.begin(9600); // ตั้งค่าความเร็วสื่อสารที่ 9600 และส่งให้เริ่มทำงาน  
3   Serial.println("Arduino-Test-Serial-Monitor");  
4 }  
5 void loop() {  
6   Serial.print(" ON "); // แสดงคำว่า "ON" - ออกหน้าจอ  
7   delay(1000);  
8   Serial.print(" OFF "); // แสดงคำว่า "OFF" - ออกหน้าจอ  
9   delay(1000);  
10  Serial.println(" Enter "); // ขึ้นบรรทัดใหม่  
11 }  
12  
13
```
- Serial Monitor:** The Serial Monitor is open, showing the output of the sketch. The output is:

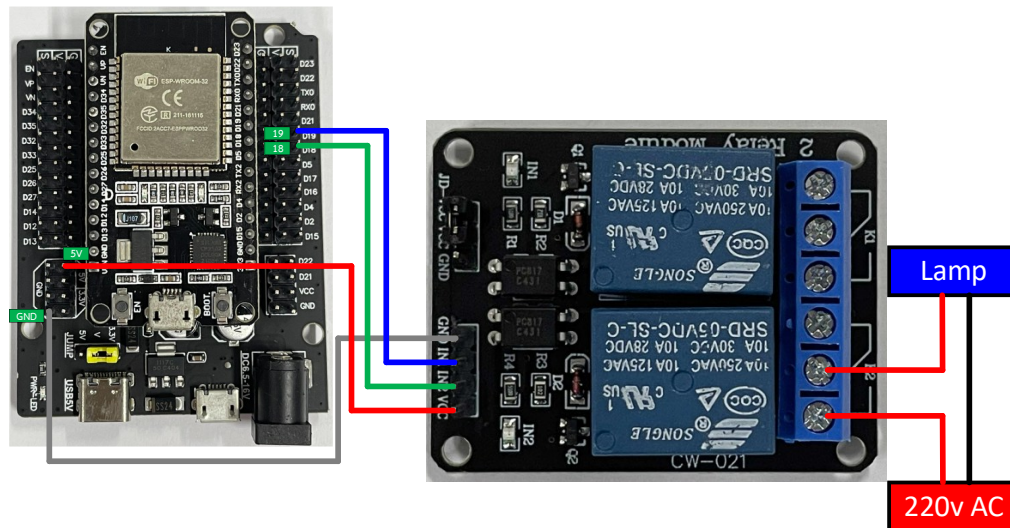
```
Message (Enter to send message to 'ESP32 Dev Module' on 'COM3')  
08:45:15.033 -> OFF Enter  
08:45:16.074 -> ON OFF Enter  
08:45:18.084 -> ON OFF Enter  
08:45:20.043 -> ON OFF Enter  
08:45:22.065 -> ON OFF
```

ตอนที่ 5

การเขียนโปรแกรมควบคุมอุปกรณ์ด้วย ESP32

การเขียนโปรแกรมควบคุมโมดูลรีเลย์ (Relay)

- 1) ทำการเชื่อมต่อวงจรโมดูลควบคุมโมดูลรีเลย์ ดังต่อไปนี้



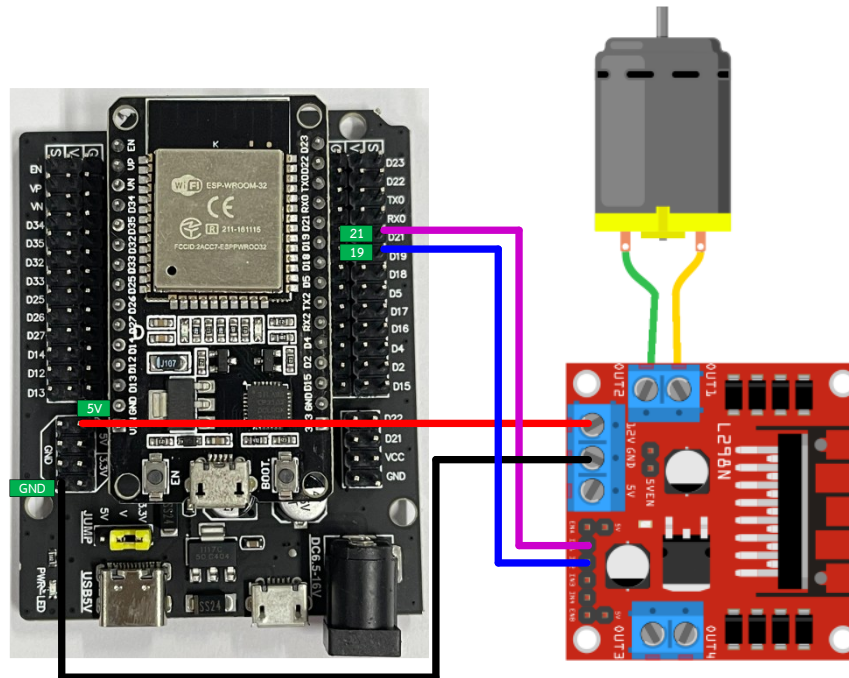
- 2) เขียนโค้ดต่อไปนี้ลงใน Arduino IDE

```
int pin1 = 19;
int pin2 = 18;
void setup() {
  pinMode(pin1, OUTPUT);
  pinMode(pin2, OUTPUT);}
void loop() {
  digitalWrite(pin1, HIGH);
  digitalWrite(pin2, HIGH);
  delay(2000);
  digitalWrite(pin1, LOW);
  digitalWrite(pin2, LOW);
  delay(1000);
}
```

- 3) จากนั้นให้ทำ Save และรันอัปโหลดโปรแกรมลงใน ESP32 สังเกตผลการทำงาน

การเขียนโปรแกรมควบคุมการทำงานของมอเตอร์

- 1) ทำการเชื่อมต่อวงจรควบคุมการทำงานของมอเตอร์ ดังต่อไปนี้



- 2) เขียนโค้ดต่อไปนี้ลงใน Arduino IDE

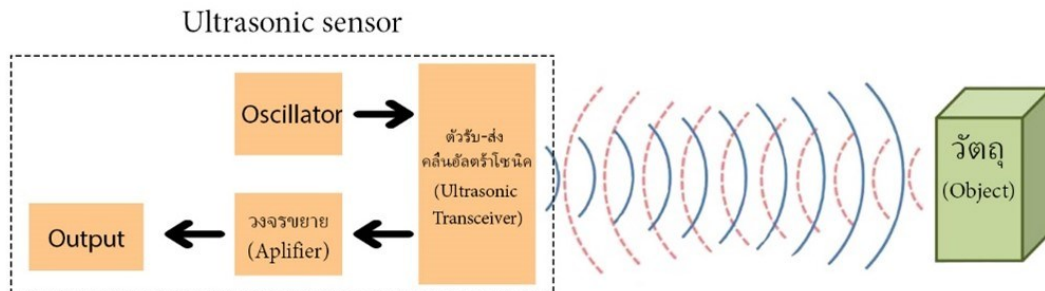
```
int dir1PinA = 21;
int dir2PinA = 19;
void setup() {
  Serial.begin(9600);
  pinMode(dir1PinA,OUTPUT);
  pinMode(dir2PinA,OUTPUT);
}
void loop() {
  digitalWrite(dir1PinA, LOW);
  digitalWrite(dir2PinA, HIGH);
  Serial.println("Motor 1 Forward");
  delay(4000);
  digitalWrite(dir1PinA, HIGH);
  digitalWrite(dir2PinA, HIGH);
  Serial.println("Motor 1 brake");
  delay(2000);
  digitalWrite(dir1PinA, HIGH);
  digitalWrite(dir2PinA, LOW);
  Serial.println("Motor 1 Back");
  delay(4000);
  digitalWrite(dir1PinA, LOW);
  digitalWrite(dir2PinA, LOW);
  Serial.println("Motor 1 brake");
  delay(2000);
}
```

- 3) จากนั้นให้ทำ Save และรันอัปโหลดโปรแกรมลงใน ESP32 สังเกตผลการทำงาน

การเขียนโปรแกรมอ่านค่าจากโมดูลวัดระยะทาง (Ultrasonic)

1) หลักการของ Ultrasonic

Ultrasonic sensor คือ อุปกรณ์สำหรับวัดระดับหรือระยะทางชนิดหนึ่งที่ใช้คลื่น Ultrasonic ซึ่งอาศัยหลักการสะท้อนของคลื่นความถี่สูง Ultrasonic โดยอุปกรณ์จะปล่อยคลื่น Ultrasonic ให้กระทบกับวัตถุ จากนั้นรอคลื่น Ultrasonic สะท้อนกลับมาที่เซ็นเซอร์เพื่อคำนวณหาระยะทางที่วัดได้



Ultrasonic sensor สามารถคำนวณหาระยะของคลื่น Ultrasonic จะเป็นไปตามสูตรการเคลื่อนที่แนวราบดังนี้

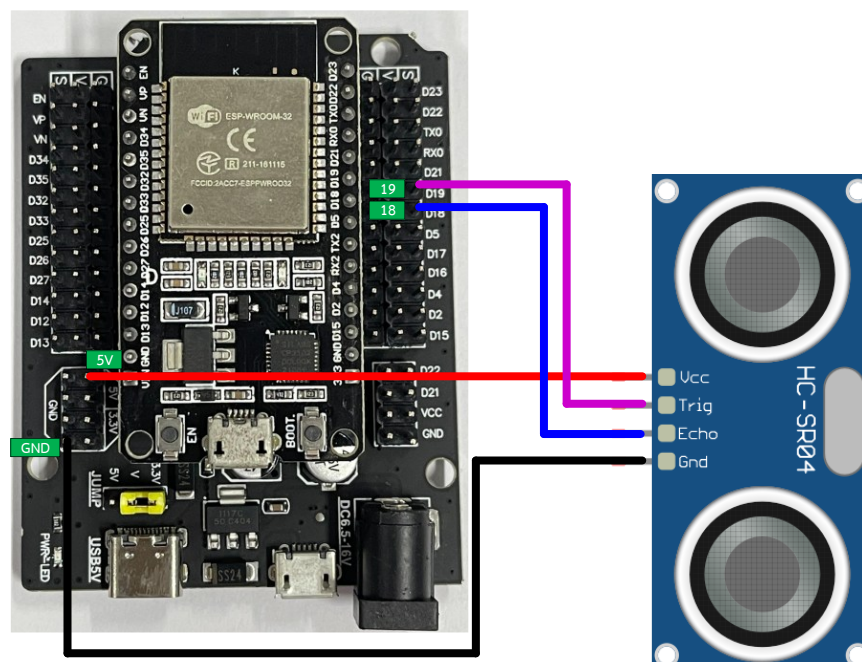
$$S = V_{\text{เสียง}} \left(\frac{t}{2} \right)$$

โดยที่ S = ระยะทาง (m)

$V_{\text{เสียง}}$ = ความเร็วเสียง (m/s)

t = เวลาในการเดินทางของคลื่น Ultrasonic ทั้งขาไป-ขากลับ (s)

1) ทำการเชื่อมต่อวงจรอ่านค่าจากโมดูลวัดระยะทาง (Ultrasonic) ดังต่อไปนี้



2) เขียนโค้ดต่อไปนีลงใน Arduino IDE

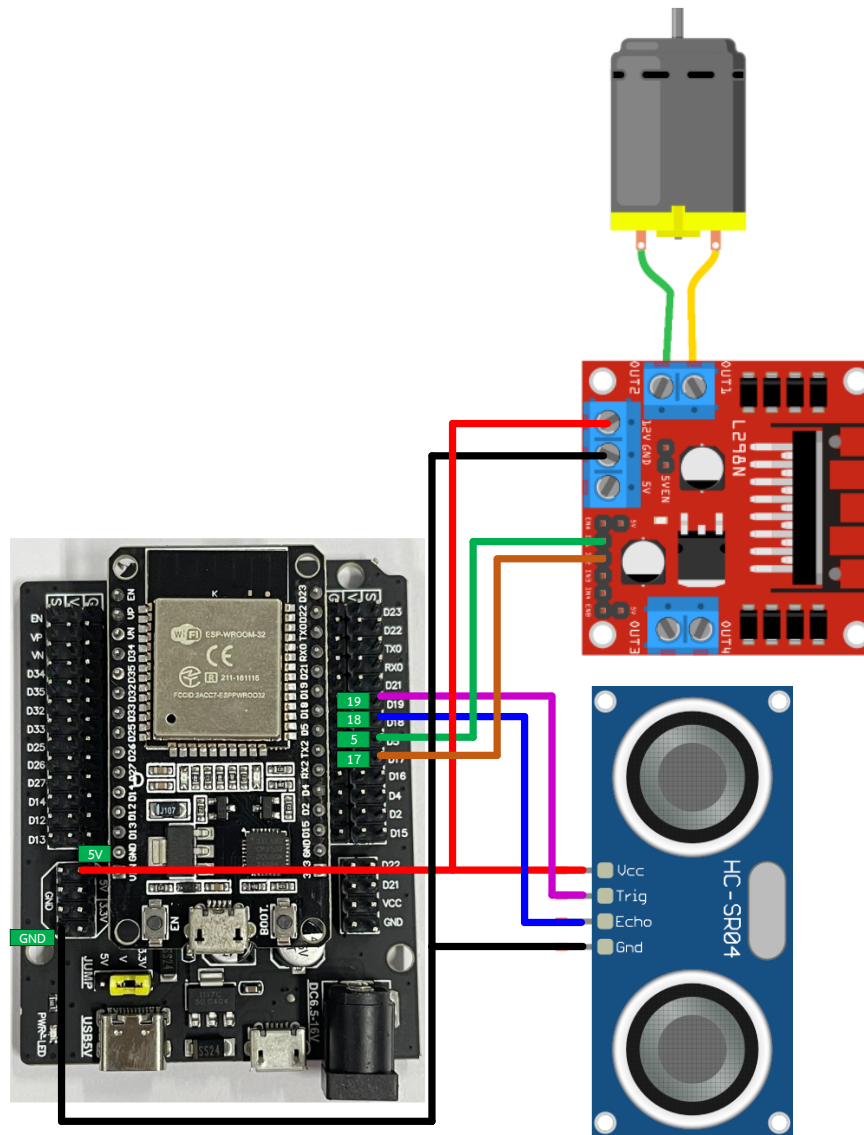
```
int trigPin = 19;
int echoPin = 18;
long duration;
long distance;
unsigned long previousTime = 0;
void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  Serial.begin(9600);
}
void loop() {
  if((millis() - previousTime) >= 1000){
    previousTime = millis();
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    duration = pulseIn(echoPin, HIGH);
    distance= duration*0.034/2;
    Serial.print("Distance: ");
    Serial.println(distance);
  }
}
```

3) จากนั้นให้ทำ Save และรันอัฟโหลดโปรแกรมลงใน ESP32 สังเกตผลการทำงาน

แบบฝึกหัด

ทำการควบคุมการทำงานของมอเตอร์ เมื่อมีสิ่งกีดขวางเซ็นเซอร์ โดยให้ทำการเขียนโปรแกรม มีเงื่อนไขดังนี้
 ถ้า **ไม่มี**สิ่งกีดขวางเซ็นเซอร์ ในระยะ 30 cm ให้มอเตอร์**หมุนไปข้างหน้า**
 แต่ถ้า **มี**สิ่งกีดขวางเซ็นเซอร์ **น้อยกว่าเท่ากับ 30 cm** ให้มอเตอร์**หมุนถอยหลัง**

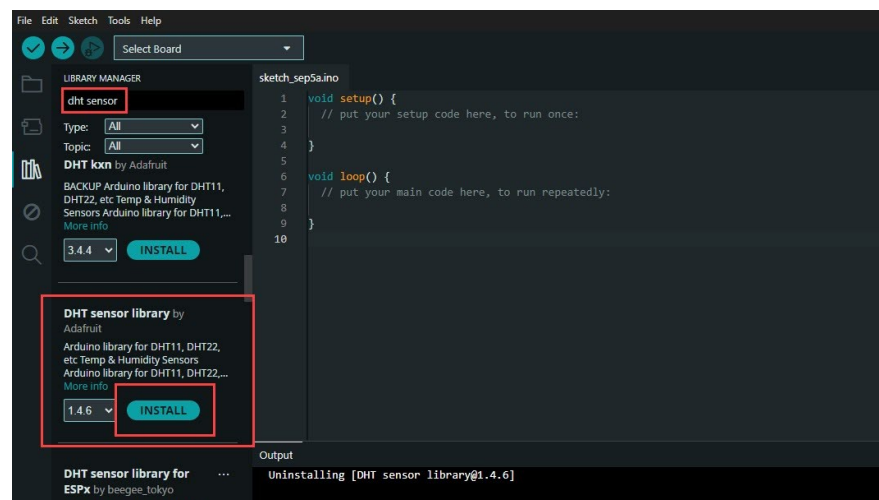
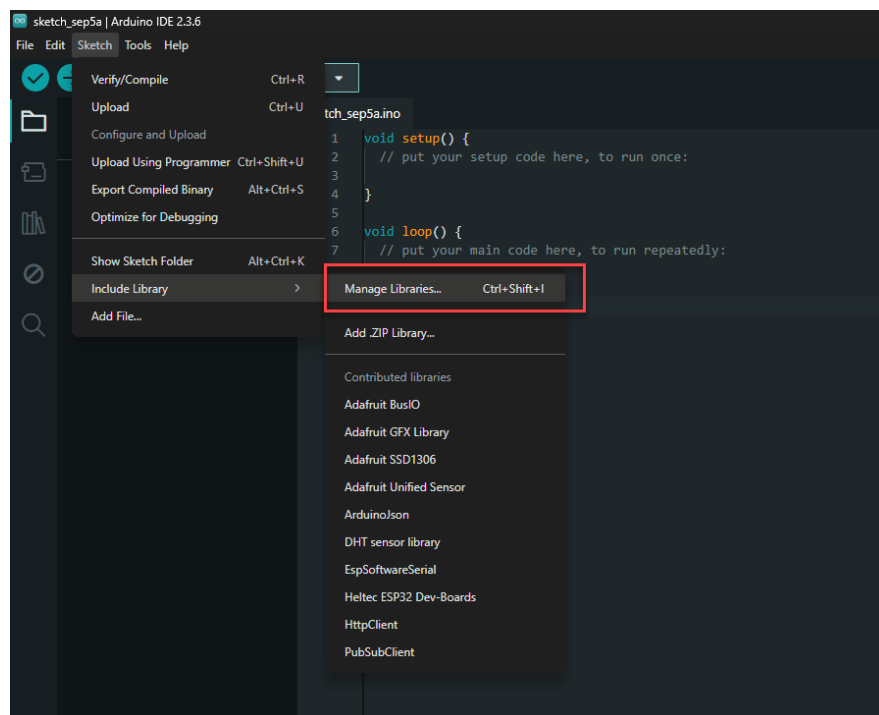
แสดงรูปวงจร

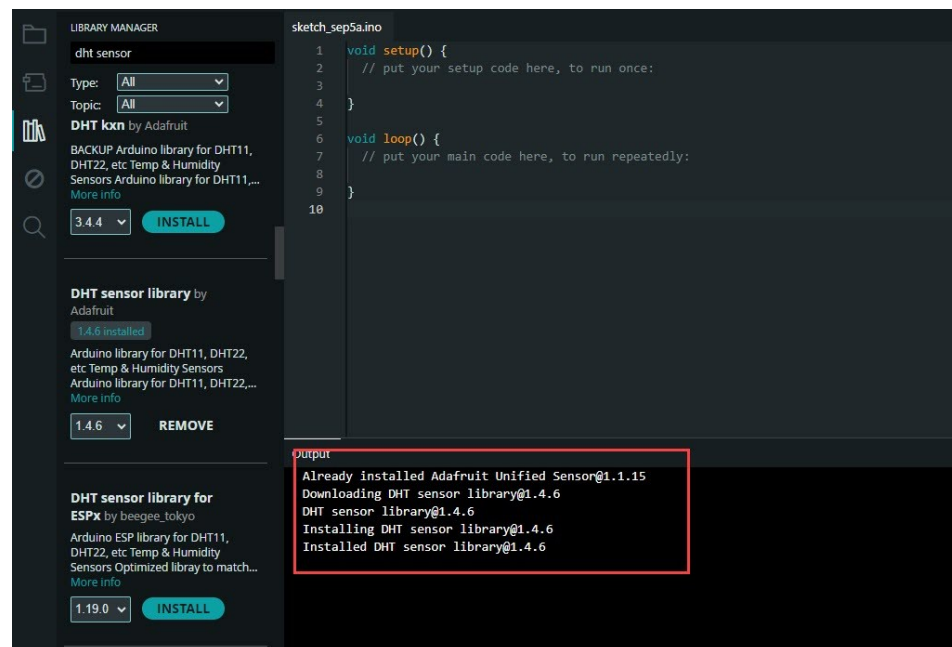
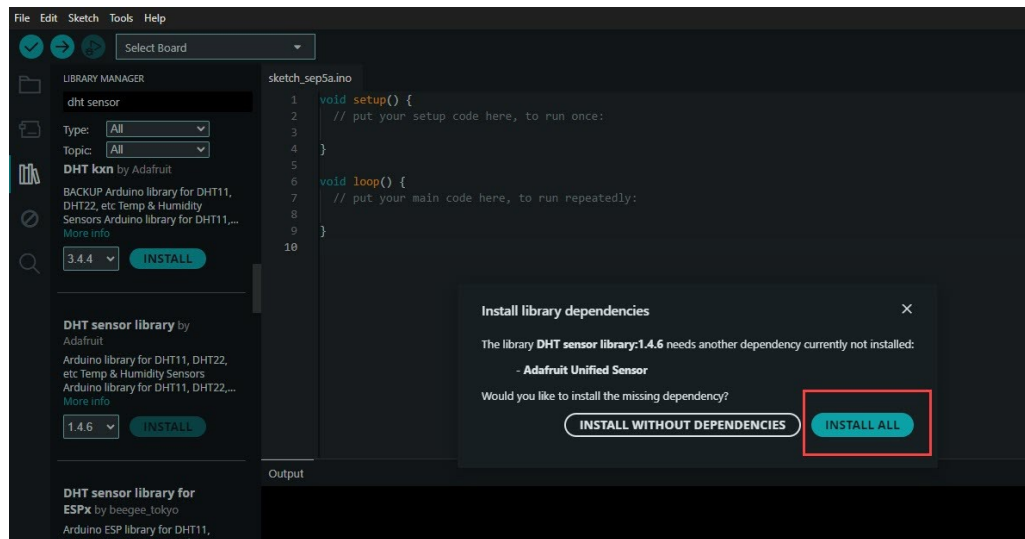


การเขียนโปรแกรมอ่านค่าจากโมดูลวัดอุณหภูมิและความชื้น (DHT11)

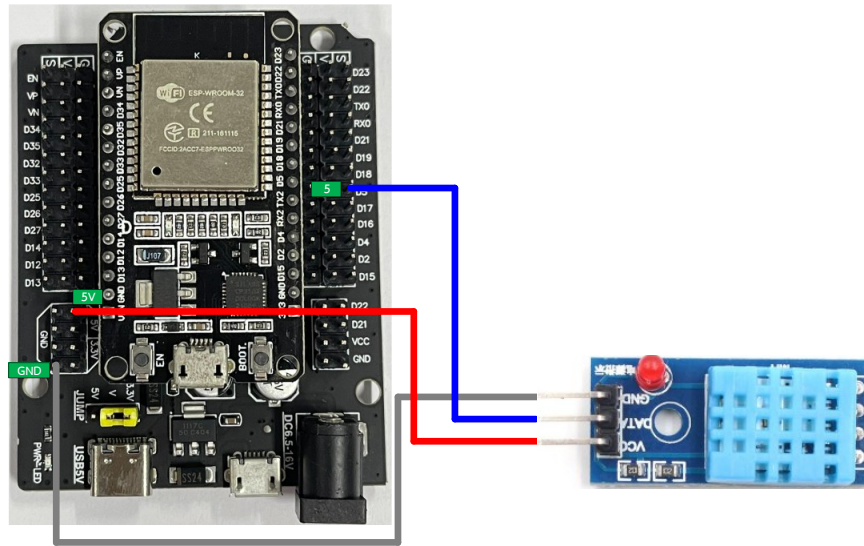
1) DHT11 คือ เป็นโมดูลที่สามารถวัดอุณหภูมิและความชื้นบริเวณรอบๆทั่วไปหรือในห้อง หรือประยุกต์ใช้งานอื่นเช่น Testing, Inspection Equipment, Automatic Control, Data Logger, Weather Station, Humidity Regulator ขึ้นอยู่กับการเขียนโปรแกรมและการต่อใช้งานภายนอก ผ่านการสอบเทียบตามมาตรฐาน มีความน่าเชื่อถือ ราคาถูก การตอบสนองที่รวดเร็ว และความแม่นยำในการวัดค่าสูง สามารถใช้งานกับไมโครคอนโทรลเลอร์ทั่วไปได้ ในโมดูลประกอบไปด้วย ส่วนวัดความชื้นแบบ Resistive type และส่วนวัดอุณหภูมิแบบNTC ให้สัญญาณเอาต์พุตแบบ Digital Output, การตรวจวัดคงที่กับ DHT11 Sensor, ย่านวัดอุณหภูมิ 0-50 องศาเซลเซียส, ย่านวัดความชื้น 20-90% RH, ง่ายต่อการติดตั้งใช้งาน

2) ทำการติดตั้ง library ของ DHT11 ตามวิธีการดังต่อไปนี้





3) ทำการเชื่อมต่อวงจรอ่านค่าจากโมดูลวัดอุณหภูมิและความชื้น (DHT11) ดังต่อไปนี้



4) เขียนโค้ดต่อไปนี้ลงใน Arduino IDE

```
#include "DHT.h"
#define DHTPIN 5
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
unsigned long previousTime = 0;
void setup() {
  Serial.begin(9600);
  Serial.println(F("DHTxx test!"));
  dht.begin();
}

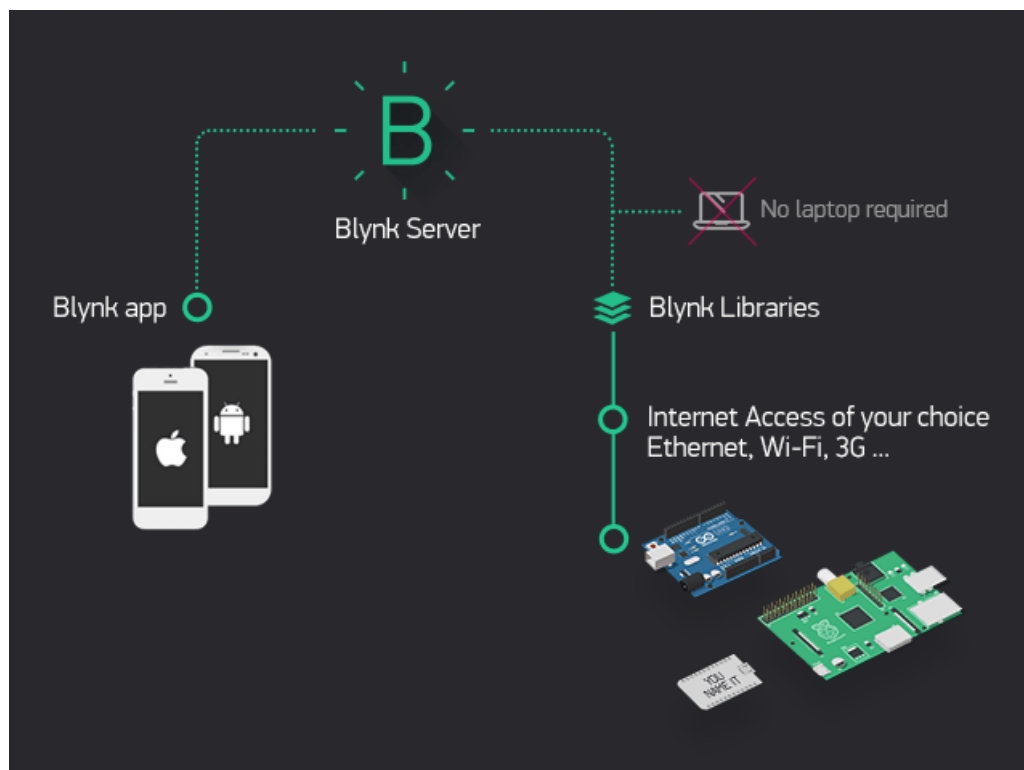
void loop() {
  if((millis() - previousTime) >= 2000){
    previousTime = millis();
    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (isnan(h) || isnan(t)) {
      Serial.println(F("Failed to read from DHT sensor!"));
      return;
    }
    Serial.print(F("Humidity: "));
    Serial.print(h);
    Serial.print(F("% Temperature: "));
    Serial.print(t);
    Serial.println(F("°C "));
  }
}
```

5) จากนั้นให้ทำ Save และรันอัปโหลดโปรแกรมลงใน ESP32 สังเกตผลการทำงาน

ตอนที่ 6

การเขียนโปรแกรมเพื่อแสดงค่าบน Blynk Platform

Blynk Platform คือ Application สำเร็จรูป ที่ถูกออกแบบมาเพื่อใช้ในการควบคุมอุปกรณ์ IOT ซึ่งมีคุณสมบัติในการควบคุมจากระยะไกลผ่านเครือข่ายอินเทอร์เน็ต และยังสามารถแสดงผลค่าจาก เซนเซอร์ต่าง ๆ ได้ แบบ Real time สามารถเชื่อมต่อ Device ต่าง ๆ รองรับทั้งระบบ Android และ iOS อีกด้วย



1) เข้าหน้าเว็บ <https://www.blynk.io/> และสมัครสมาชิก

B

Log In

EMAIL

PASSWORD

[Forgot password?](#)

Log In

Don't have an account yet? [Sign Up](#)

B

Sign Up

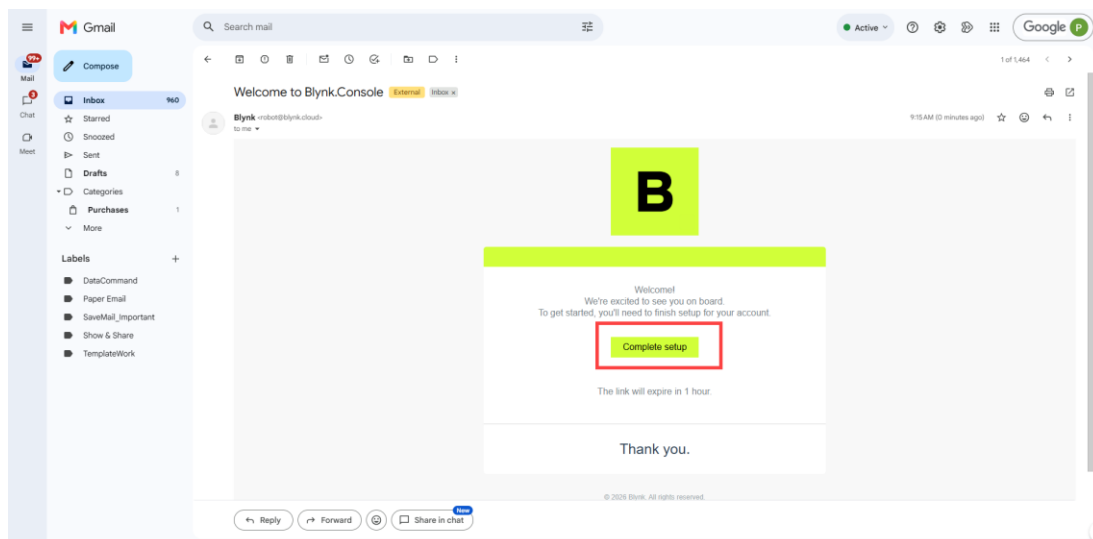
Welcome! Fill in your email address and we will send an account activation link.

EMAIL

☒ I agree to [Terms and Conditions](#) and accept [Privacy Policy](#)

Sign Up

[Back to Login](#)





Create Password

Create a password which is hard to guess.

PASSWORD

🔒
••••••••
👁

Good

Tips:

- Password should be at least 8 symbols long
- Use special characters (!, @, #, etc.)
- Use uncommon words
- Use non-standard uPPerCaSing
- Use creatif spellllllling
- Use non-obvii0u\$ number\$ & symbo1s

[Log In](#)

Next



Profile

Fill in your personal data

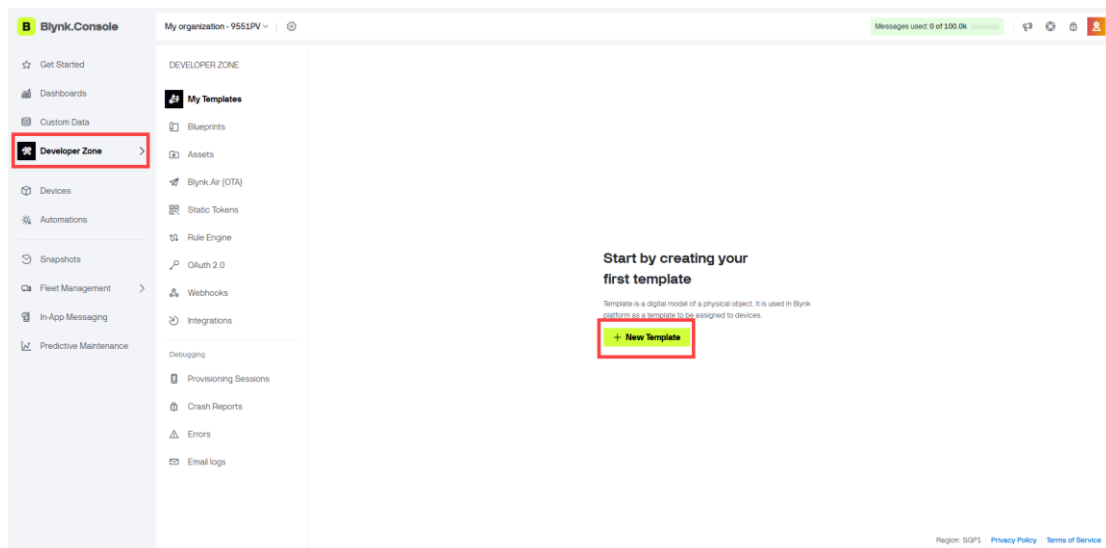
FIRST NAME

Ponglert

[Back to password creation](#)

Done

2) สร้างโปรเจ็คบน Blynk



Create New Template

NAME
testesp32 9 / 50

HARDWARE ESP32 CONNECTION TYPE WiFi

DESCRIPTION
Description 0 / 128

Cancel Done

Blynk.Console My organization - 9551PV Messages used 0 of 100.0k

Get Started Dashboards Custom Data Developer Zone Devices Automations Users Organizations Locations Snapshots Fleet Management In-App Messaging Predictive Maintenance

TESTESP32

Home Datastreams Dashboard Events & Notifications Connection Lifecycle Data Converters Metadata App Dashboard User Guides Automation Templates Voice Assistants

testesp32 # Template ID: TEMPL65T6a2Pa

Datastreams

No datastreams yet

Datastreams are the values your device sends and receives — readings, states and controls. Search the library, or describe your project and let AI pick.

Browse Presets Create New

Region: BQF1 Privacy Policy Terms of Service

Blynk.Console My organization - 6475IC Messages used 0 of 30k

Developer Zone Devices Users Organizations Locations

TESTESP32

Home Datastreams Web Dashboard Metadata Connection Lifecycle Events & Notifications User Guides Mobile Dashboard

testesp32

Datastreams

Datastreams is a way to structure data that regularly flows in and out from device. Use it for sensor data, any telemetry, or actuators.

+ New Datastream

Virtual Pin Enum Location Digital Pin Analog Pin

UPGRADE

Region: sgp1 Privacy Policy

New Datastream

General

- Semantic
- Units
- History
- Expiration
- Sync
- Automations
- Visibility
- Notes
- API reference

General

DATASTREAM NAME → DISPLAY NAME

For your reference and for MQTT topic naming Shown in widgets, dashboards and reports.

VIRTUAL PIN COLOR ICON

DATA TYPE Double String Enum Location

MIN VALUE MAX VALUE DEFAULT VALUE

Semantic

Semantic tags enable Blynk to understand the meaning of the datastream. This will be used in data aggregations, dashboards, automations, and beyond.

+ From presets + Custom tag

Cancel Save

Blynk Console

My organization - 9551PV

Messages used 0 of 100.0k

Get Started

Dashboards

Custom Data

Developer Zone

Devices

Automations

Users

Organizations

Locations

Snapshots

Fleet Management

In-App Messaging

Predictive Maintenance

TESTESP32

Home

Datastreams

Dashboard

Events & Notifications

Connection Lifecycle

Data Converters

Metadata

App Dashboard

User Guides

Automation Templates

Voice Assistants

testesp32

Template ID: TMP1A5TGwQPa

Cancel Save

Datastreams

Search datastream...

Browse Presets New Datastream

Integer Double String Enum Location Digital Analog

Pin	Name	Data type	Semantics	Min	Max	Default	Units	Automations	Actions
V1	Relay1	Integer	+ Add	0	1	0	-	[H] None	

Region: GCP1 Privacy Policy Terms of Service

Blynk Console

My organization - 9551PV

Messages used 0 of 100.0k

Get Started

Dashboards

Custom Data

Developer Zone

Devices

Automations

Users

Organizations

Locations

Snapshots

Fleet Management

In-App Messaging

Predictive Maintenance

TESTESP32

Home

Datastreams

Dashboard

Events & Notifications

Connection Lifecycle

Data Converters

Metadata

App Dashboard

User Guides

Automation Templates

Voice Assistants

testesp32

Template ID: TMP1A5TGwQPa

Cancel Save

Datastreams

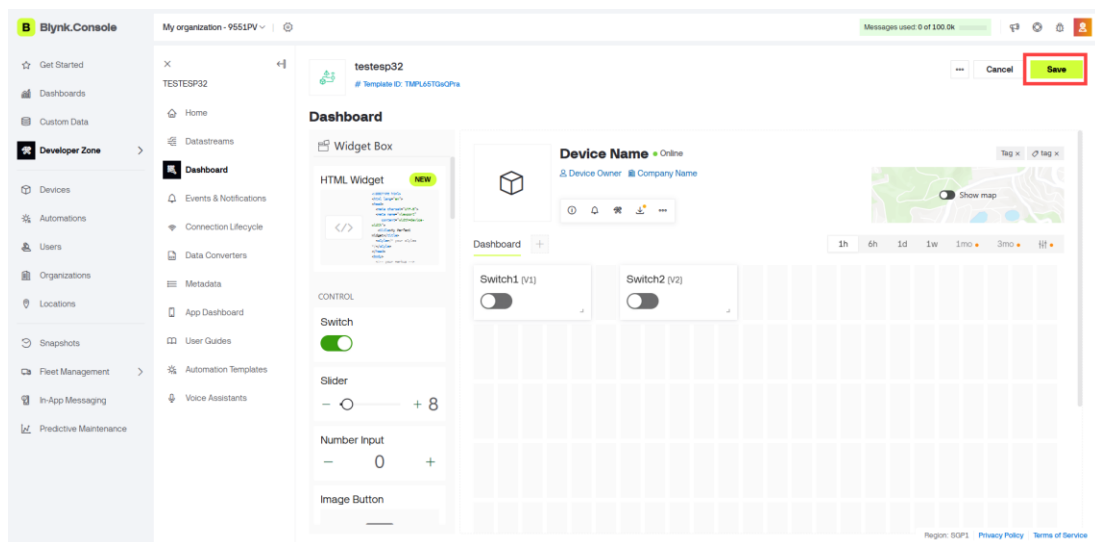
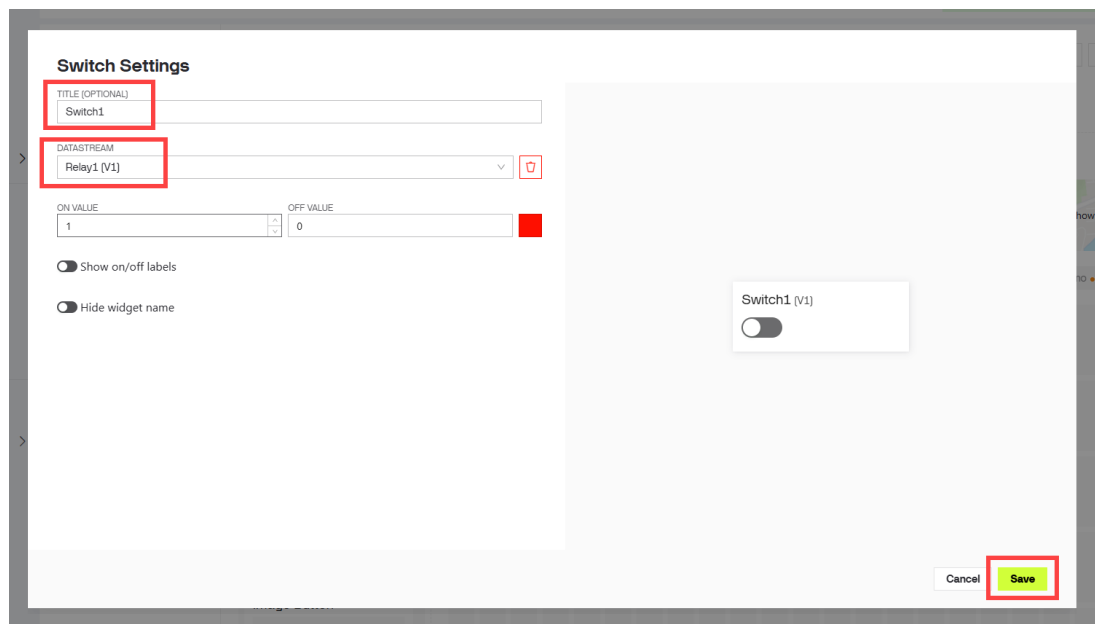
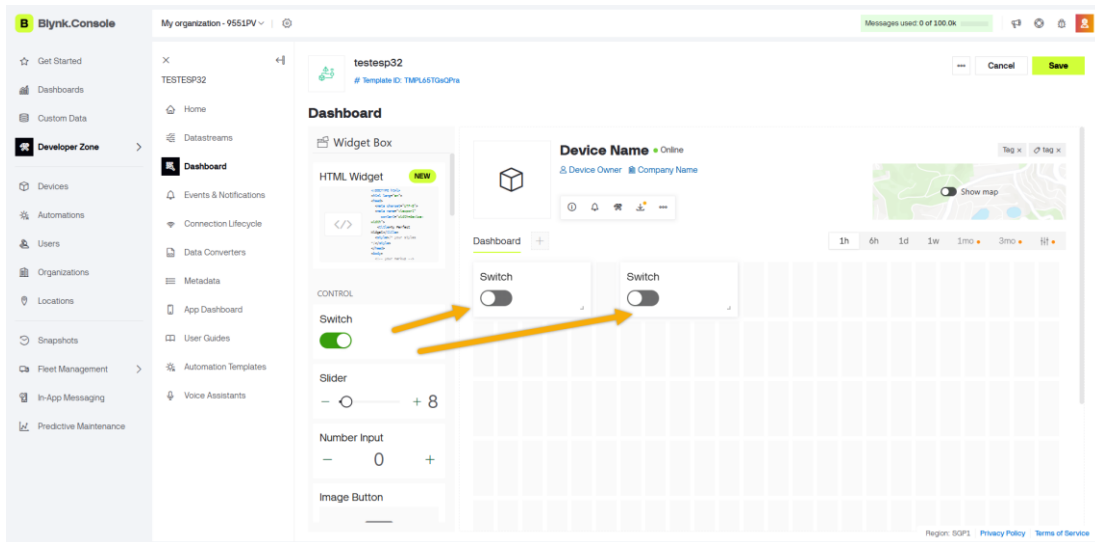
Search datastream...

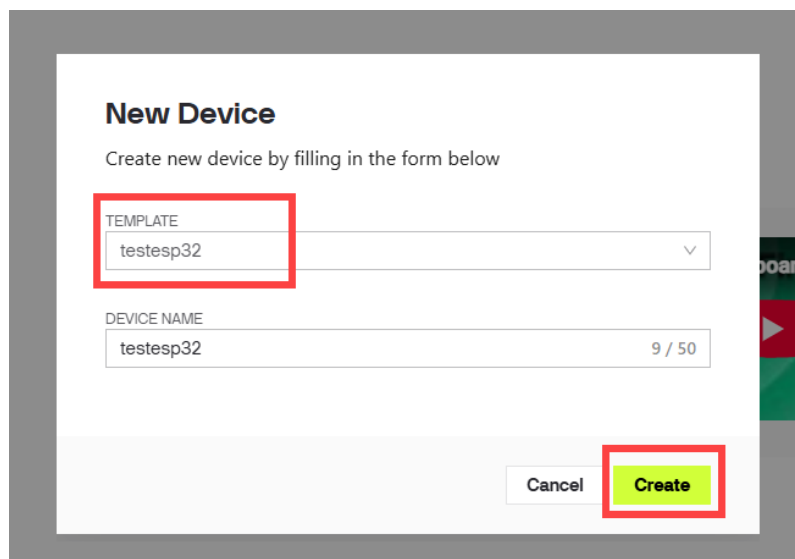
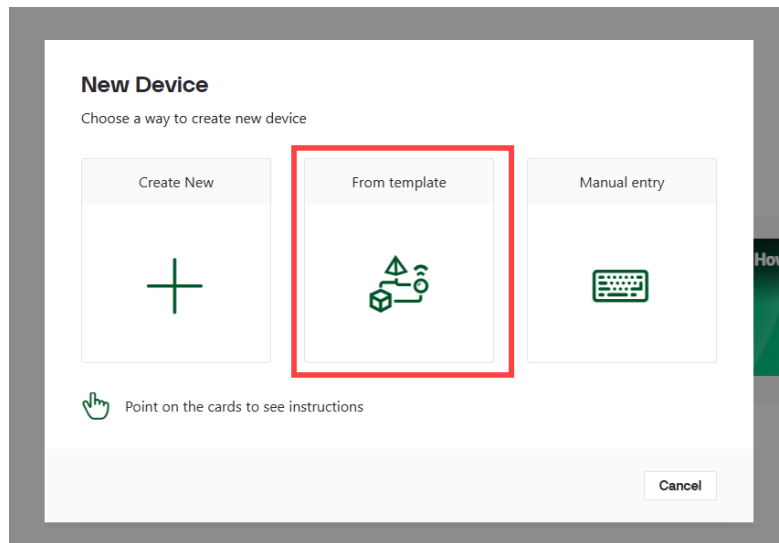
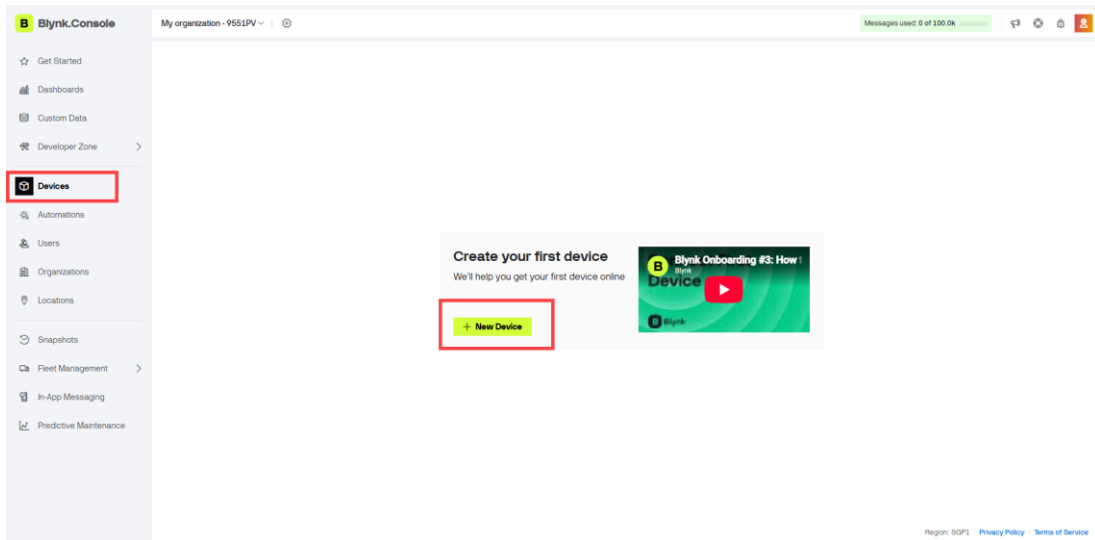
Browse Presets New Datastream

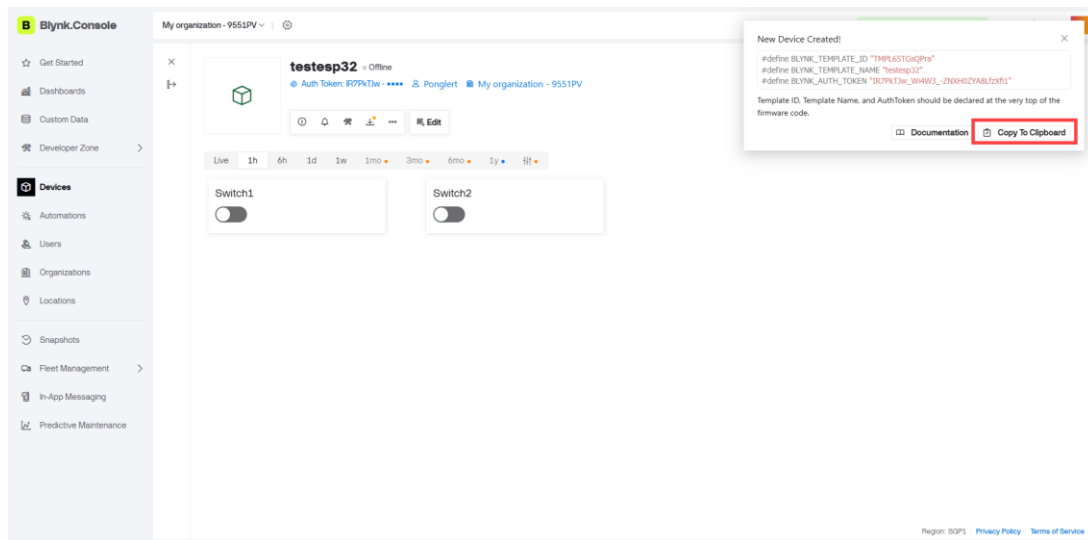
Integer Double String Enum Location Digital Analog

Pin	Name	Data type	Semantics	Min	Max	Default	Units	Automations	Actions
V1	Relay1	Integer	+ Add	0	1	0	-	[H] None	
V2	Relay2	Integer	+ Add	0	1	0	-	[H] None	

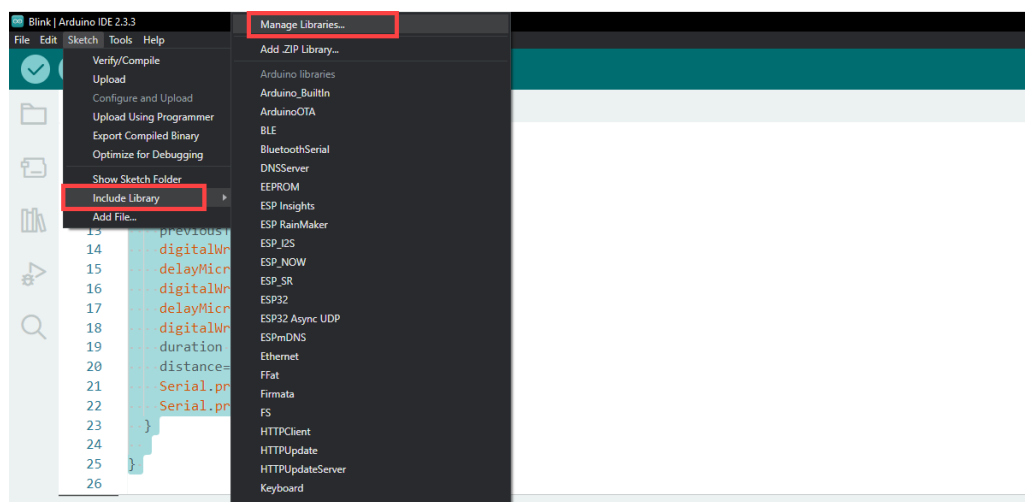
Region: GCP1 Privacy Policy Terms of Service

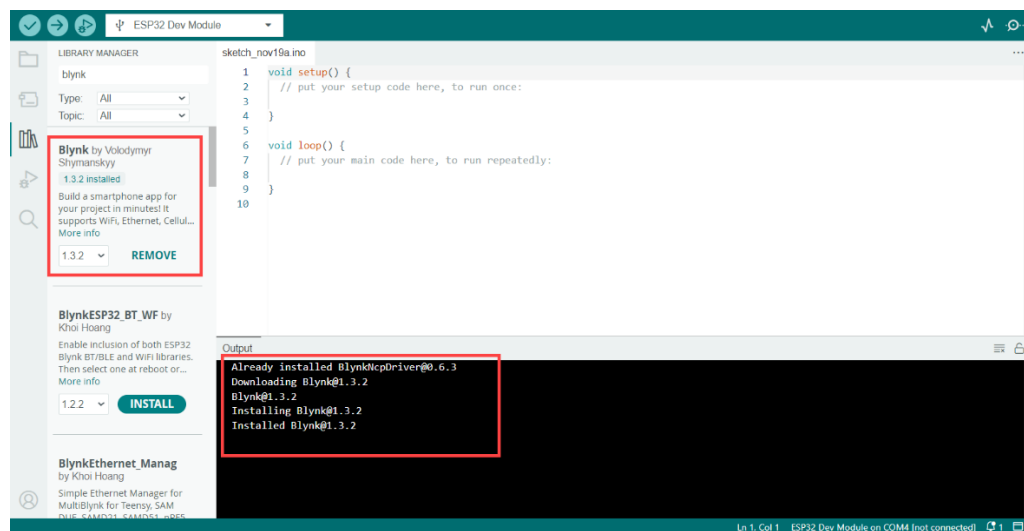
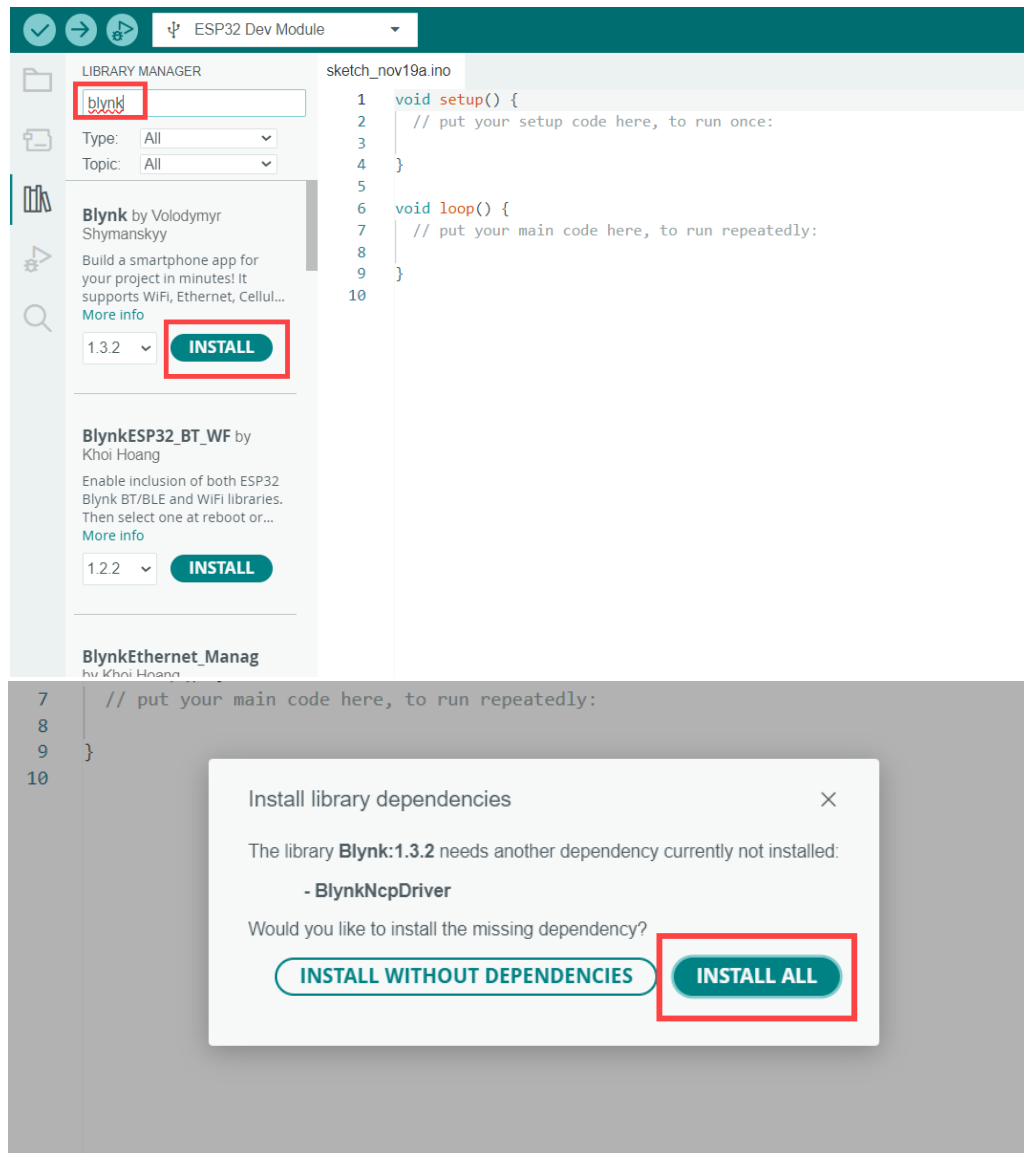




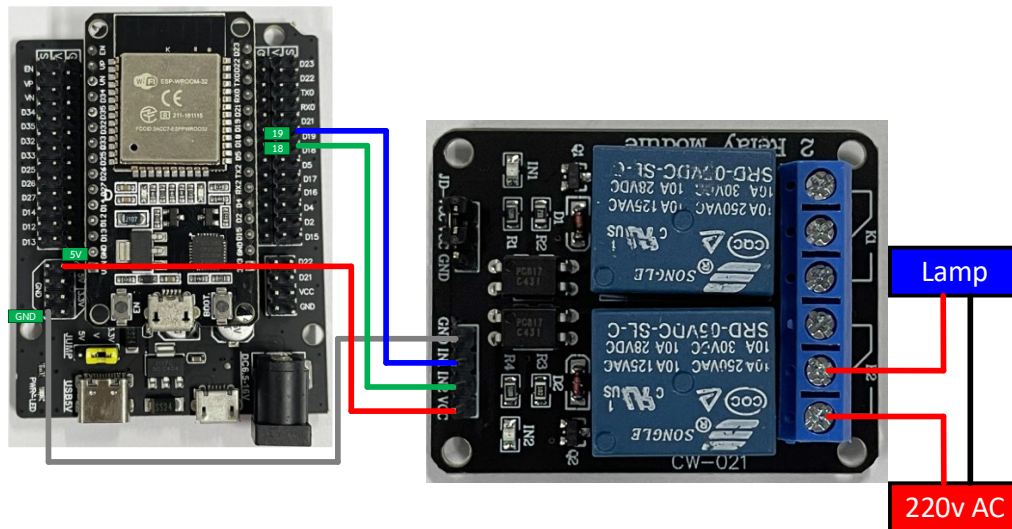


3) ติดตั้ง Library ของ Blynk Platform





4) ทำการเชื่อมต่อวงจรโมดูลควบคุมโมดูลรีเลย์ ดังต่อไปนี้



5) การเขียนโปรแกรมเพื่อส่งข้อมูลไปแสดงผลบน Blynk Platform

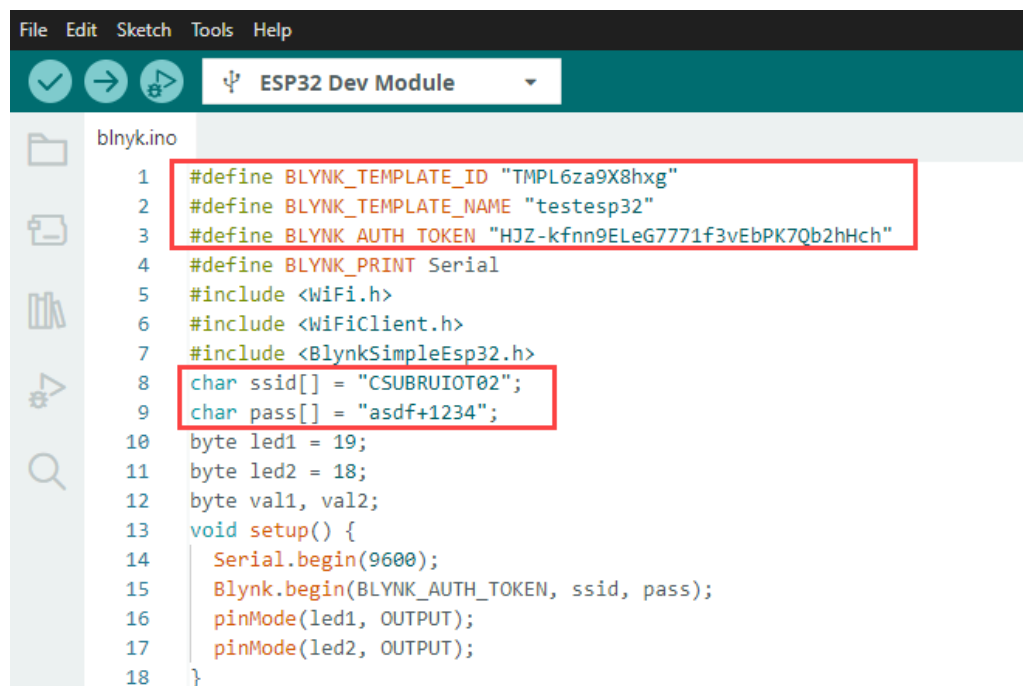
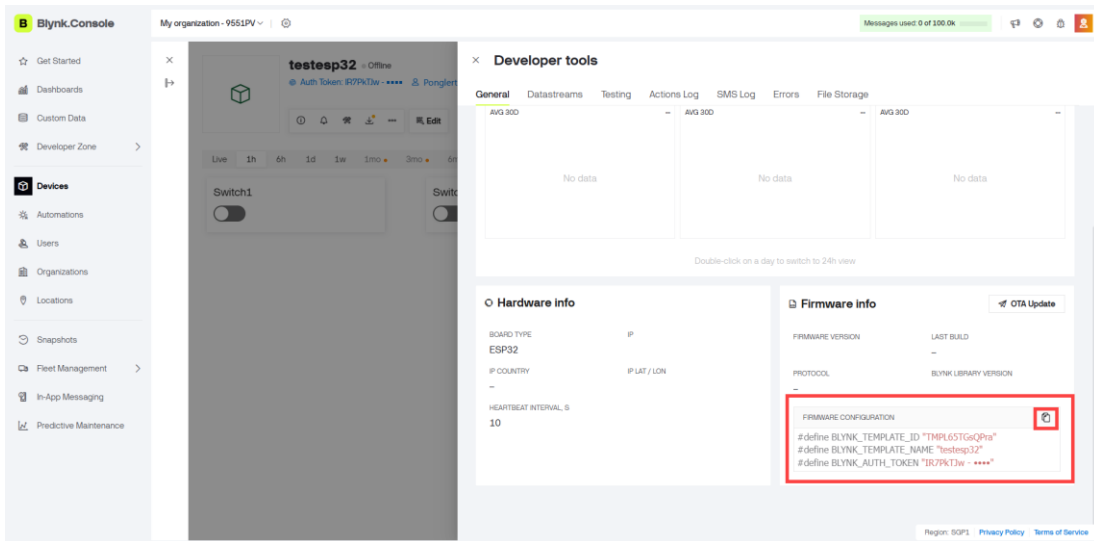
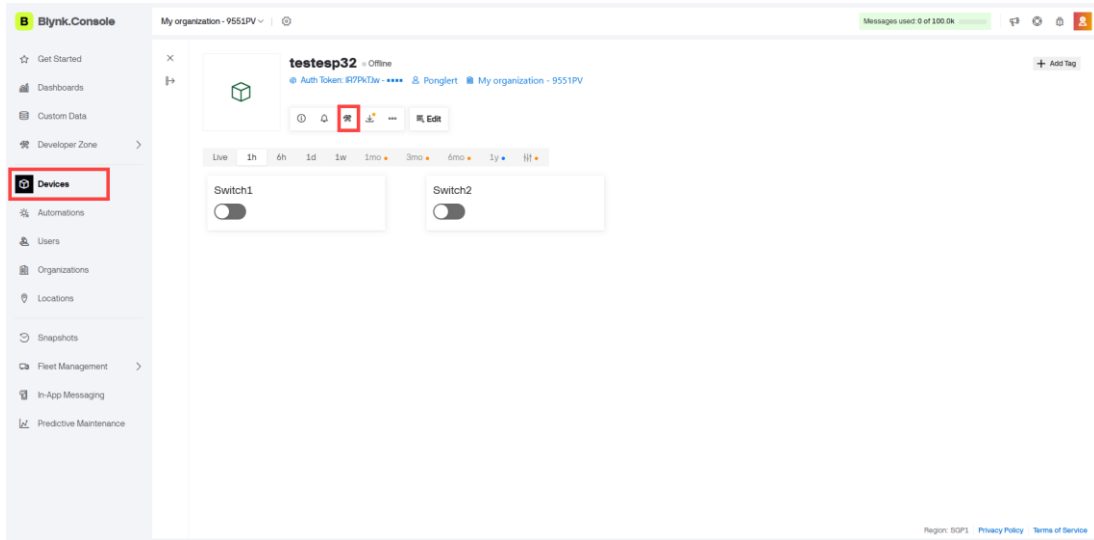
```

e
sketch_nov19a.ino
1  #define BLYNK_TEMPLATE_ID "MyTemplateID"
2  #define BLYNK_DEVICE_NAME "MyDeviceName"
3  #define BLYNK_AUTH_TOKEN "MyAuthToken"
4  #define BLYNK_PRINT Serial
5  #include <WiFi.h>
6  #include <WiFiClient.h>
7  #include <BlynkSimpleEsp32.h>
8  char ssid[] = "YourNetworkName";
9  char pass[] = "YourPassword";
10 void setup() {
11     Serial.begin(9600);
12     Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
13 }
14 void loop() {
15     Blynk.run();
16 }

```

Copy มาจากเว็บ Blynk

ชื่อ Wifi และ รหัสผ่าน



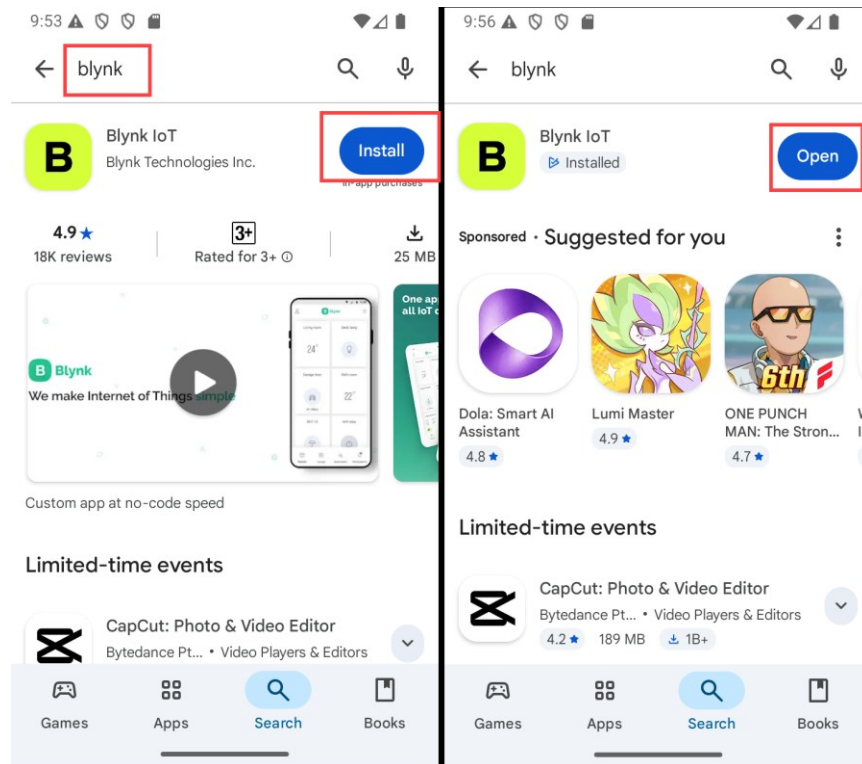
```

#define BLYNK_TEMPLATE_ID "TMPL6za9X8hxg"
#define BLYNK_TEMPLATE_NAME "testesp32"
#define BLYNK_AUTH_TOKEN "HJZ-kfnn9ELeG7771f3vEbPK7Qb2hHch"
#define BLYNK_PRINT Serial
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
char ssid[] = "CSUBRUIOT02";
char pass[] = "asdf+1234";
byte led1 = 19;
byte led2 = 18;
byte val1, val2;
void setup() {
  Serial.begin(9600);
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
}
void loop() {
  Blynk.run();
  digitalWrite(led1, val1);
  digitalWrite(led2, val2);
}
BLYNK_CONNECTED(){
  Blynk.syncVirtual(V1,V2);
}
BLYNK_WRITE(V1){
  val1 = param.asInt();
}
BLYNK_WRITE(V2){
  val2 = param.asInt();
}

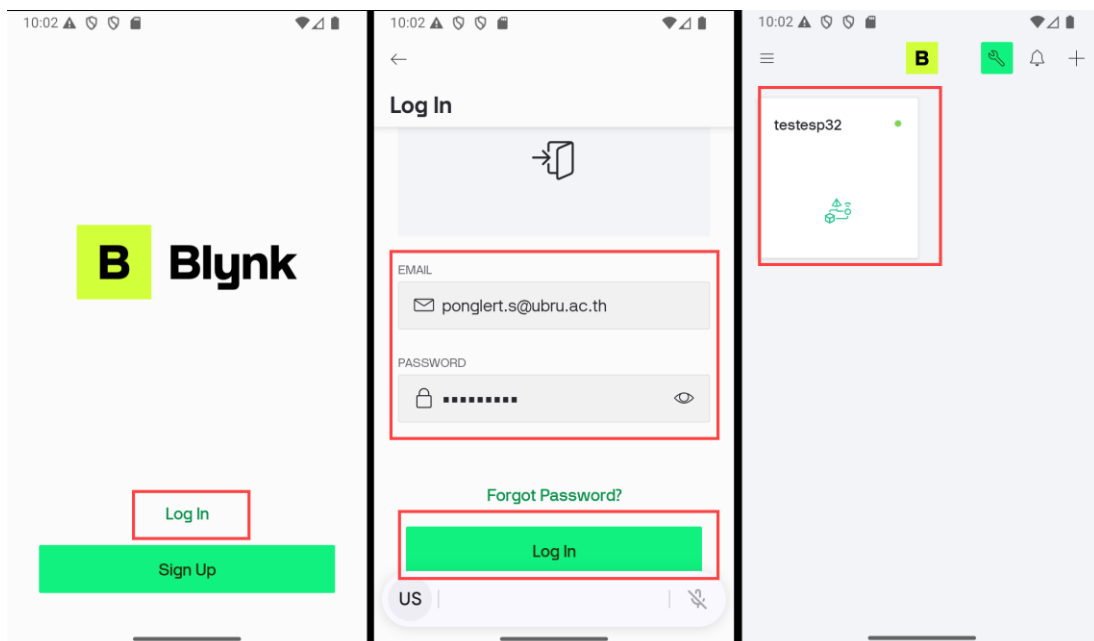
```

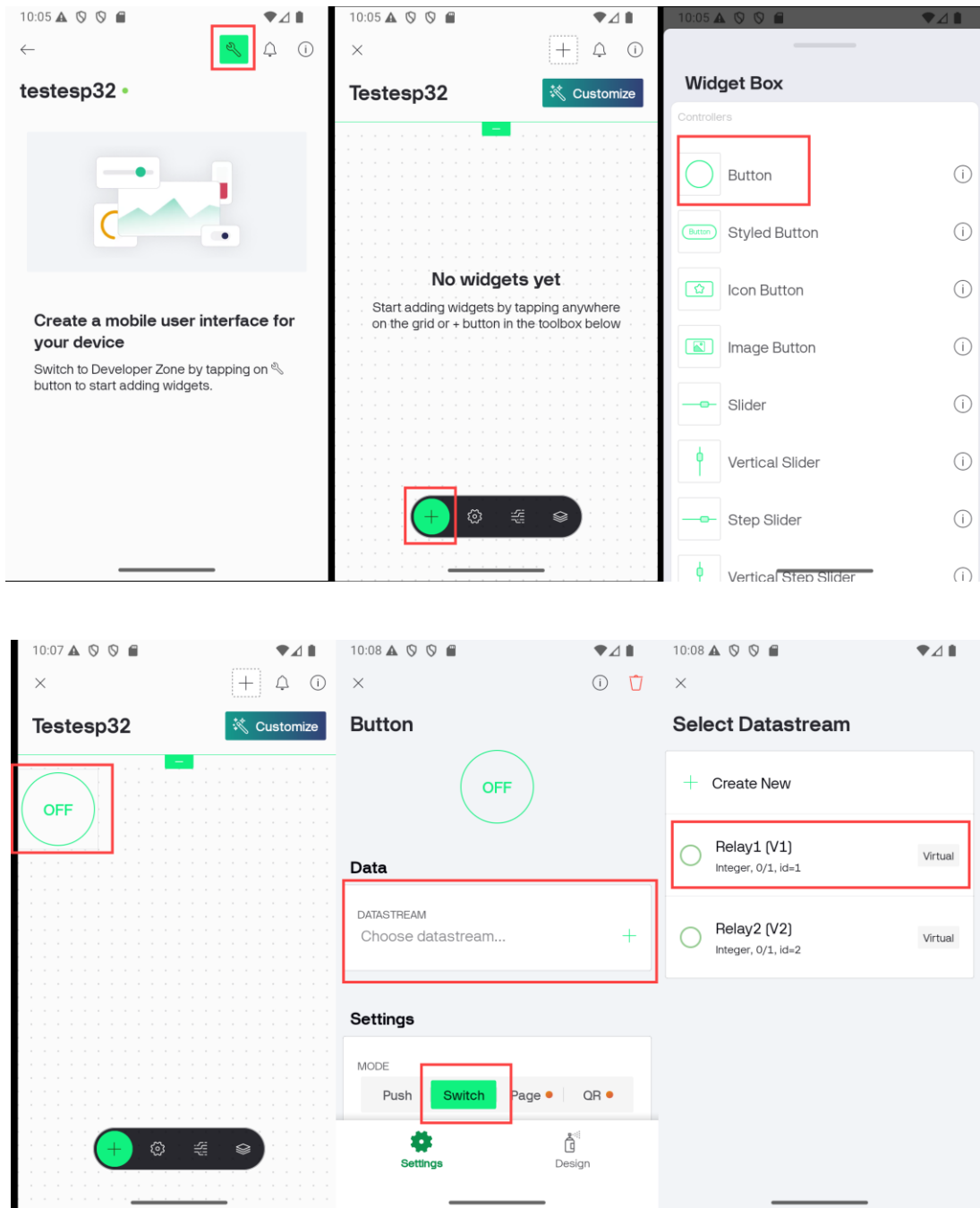
6) จากนั้นให้ทำ Save และรันอัปโหลดโปรแกรมลงใน ESP32

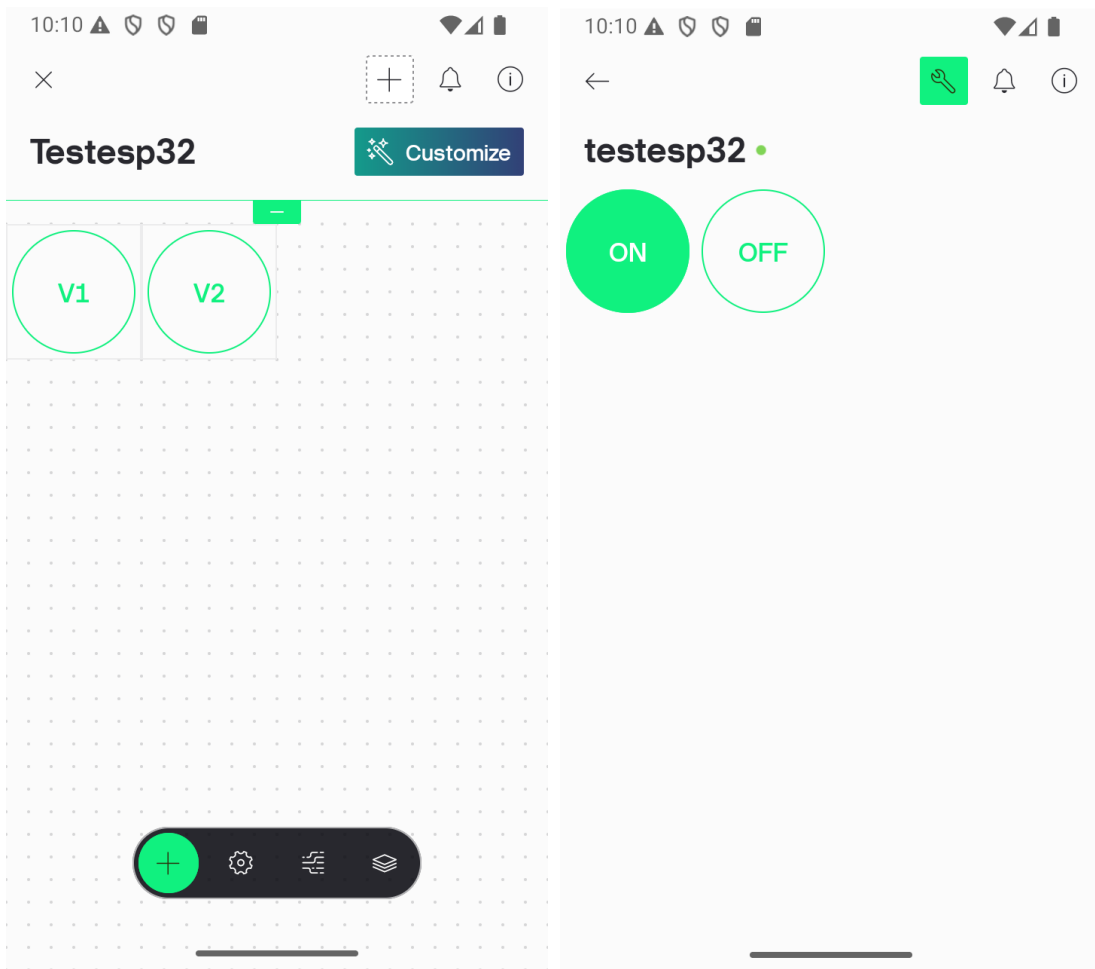
7) ติดตั้ง Blynk Application ในสมาร์ทโฟน



8) เข้าสู่ระบบและตั้งค่าใช้งาน Blynk Application ควบการทำงานของอุปกรณ์







9) ทดสอบการทำงาน

การส่งค่าเซนเซอร์ข้อมูลไปแสดงผลบน Blynk

- 1) เพิ่ม Virtual Pin ลงใน Datastreams

Blynk.Console My organization - 9551PV Messages used: 9 of 100.0k

testesp32 # Template ID: TMP1a5T0aQPha

Datastreams

Search datastream...

Integer Double String Enum Location Digital Analog

Pin	Name	Data type	Semantics	Min	Max	Default	Units	Automations
V1	Relay1	Integer		0	1	0	-	[None]
V2	Relay2	Integer		0	1	0	-	[None]

Region: BGP1 Privacy Policy Terms of Service

Blynk.Console My organization - 9551PV Messages used: 9 of 100.0k

testesp32 # Template ID: TMP1a5T0aQPha

Datastreams

Search datastream...

Browse Presets **New Datastream**

Integer Double String Enum Location Digital Analog

Pin	Name	Data type	Semantics	Min	Max	Default	Units	Automations	Actions
V1	Relay1	Integer	+ Add	0	1	0	-	[None]	
V2	Relay2	Integer	+ Add	0	1	0	-	[None]	

Region: BGP1 Privacy Policy Terms of Service

New Datastream

General
Semantic
Units
History
Expiration
Sync
Automations
Visibility
Notes
API reference

General

DATASTREAM NAME

Temp

DISPLAY NAME

Temp

For your reference and for MQTT topic naming

Shown in widgets, dashboards and reports.

VIRTUAL PIN

V3

COLOR

ICON

DATA TYPE

Integer Double String Enum Location

MIN VALUE

0

MAX VALUE

100

DECIMALS

###

DEFAULT VALUE

0

Semantic

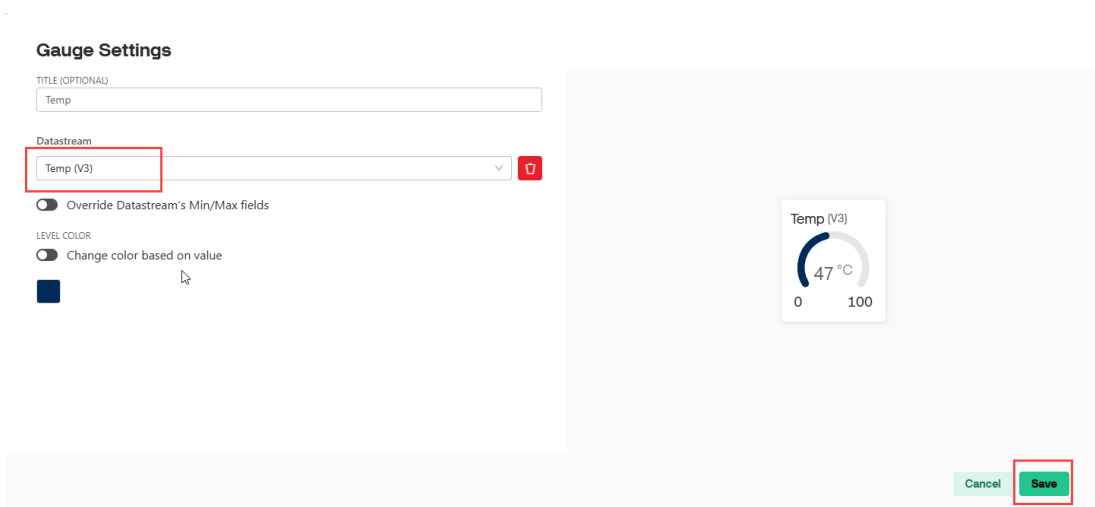
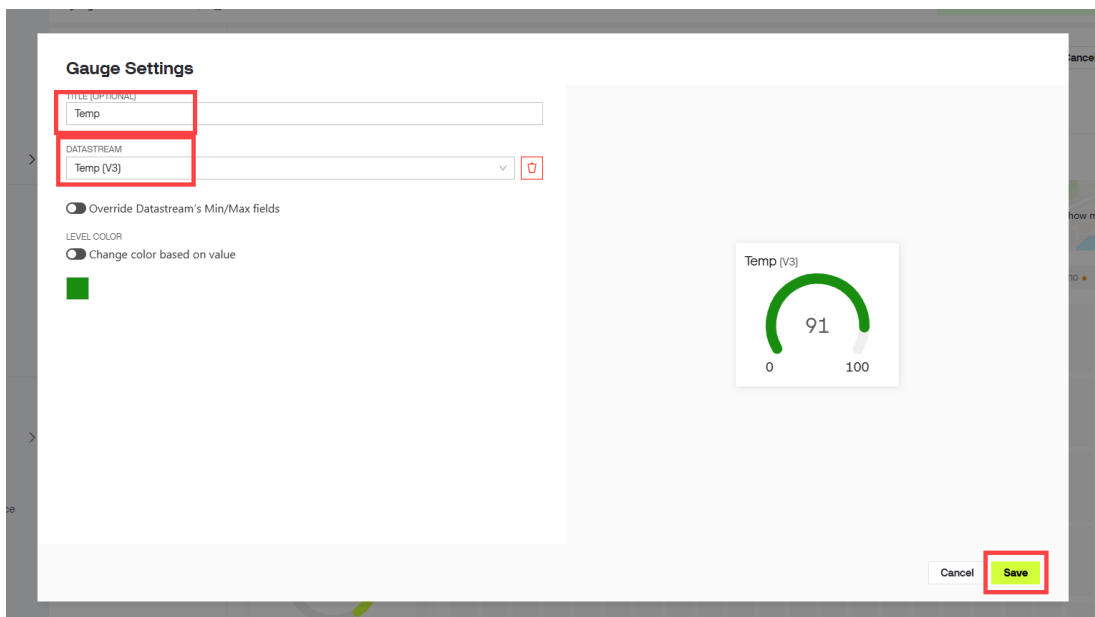
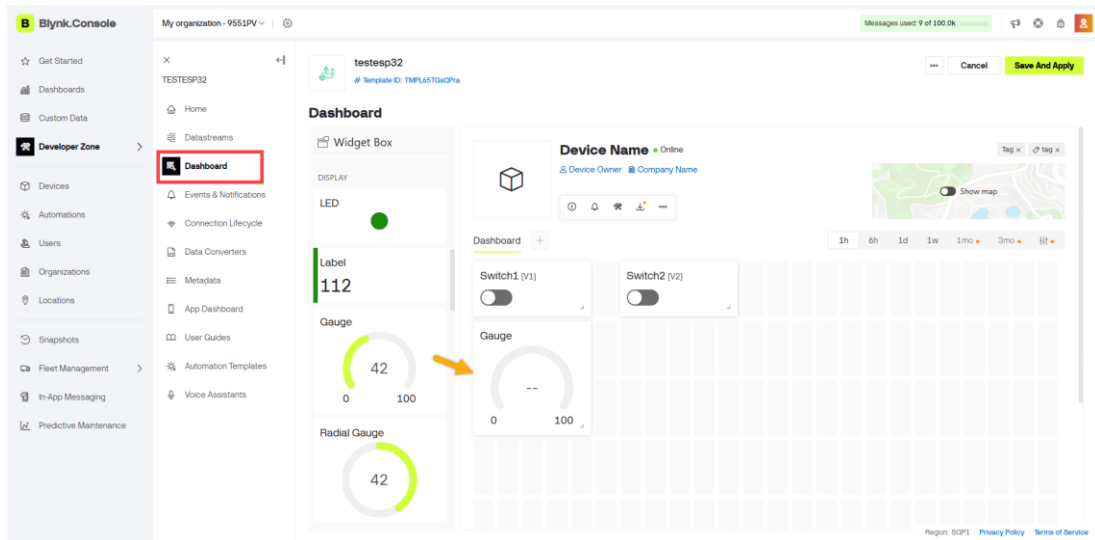
Semantic tags enable Blynk to understand the meaning of the datastream. This will be used in data aggregations, dashboards, automations, and beyond.

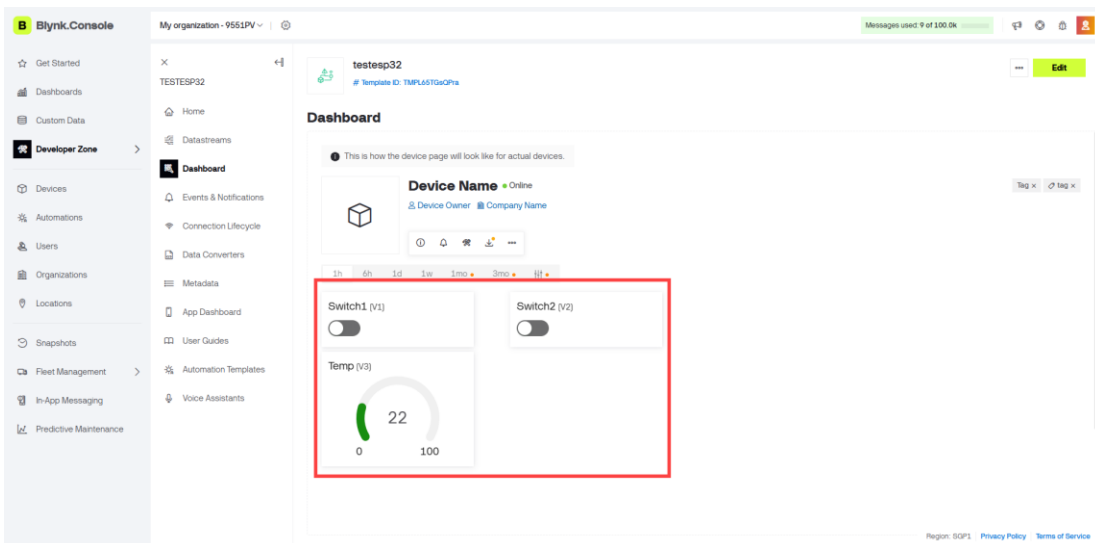
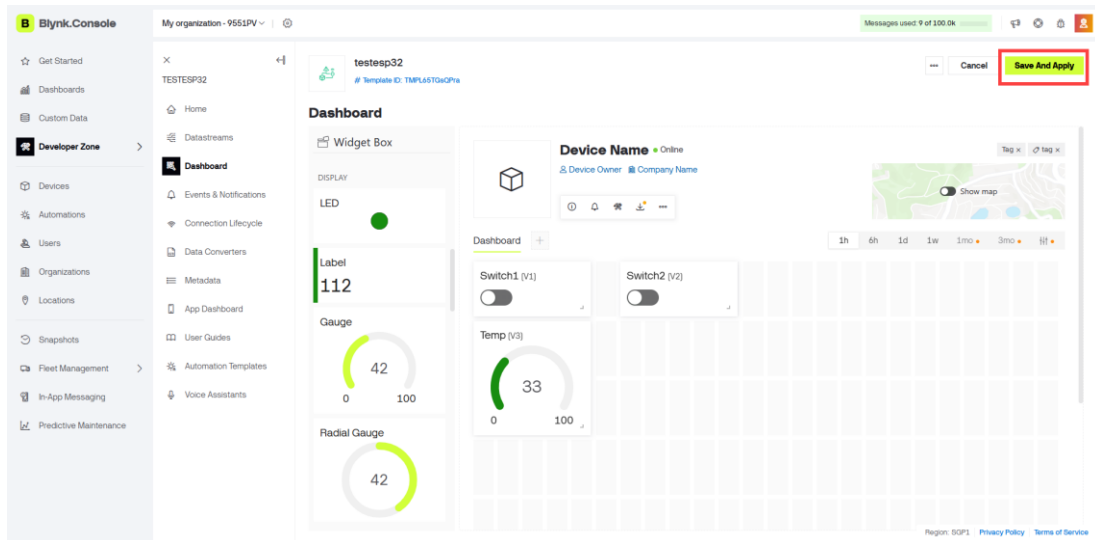
+ From presets

+ Custom tag

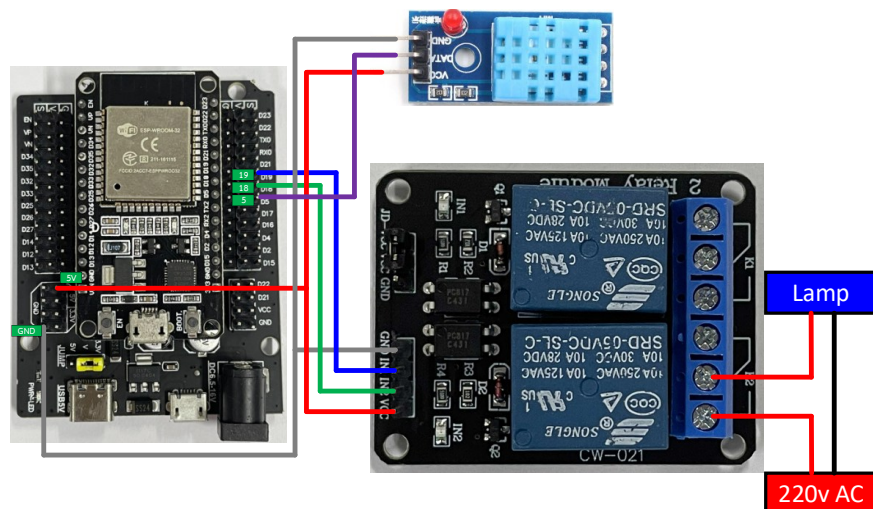
Cancel

Save





2) ทำการเชื่อมต่อวงจรโมดูลเซนเซอร์ DHT11 เพิ่มเข้าไป ดังต่อไปนี้



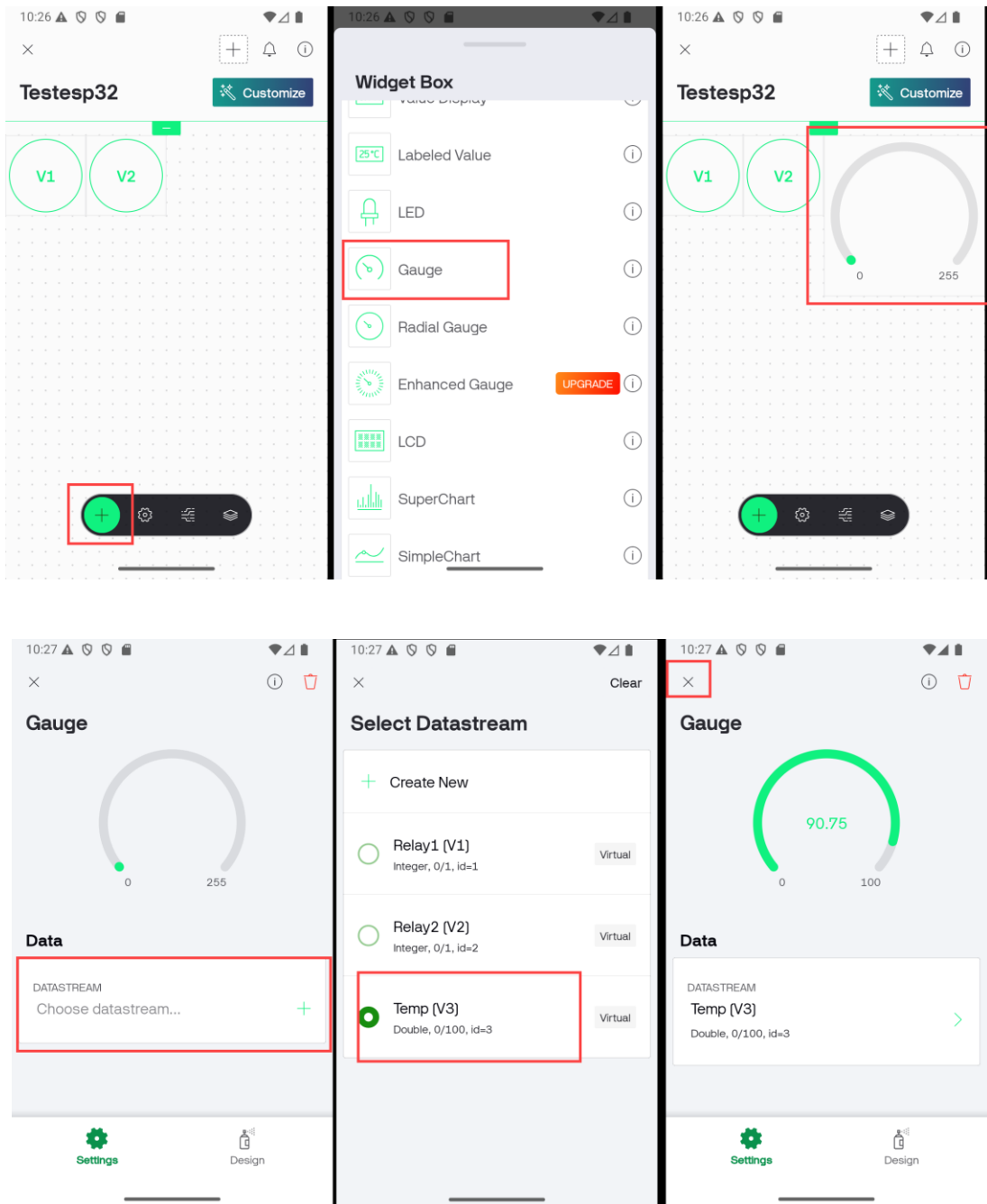
3) การเขียนโปรแกรมเพิ่มเพื่อส่งข้อมูลไปแสดงผลบน Blynk Platform

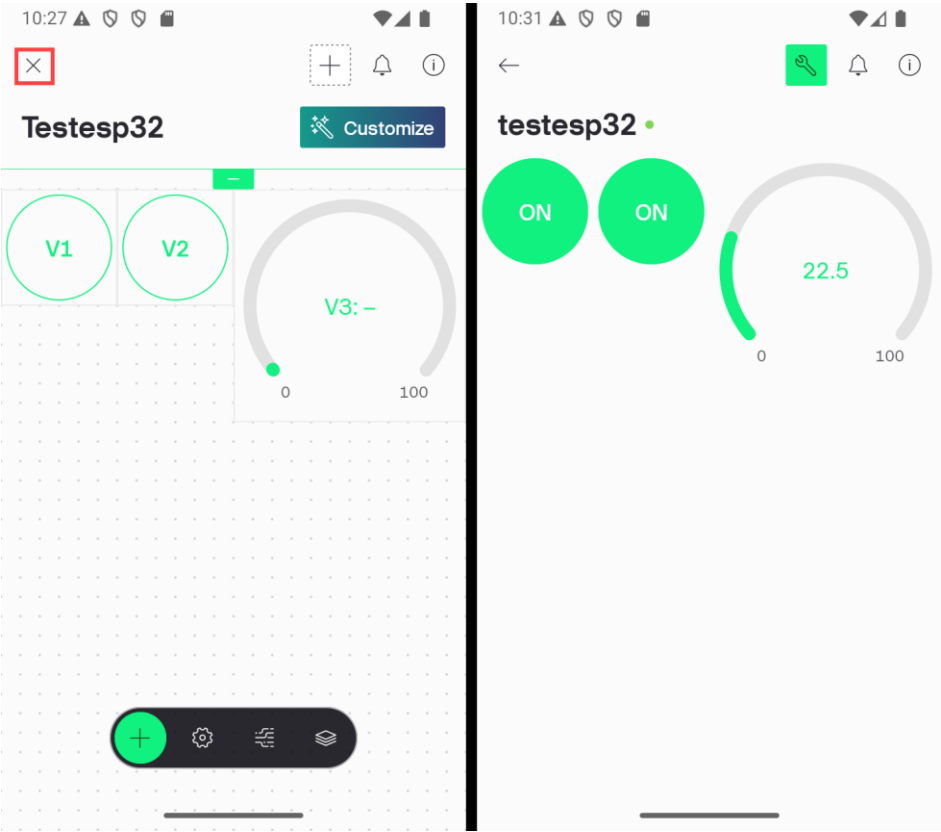
```
#define BLYNK_TEMPLATE_ID "TMPL6za9X8hXg"
#define BLYNK_TEMPLATE_NAME "testesp32"
#define BLYNK_AUTH_TOKEN "HJZ-kfnn9ELeG7771f3vEbPK7Qb2hHch"
#define BLYNK_PRINT Serial
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
char ssid[] = "CSUBRUIOT02";
char pass[] = "asdf+1234";

#include "DHT.h"
#define DHTPIN 5
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
unsigned long previousTime = 0;

byte led1 = 19;
byte led2 = 18;
byte val1, val2;
void setup() {
  Serial.begin(9600);
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  dht.begin();
}
void loop() {
  if((millis() - previousTime) >= 2000){
    previousTime = millis();
    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (isnan(h) || isnan(t)) {
      Serial.println(F("Failed to read from DHT sensor!"));
    }
    else{
      Blynk.virtualWrite(V3, t);
      Serial.print(F("Humidity: "));
      Serial.print(h);
      Serial.print(F("% Temperature: "));
      Serial.print(t);
      Serial.println(F("°C "));
    }
  }
  Blynk.run();
  digitalWrite(led1, val1);
  digitalWrite(led2, val2);
}
BLYNK_CONNECTED(){
  Blynk.syncVirtual(V1,V2);
}
BLYNK_WRITE(V1){
  val1 = param.asInt();
}
BLYNK_WRITE(V2){
  val2 = param.asInt();
}
```

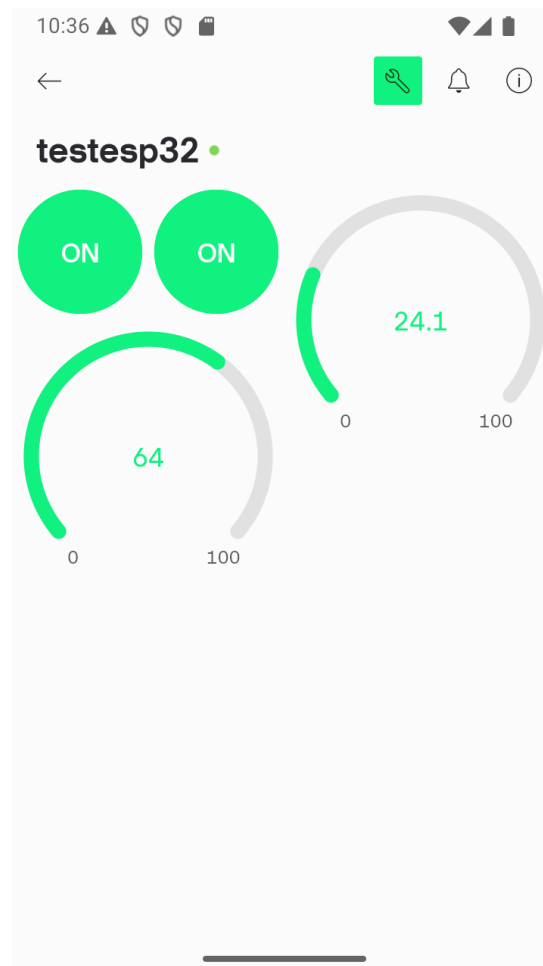
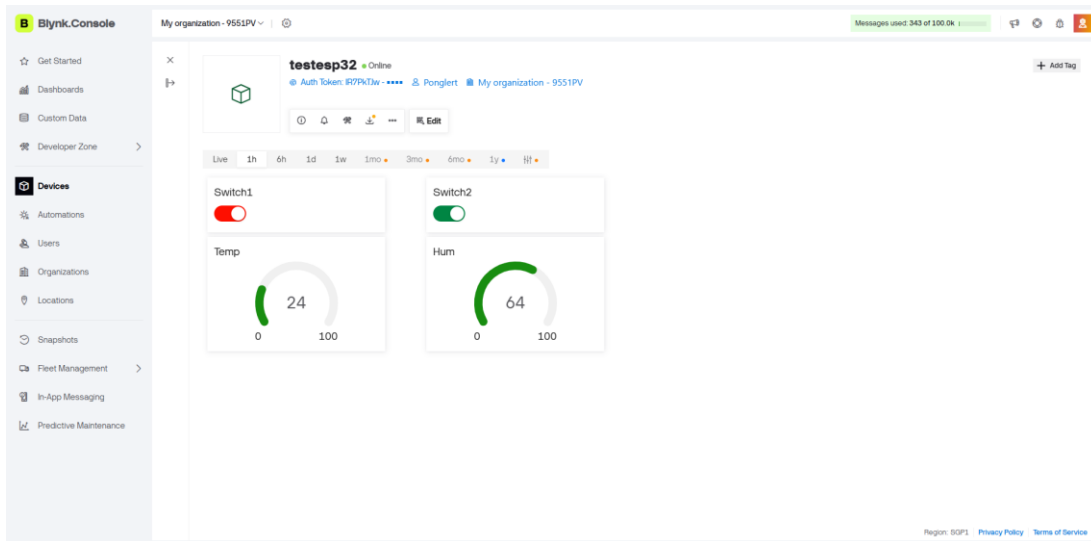
7) เข้าสู่ระบบและตั้งค่าใช้งานเพิ่มเติม Blynk Application





แบบฝึกหัด

ทำการแสดงค่าความชื้นในอากาศจากเซนเซอร์บนแอปพลิเคชัน blynk และเว็บ Dashboard blynk โดยให้ทำการเขียนโปรแกรมเพิ่มจากโค้ดก่อนหน้านี้ และตั้งค่าต่าง ๆ เพิ่มในแพลตฟอร์ม blynk ให้ได้ผลลัพธ์ดังรูป

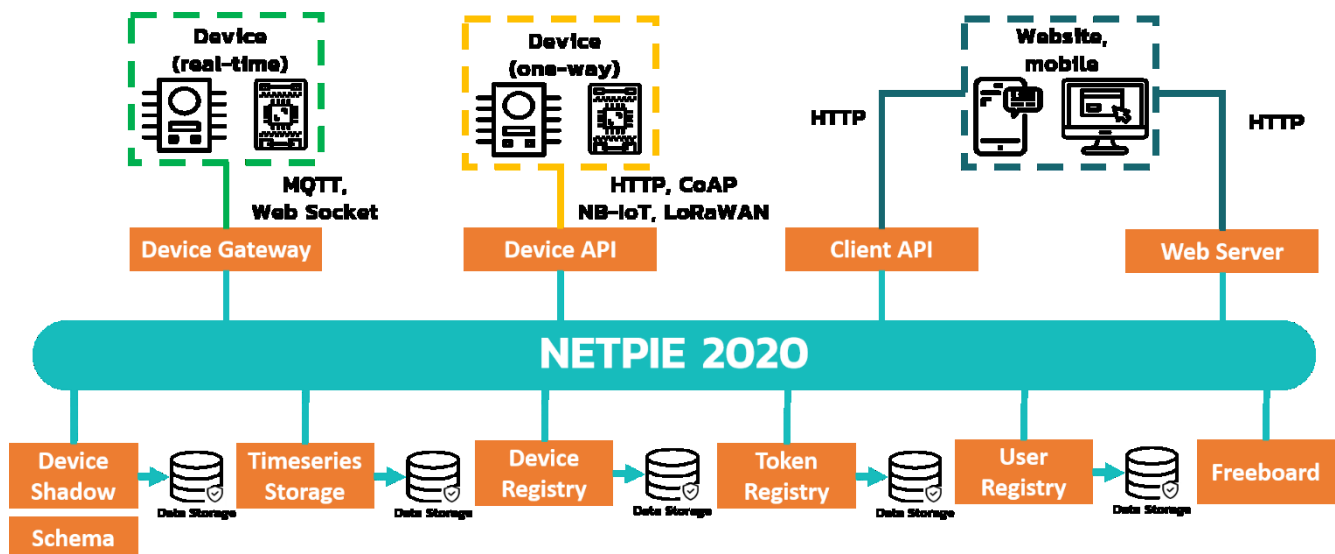


ตอนที่ 7

การเขียนโปรแกรมผ่าน NETPIE

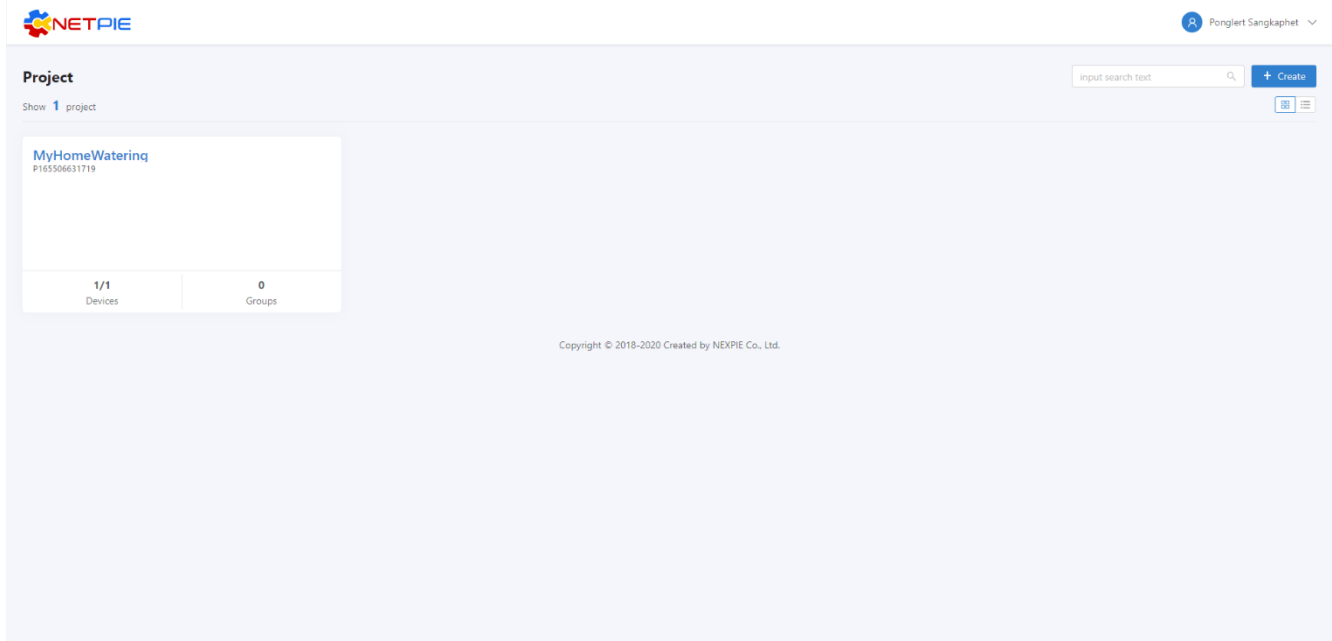
NETPIE 2020 คือแพลตฟอร์มที่ถูกพัฒนาขึ้นเพื่อตอบสนองผู้ใช้งานเชิงพาณิชย์ เช่น ผู้ผลิตอุปกรณ์ IoT, อุตสาหกรรม, โรงงาน และองค์กรที่พัฒนาสู่ยุค Digital Transformation 4.0 ซึ่งจะช่วยธุรกิจให้มีประสิทธิภาพยิ่งขึ้น ด้วยเทคโนโลยีการเชื่อมต่อทุกสรรพสิ่ง หรือ Internet of Things (IoT)

โดยแพลตฟอร์มจะช่วยให้อุปกรณ์ต่างๆ สามารถสื่อสารกันได้ เกิดการรับ-ส่งข้อมูลระหว่างอุปกรณ์แบบ real-time ทำให้ผู้ใช้งานทราบถึงข้อมูลของอุปกรณ์ ณ เวลานั้นๆ ไม่ว่าผู้ใช้งานจะอยู่ที่ไหนเวลาใดก็ตาม ทั้งยังรองรับการเชื่อมต่อกับอุปกรณ์ IoT ได้จำนวนมากมหาศาล ทำให้ตอบโจทย์กลุ่มผู้ใช้งานเชิงพาณิชย์ที่มีอุปกรณ์ IoT จำนวนมากอย่างแน่นอน

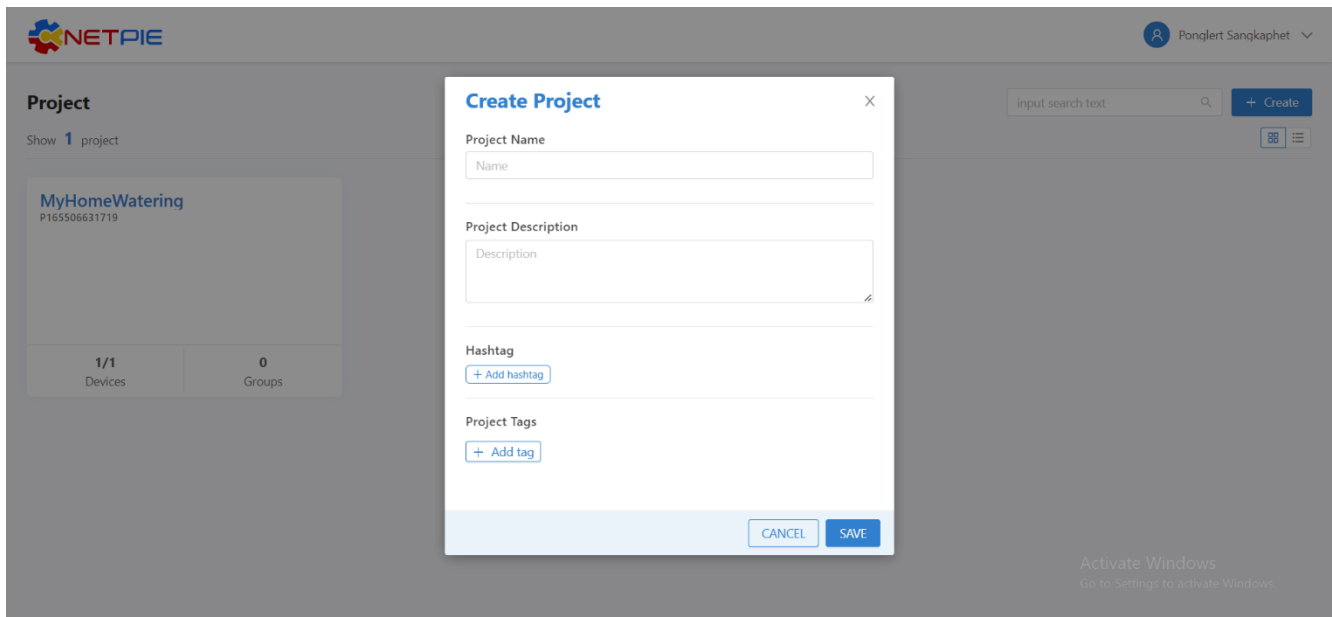


สร้าง Project ใน NETPIE

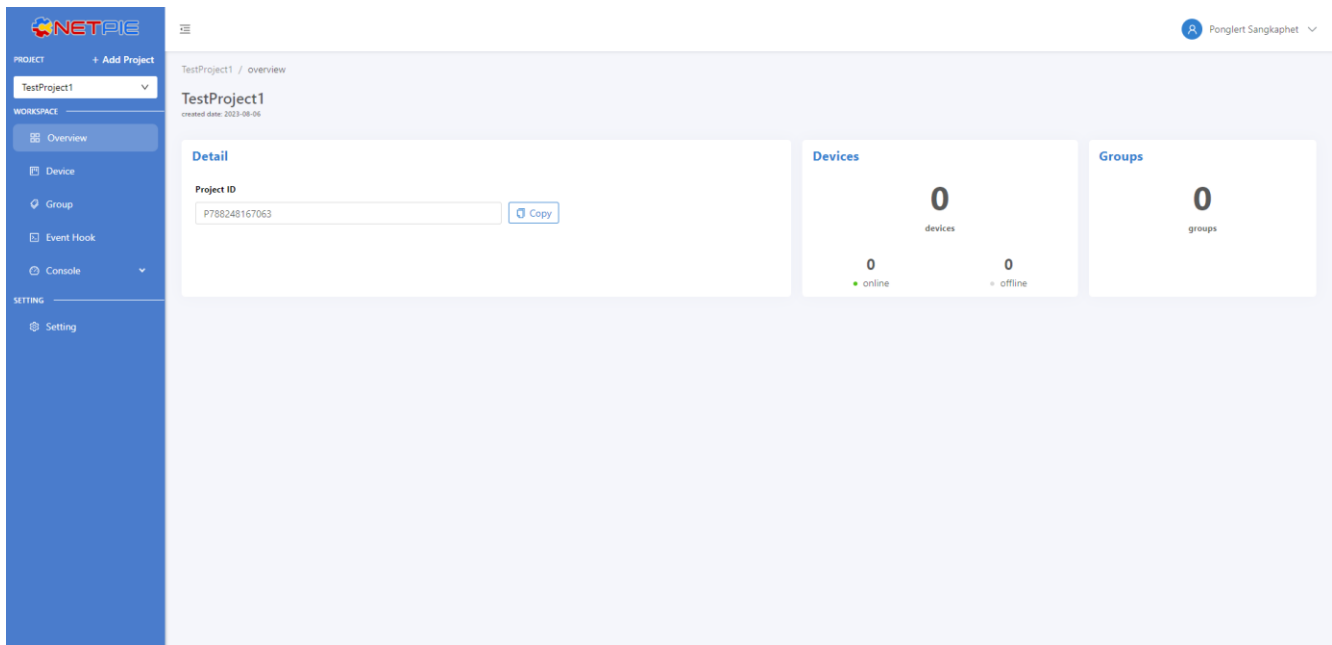
1) เมื่อ login เข้าสู่ <https://portal.netpie.io> เรียบร้อยแล้ว หน้าจอที่ปรากฏจะแสดงรายการ Project ทั้งหมดที่เคยสร้างไว้แล้ว



2) คลิกที่ + Create

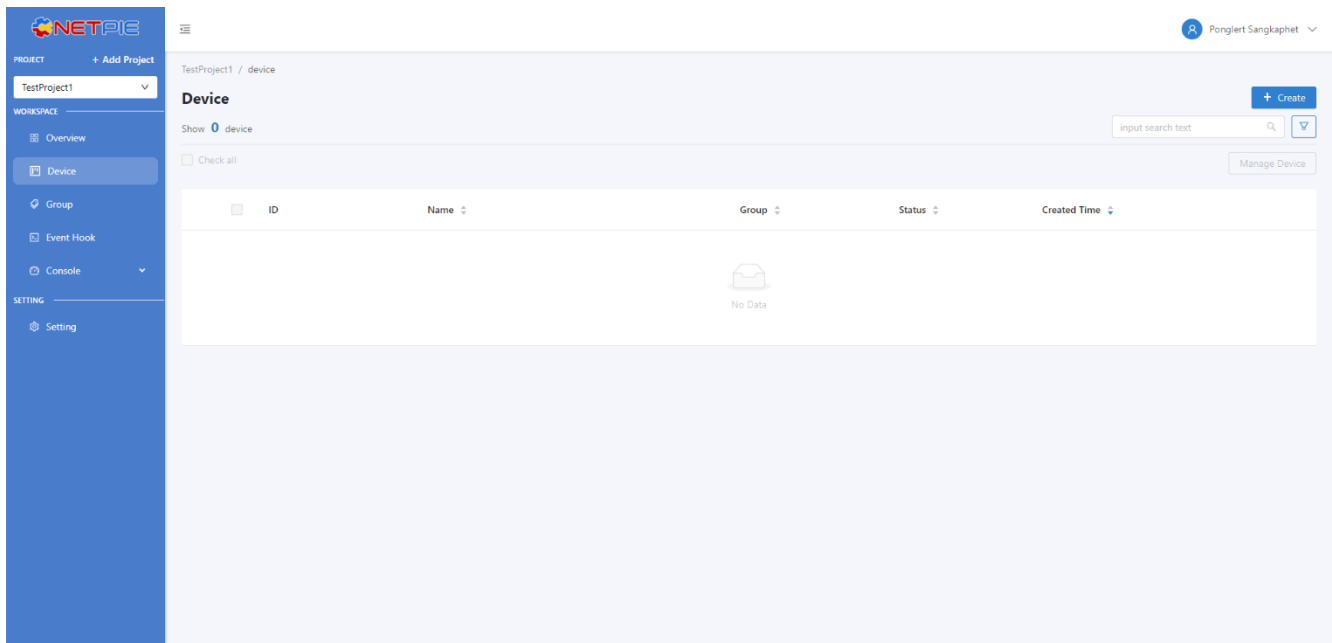


3) กรอกข้อมูล Project จากนั้นคลิกที่ปุ่ม “SAVE” ระบบจะทำการสร้าง Project ให้

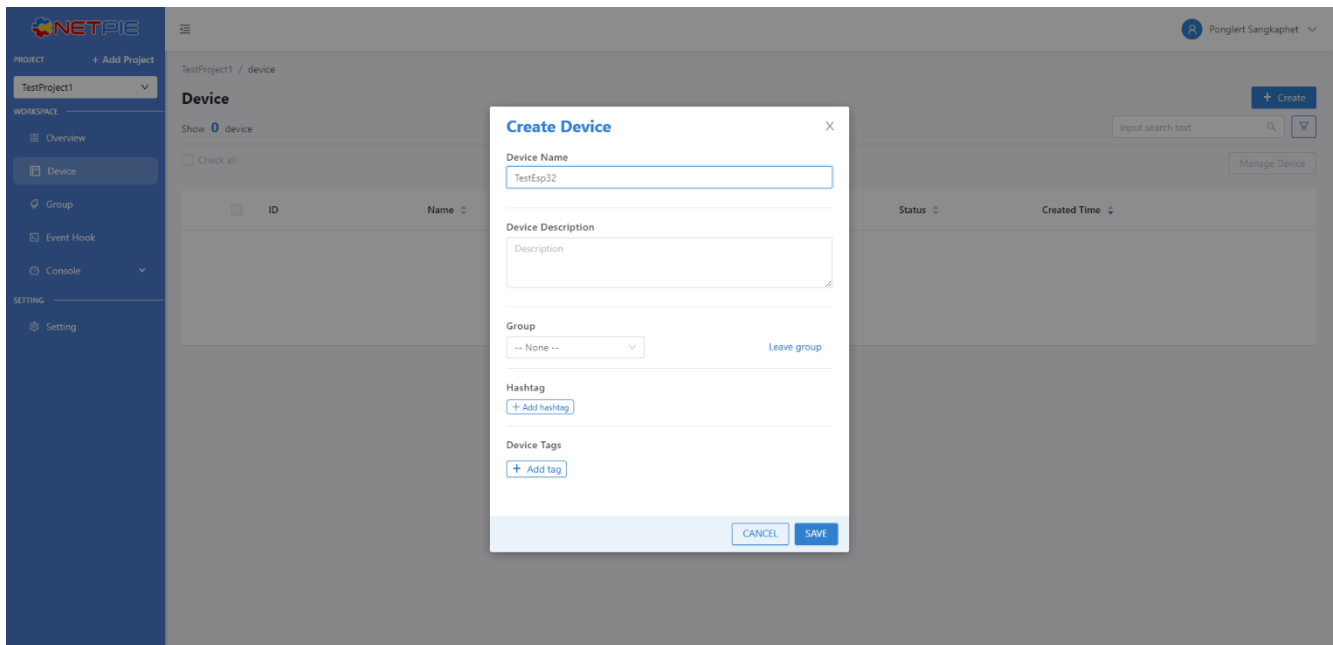


สร้าง Device

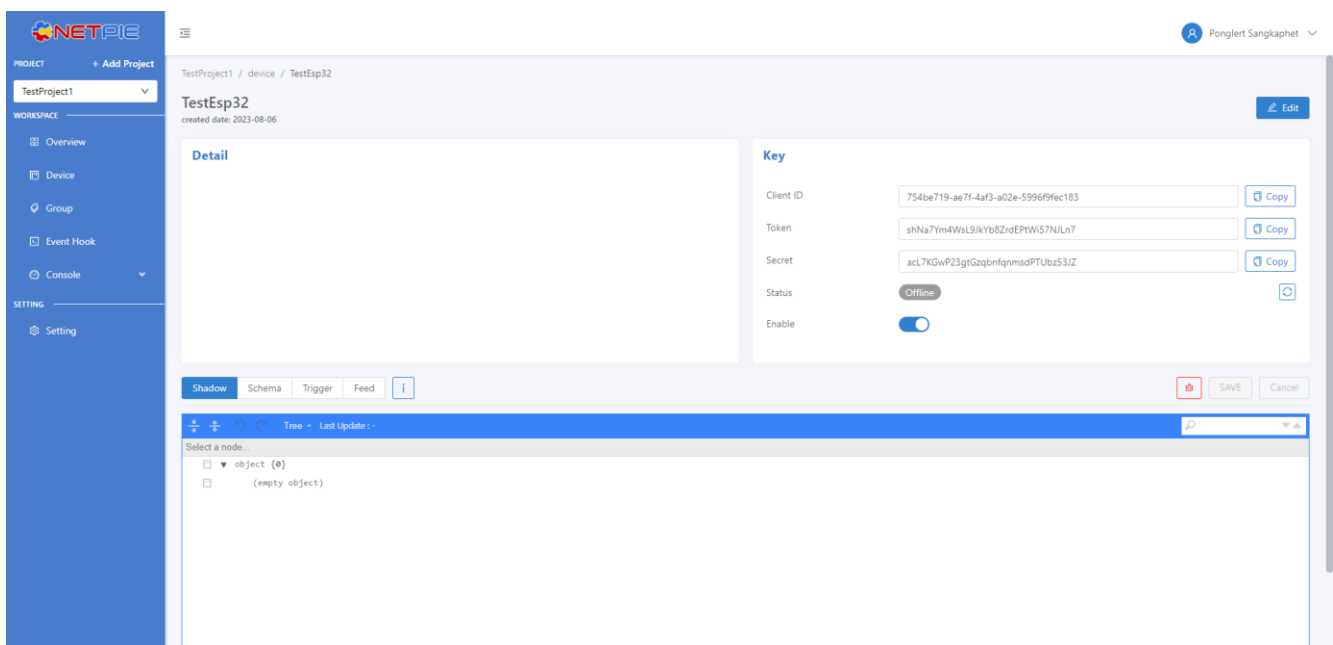
4) เมื่อเข้ามาในแต่ละ Project แล้วจะปรากฏหน้าจอแสดงข้อมูลภาพรวมของ Project ให้ทำการสร้าง Device โดยคลิกที่เมนู “Device” ทางด้านขวามือ



5) คลิกที่ + Create



6) คลิกเข้าไปที่ Device แต่ละรายการจะปรากฏหน้าจอแสดงรายละเอียดข้อมูลต่าง ๆ ของ Device เหล่านั้น รวมถึง Key, Token และ Secret ที่จะนำไปใช้เพื่อให้ Device สามารถเชื่อมต่อเข้ามายัง Platform ได้

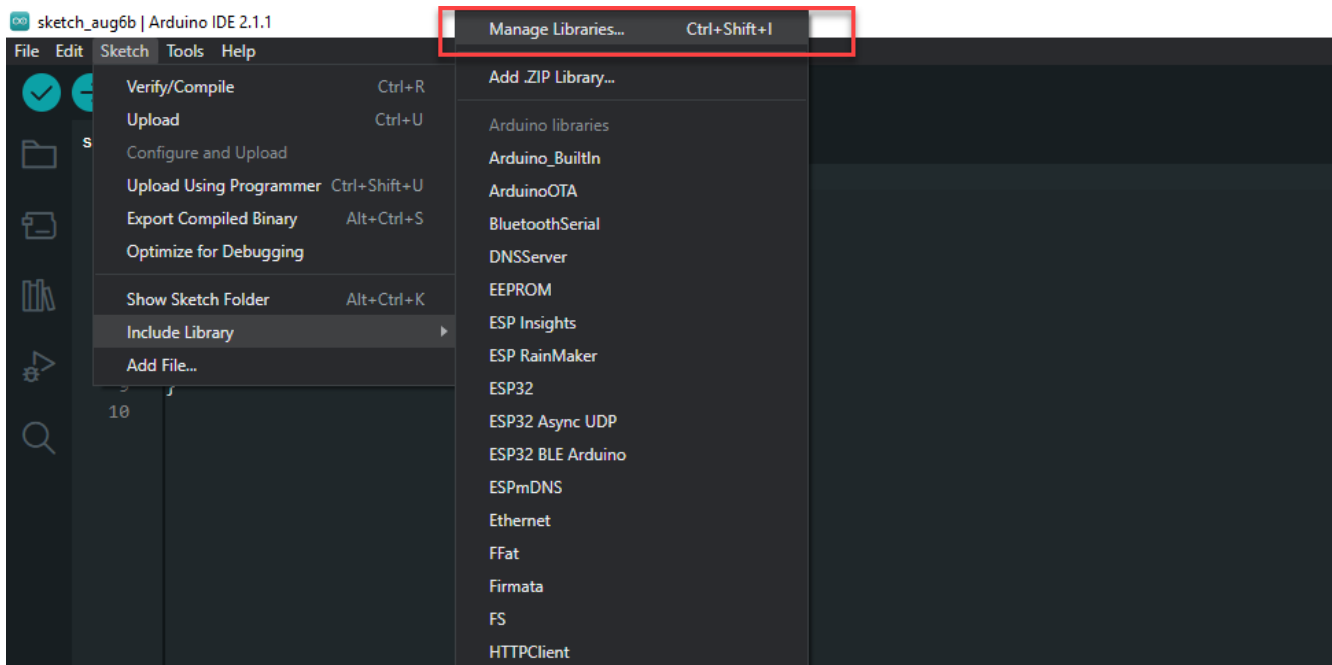


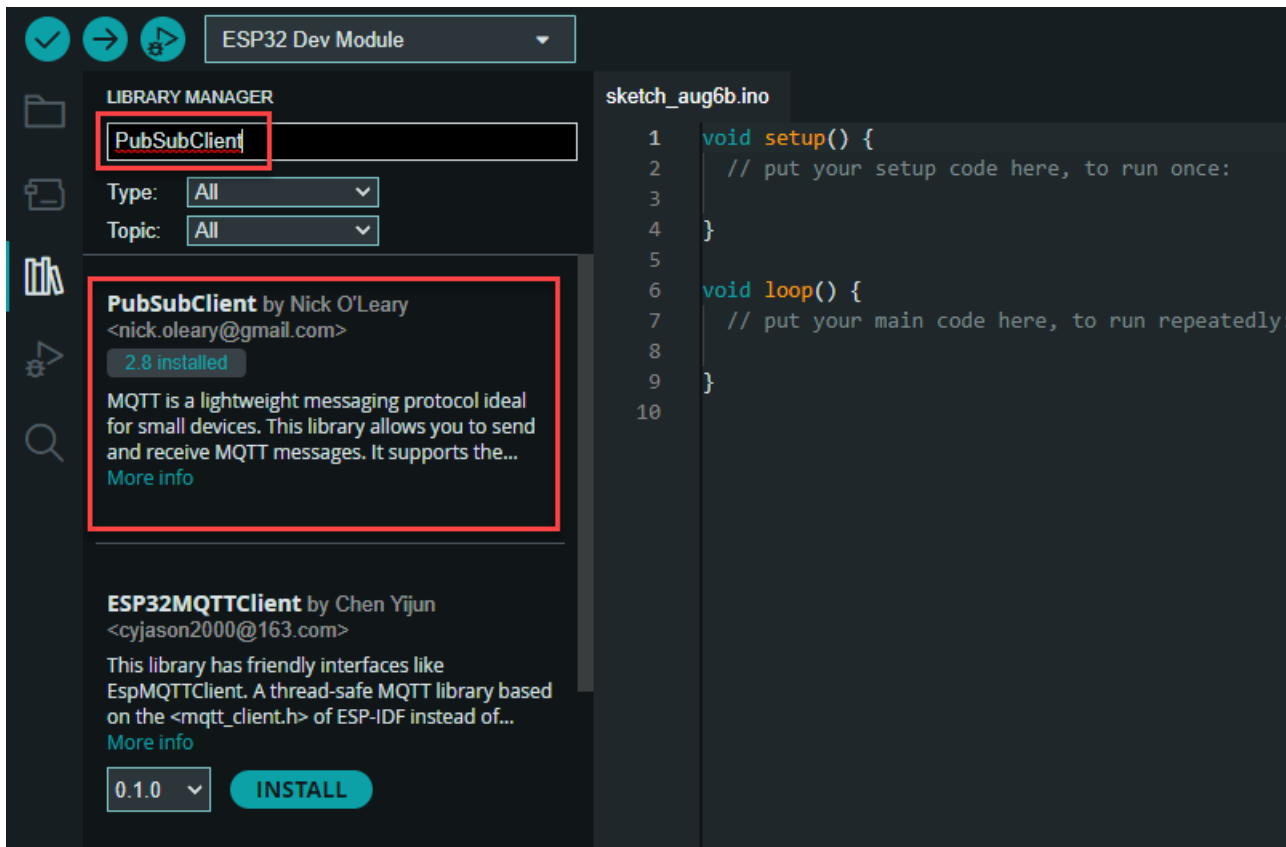
Key ใช้สำหรับการเชื่อมต่อของ Device มายัง Platform จะต้องใช้ 4 Parameters คือ Host, Client id, Username และ Password โดยดูข้อมูลที่จะนำมาใช้ สามารถระบุค่าได้ดังนี้

Host	broker.netpie.io
Port	1883
Client ID	Client ID ของ Device ที่สร้างขึ้นใน NETPIE
Username	Token ของ Device ที่สร้างขึ้นใน NETPIE
Password	Secret ของ Device ที่สร้างขึ้นใน NETPIE

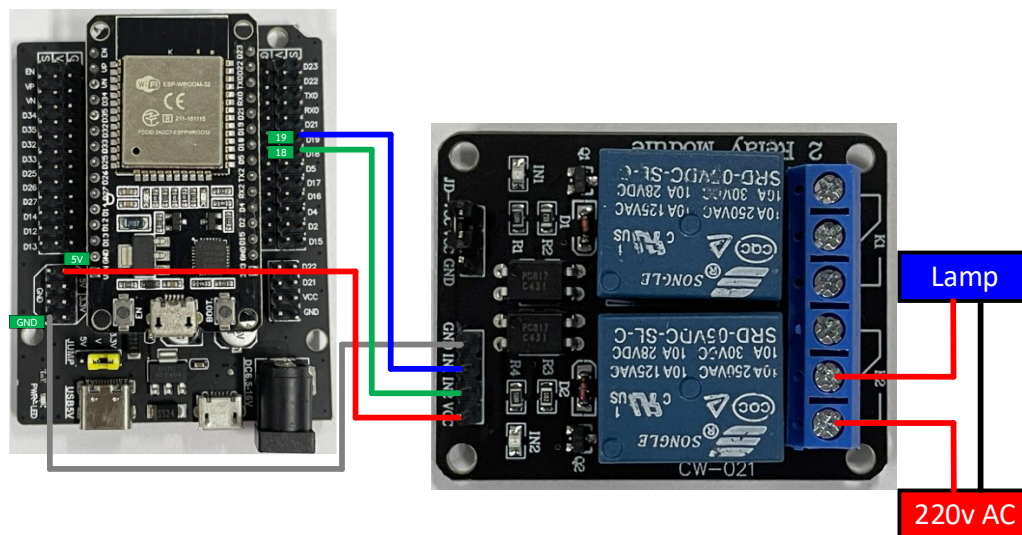
เขียนโปรแกรม Project Esp32 เชื่อมต่อ NETPIE เพื่อควบคุม Relay

7) เปิดโปรแกรม Arduino IDE ให้ติดตั้ง PubSubClient library





8) ต่อวงจรระหว่างบอร์ดกับโมดูลรีเลย์



9) ให้ทำการเขียนโปรแกรมส่วนที่ 1 ดังนี้

```
#include <WiFi.h>
#include <PubSubClient.h>
const char* ssid = "CSUBRUIOT02"; //ชื่อ Wifi
const char* password = "asdf+1234"; //รหัส Wifi
const char* mqtt_server = "broker.netpie.io";
const int mqtt_port = 1883;
const char* mqtt_Client = "73eaf3e2-0efd-48e1-b83e-b26b10d6958c"; //ข้อมูล Client ID
const char* mqtt_username = "zoxyBc53GubFaW8a25DMWmHBmZhGAPpU"; //ข้อมูล Token
const char* mqtt_password = "yvMmhSqvrvpL3yZ4x45NzBxUomGgreER"; //ข้อมูล Secret
WiFiClient espClient;
PubSubClient client(espClient);
char msg[250];
#define relaypin1 19
#define relaypin2 18
int relay1=0;
int relay2=0;
```

10) ให้ทำการเขียนโปรแกรมส่วนที่ 2 สร้างฟังก์ชัน reconnect ดังนี้

```
void reconnect() {
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    if (client.connect(mqtt_Client, mqtt_username, mqtt_password)) {
      Serial.println("connected");
      client.subscribe("@msg/#");
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      delay(5000);
    }
  }
}
```

11) ให้ทำการเขียนโปรแกรมส่วนที่ 4 สร้างฟังก์ชัน getMsg ดังนี้

```
void getMsg(String topic_, String message_) {
  if (topic_ == "@msg/relay1") {
    if (message_ == "1") {
      relay1 = 1;
    } else if (message_ == "0") {
      relay1 = 0;
    }
    String data = "{\"data\":{\"relay1\": " + String(relay1) + "}"};
    Serial.println(data);
    data.toCharArray(msg, (data.length() + 1));
    client.publish("@shadow/data/update", msg);
  } else if (topic_ == "@msg/relay2") {
    if (message_ == "1") {
      relay2 = 1;
    } else if (message_ == "0") {
      relay2 = 0;
    }
  }
}
```

```
String data = "{\"data\":{\"relay2\": \" + String(relay2) + \"}}\"";
Serial.println(data);
data.toCharArray(msg , (data.length() + 1));
client.publish("@shadow/data/update", msg);
}
}
```

12) ให้ทำการเขียนโปรแกรมส่วนที่ 3 สร้างฟังก์ชัน callback ดังนี้

```
void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  String message;
  for (int i = 0; i < length; i++) {
    message = message + (char)payload[i];
  }
  Serial.println(message);
  getMsg(topic, message);
}
```

13) ให้ทำการเขียนโปรแกรมส่วนที่ 4 ในฟังก์ชัน setup ดังนี้

```
void setup() {
  pinMode(relaypin1, OUTPUT);
  pinMode(relaypin2, OUTPUT);
  Serial.begin(9600);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callback);
  delay(1000);
  if(!client.connected()) {
    reconnect();
  }
}
```

14) ให้ทำการเขียนโปรแกรมส่วนที่ 4 ในฟังก์ชัน loop ดังนี้

```
void loop() {
  if (!client.connected()) {
    reconnect();
  }
  if(relay1 == 1){
    if(!digitalRead(relaypin1)){
      digitalWrite(relaypin1, HIGH);
    }
  }
  }else{
```

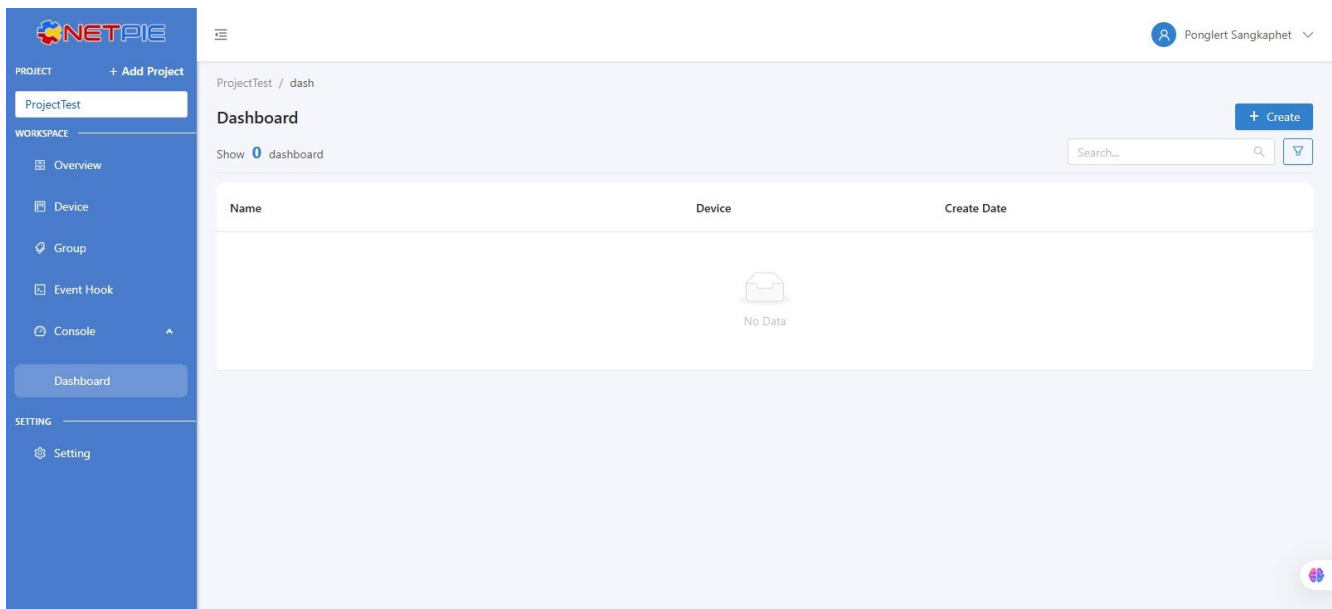
```

    if(digitalRead(relaypin1)){
        digitalWrite(relaypin1, LOW);
    }
}
if(relay2 == 1){
    if(!digitalRead(relaypin2)){
        digitalWrite(relaypin2, HIGH);
    }
}else{
    if(digitalRead(relaypin2)){
        digitalWrite(relaypin2, LOW);
    }
}
}
client.loop();
}

```

สร้าง Dashboard ควบคุมบน Netpie

15) คลิกที่เมนู “Dashboard” ในแถบซ้ายมือ ก็จะปรากฏหน้าจอแสดงรายการชื่อ Dashboard ทั้งหมดที่เคยสร้างไว้ภายใน Project นั้นๆ



16) ให้คลิกที่ปุ่ม “Create” มุมบนขวามือของหน้าจอ ก็จะปรากฏหน้าจอสำหรับให้กรอกข้อมูลทั่วไปของ Dashboard

The screenshot shows the NETPIE web interface. On the left is a sidebar with navigation options: PROJECT (+ Add Project), WORKSPACE (Overview, Device, Group, Event Hook, Console, Dashboard), and SETTING (Setting). The main area is titled 'Dashboard' and shows 'Show 0 dashboard'. A modal window titled 'Create Dashboard' is centered, with fields for 'Dashboard Name' (containing 'Testboard') and 'Dashboard Description' (containing 'Description'). At the bottom of the modal are 'Cancel' and 'SAVE' buttons. In the top right of the main area, there is a '+ Create' button and a search bar.

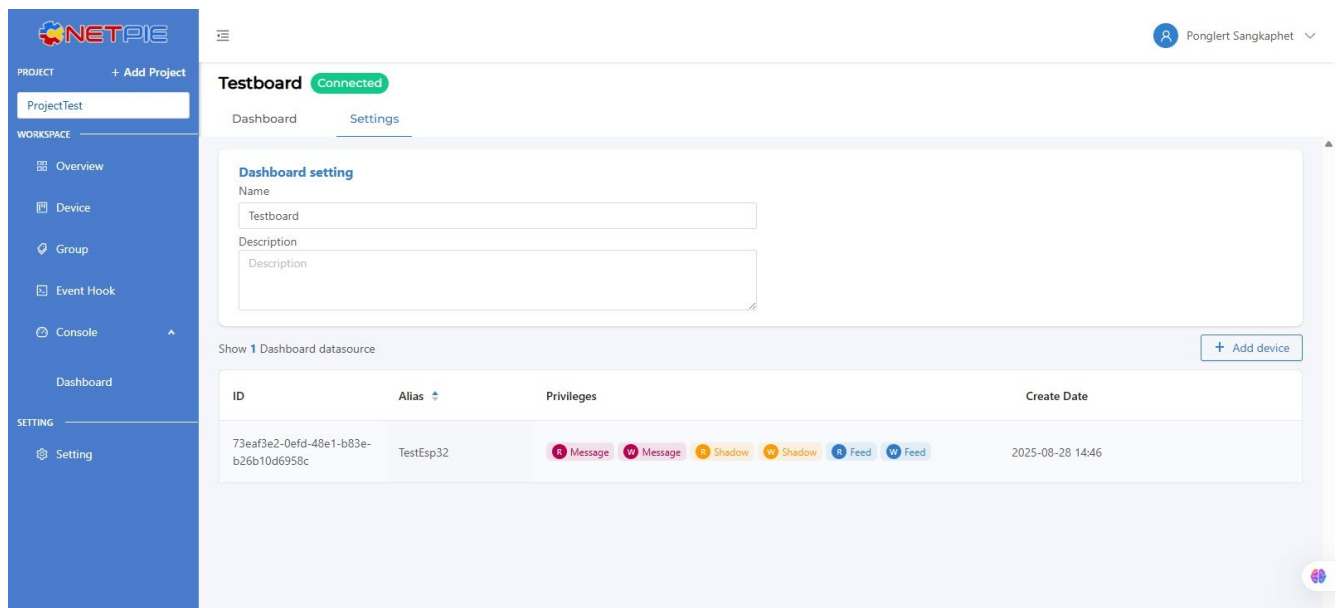
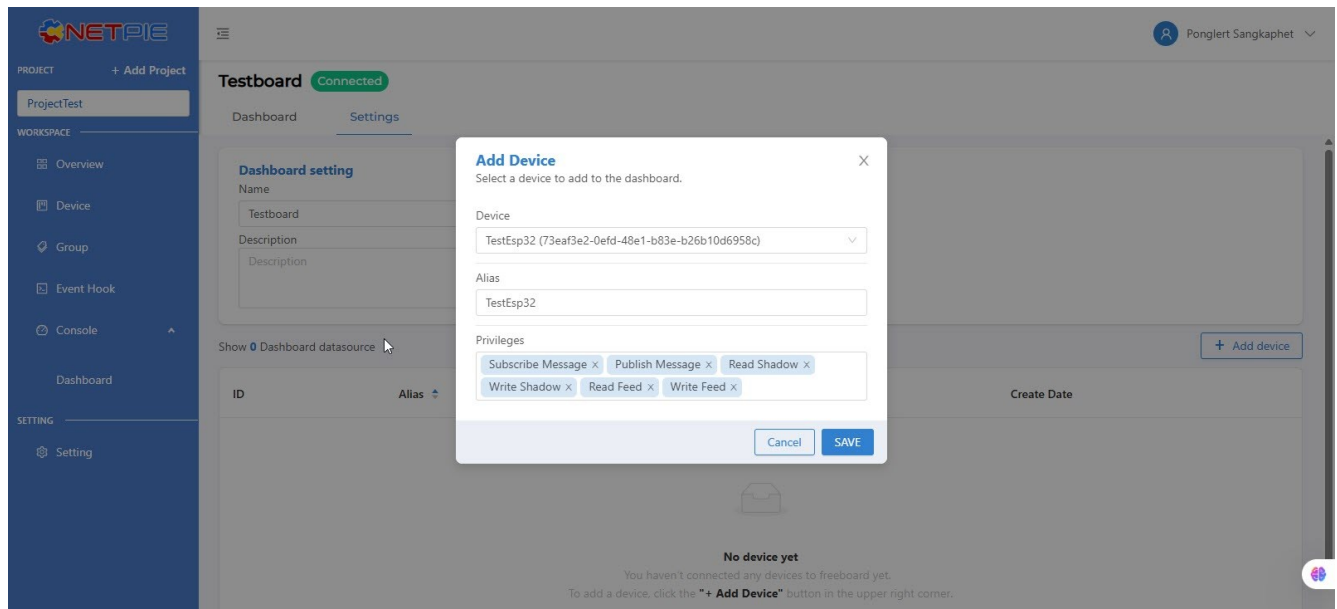
17) เมื่อกรอกข้อมูลเรียบร้อยแล้วให้คลิกที่ปุ่ม “Create” ก็จะปรากฏรายการ Dashboard ใหม่ที่เพิ่งสร้างขึ้นมา

The screenshot shows the NETPIE web interface after the dashboard has been created. The sidebar is the same. The main area is titled 'Dashboard' and now shows 'Show 1 dashboard'. A table lists the dashboard:

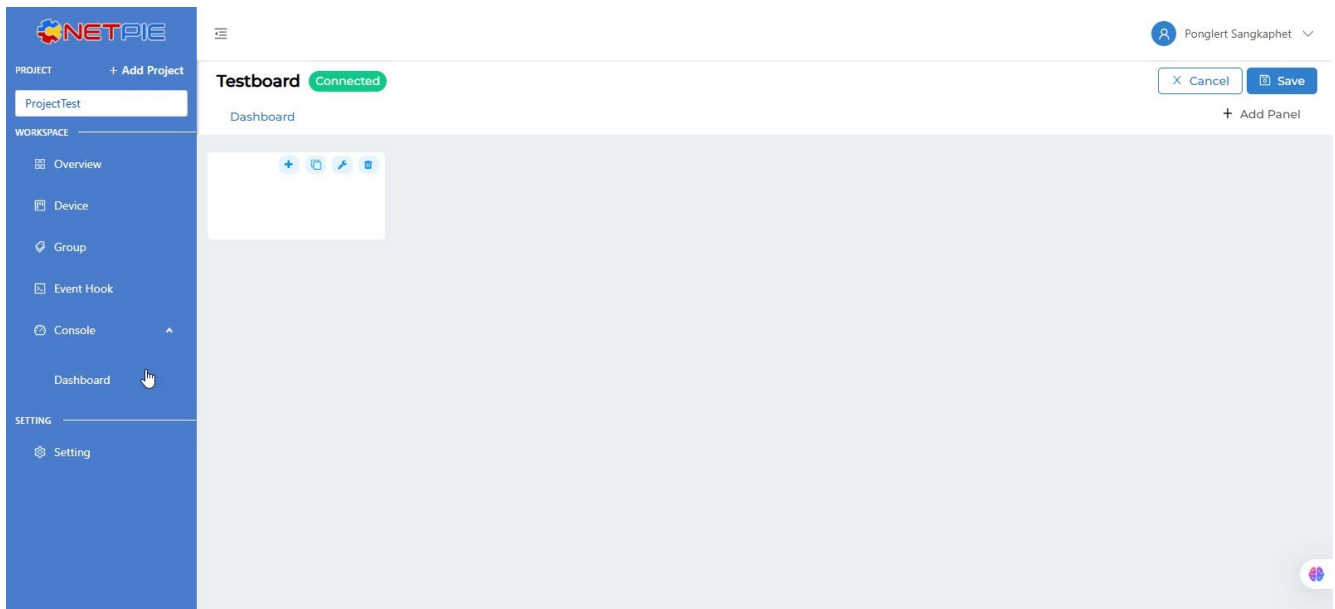
Name	Device	Create Date
Testboard	-	2025-08-28 14:41

The '+ Create' button is still present in the top right. The search bar is also visible.

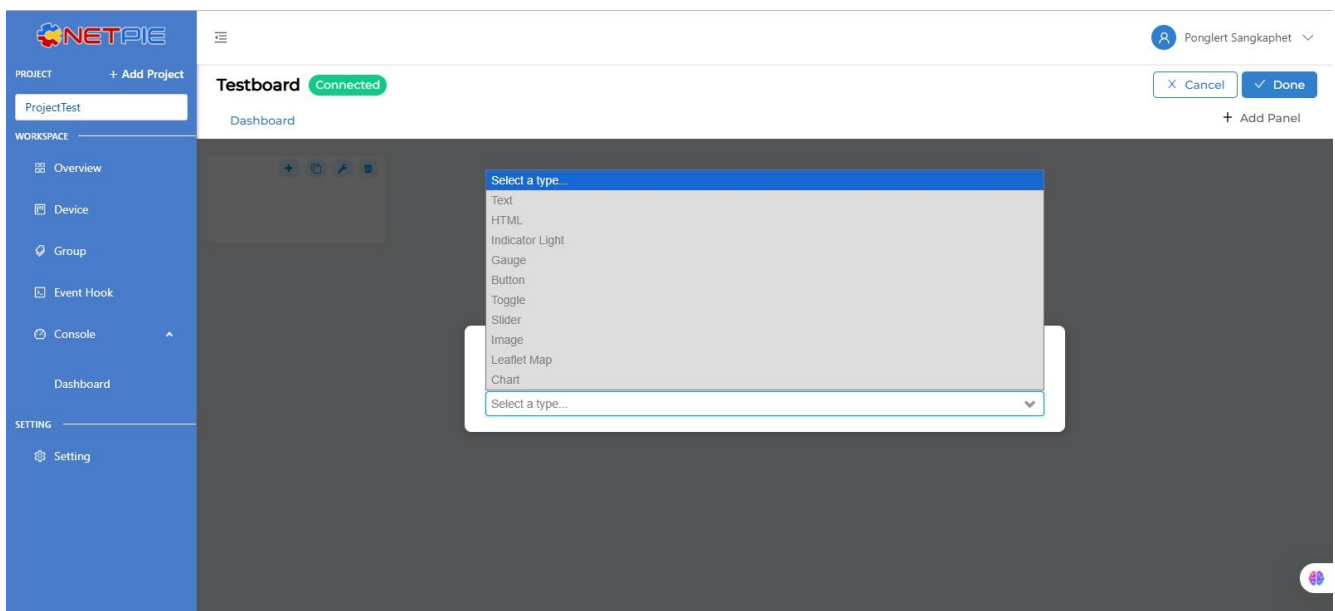
ให้เลือก Dashboard ที่ต้องการ แล้วไปที่ Setting -> Add source และเลือก Device และ Privileges



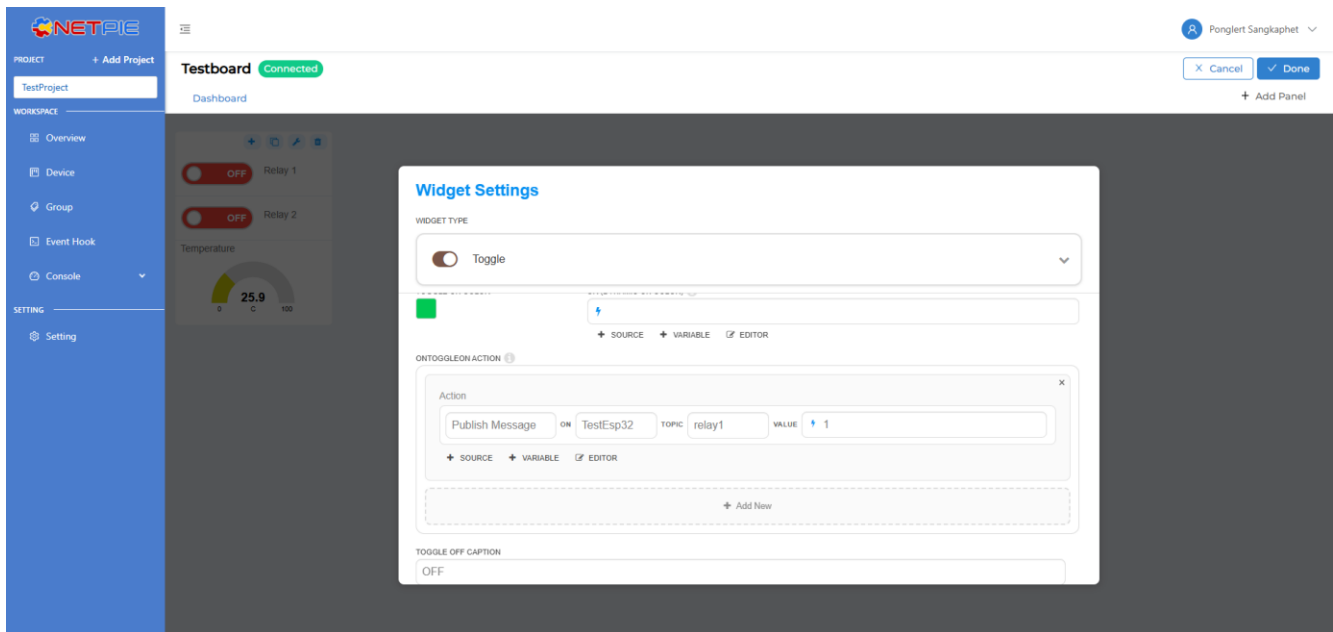
ให้คลิก Edit และ คลิกเพิ่ม Add Panel



คลิก + ที่ Panel ที่สร้างมา และ เลือกเป็น Toggle

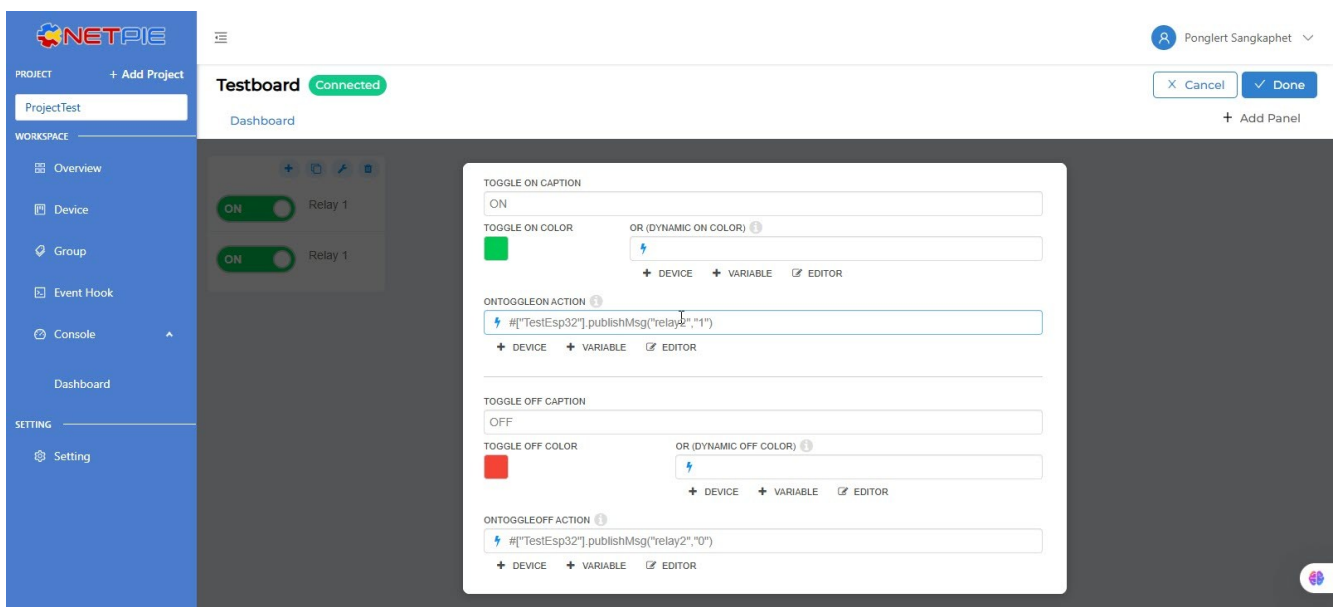


20) เพิ่ม Toggle สำหรับ Relay 1 และข้อมูลลงไป



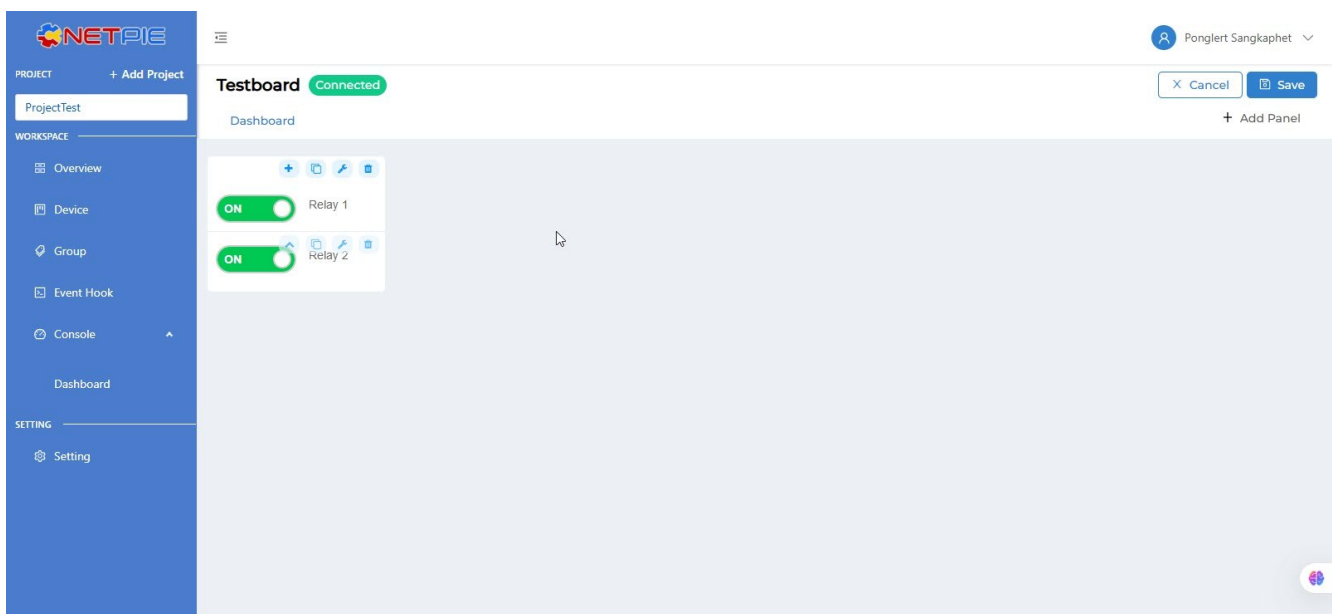
- LABEL = Relay 1
 - TOGGLE STATE = #["TestEsp32"]["shadow"]["relay1"]==1
 - ONTOGGLEON ACTION = Publish Message ON TestEsp32 TOPIC relay1 VALUE 1
 - ONTOGGLEOFF ACTION = Publish Message ON TestEsp32 TOPIC relay1 VALUE 0
- แล้วคลิก Done

21) เพิ่ม Toggle สำหรับ Relay 2 และข้อมูลลงไป



- LABAL = Relay 2
 - TOGGLE STATE = #["TestEsp32"]["shadow"]["relay2"]==1
 - ONTOGGLEON ACTION = Publish Message ON TestEsp32 TOPIC relay2 VALUE 1
 - ONTOGGLEOFF ACTION = Publish Message ON TestEsp32 TOPIC relay2 VALUE 0
- แล้วคลิก Done

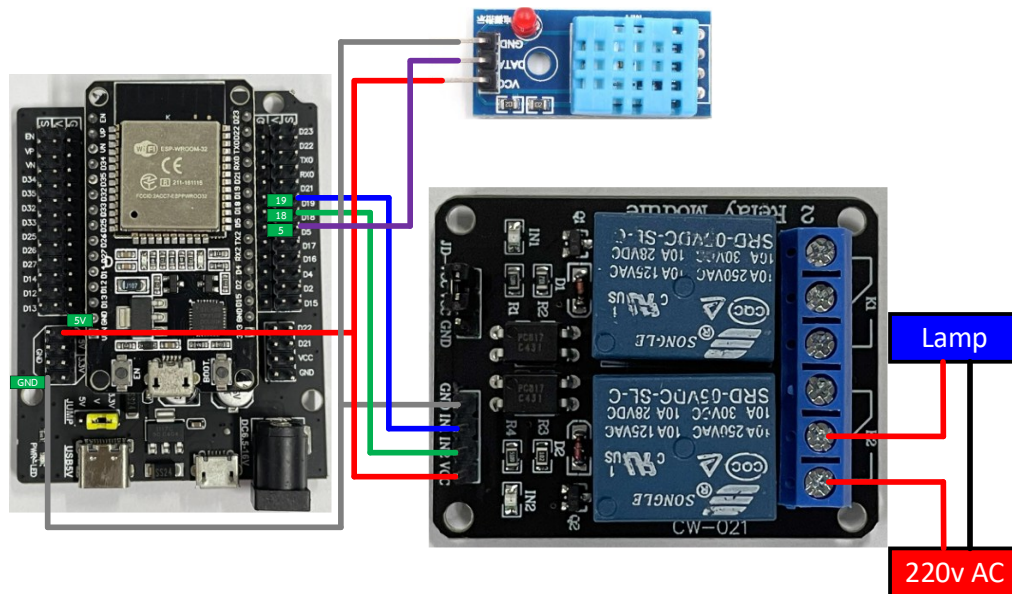
Dashboard จะแสดง TOGGLE ที่จะใช้สั่งเปิด Relay ขึ้นมา 2 ตัว



22) ให้กดปุ่ม “SAVE”

การส่งค่าเซนเซอร์ข้อมูลไปแสดงผลบน NETPIE

1) ต่อวงจรระหว่างบอร์ดกับโมดูล DHT11



2) ให้ทำการเขียนโปรแกรมส่วนที่ 1 ดังนี้

```
#include <WiFi.h>
#include <PubSubClient.h>
const char* ssid = "CSUBRUIOT02"; //ชื่อ Wifi
const char* password = "asdf+1234"; //รหัส Wifi
const char* mqtt_server = "broker.netpie.io";
const int mqtt_port = 1883;
const char* mqtt_Client = "73eaf3e2-0efd-48e1-b83e-b26b10d6958c"; //ข้อมูล Client ID
const char* mqtt_username = "zoxYBc53GubFaw8a25DMWmHBmZhGAppU"; //ข้อมูล Token
const char* mqtt_password = "yvMmhSqvrvpL3yZ4x45NzBxUomGgreER"; //ข้อมูล Secret
WiFiClient espClient;
PubSubClient client(espClient);
char msg[250];
#define relaypin1 19
#define relaypin2 18
int relay1=0;
int relay2=0;

#include "DHT.h"
#define DHTPIN 5
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
unsigned long previousTime = 0;
```

3) ให้ทำการเขียนโปรแกรมเพิ่มฟังก์ชัน setup

```
void setup() {
  pinMode(relaypin1, OUTPUT);
  pinMode(relaypin2, OUTPUT);

  dht.begin();

  Serial.begin(9600);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callback);
  delay(1000);
  if(!client.connected()) {
    reconnect();
  }
}
```

4) ให้ทำการเขียนโปรแกรมเพิ่มฟังก์ชันการส่งข้อมูลไป NETPIE

```
void setMsgTemp(String temp){
  String data = "{\"data\":{\"temp\": " + temp + "}}";
  Serial.println(data);
  data.toCharArray(msg , (data.length() + 1));
  client.publish("@shadow/data/update", msg);
}
```

5) ให้ทำการเขียนโปรแกรมเพิ่มฟังก์ชันใน loop

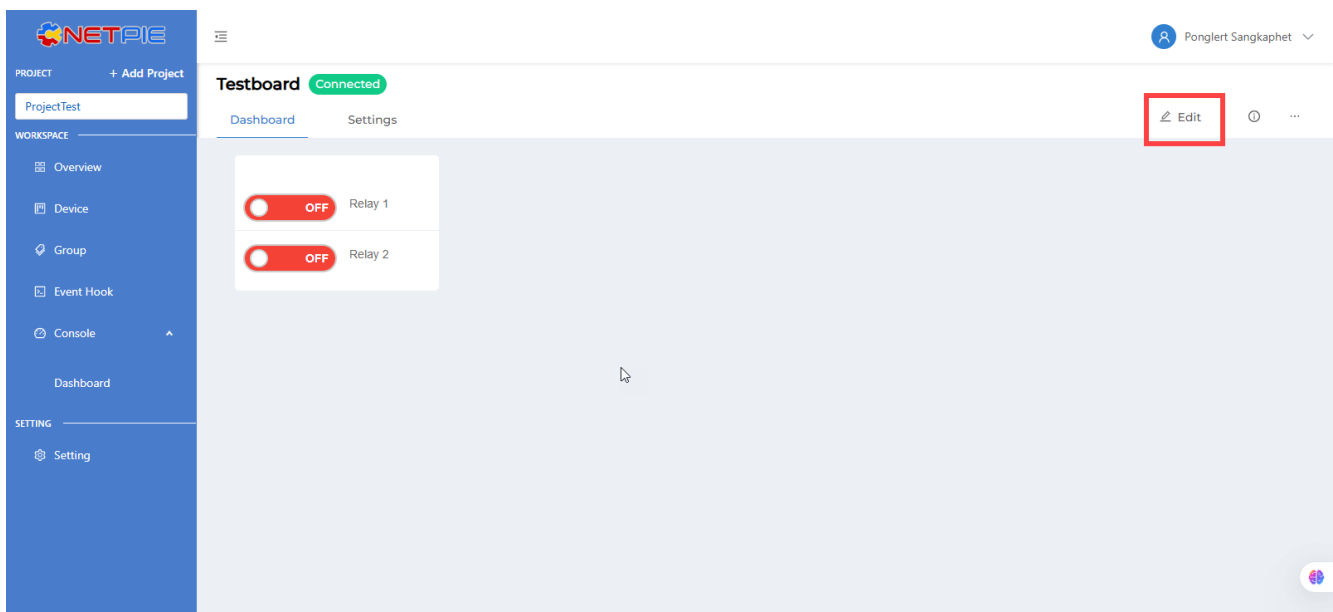
```
void loop() {
  if (!client.connected()) {
    reconnect();
  }
  if(relay1 == 1){
    if(!digitalRead(relaypin1)){
      digitalWrite(relaypin1, HIGH);
    }
  }else{
    if(digitalRead(relaypin1)){
      digitalWrite(relaypin1, LOW);
    }
  }
  if(relay2 == 1){
    if(!digitalRead(relaypin2)){
      digitalWrite(relaypin2, HIGH);
    }
  }
}
```

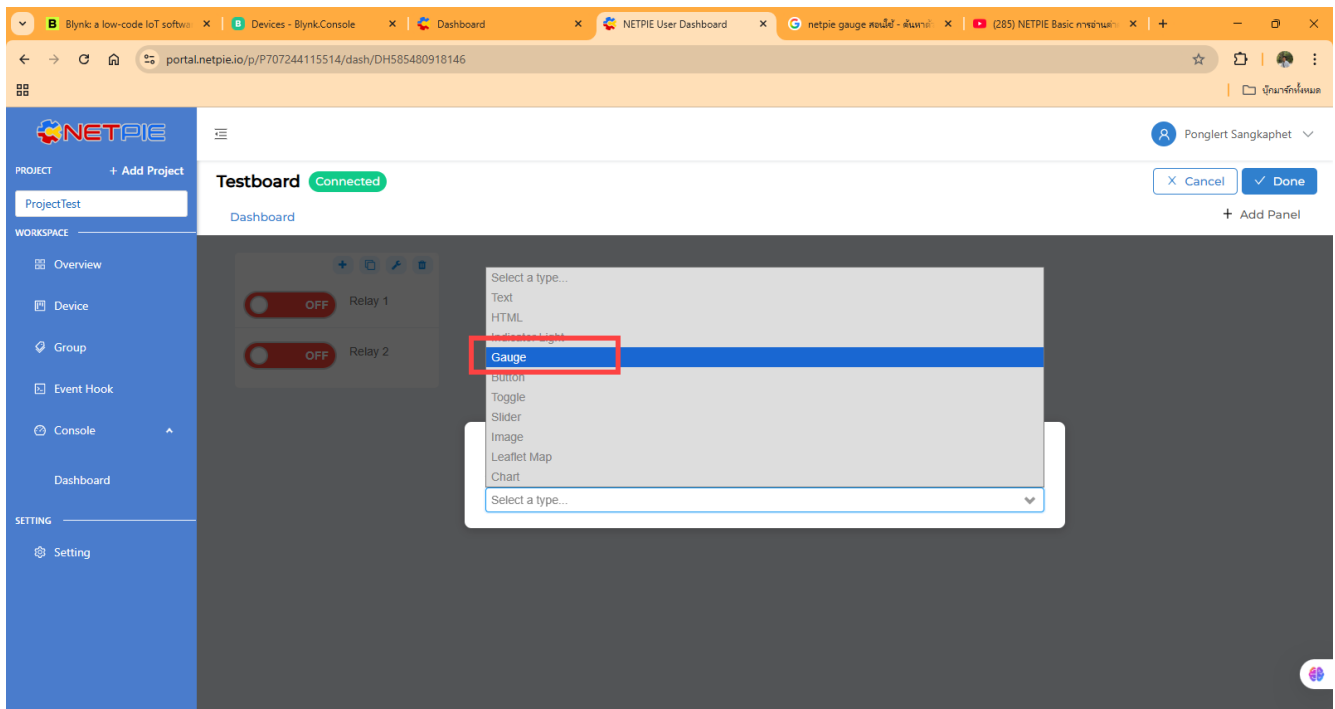
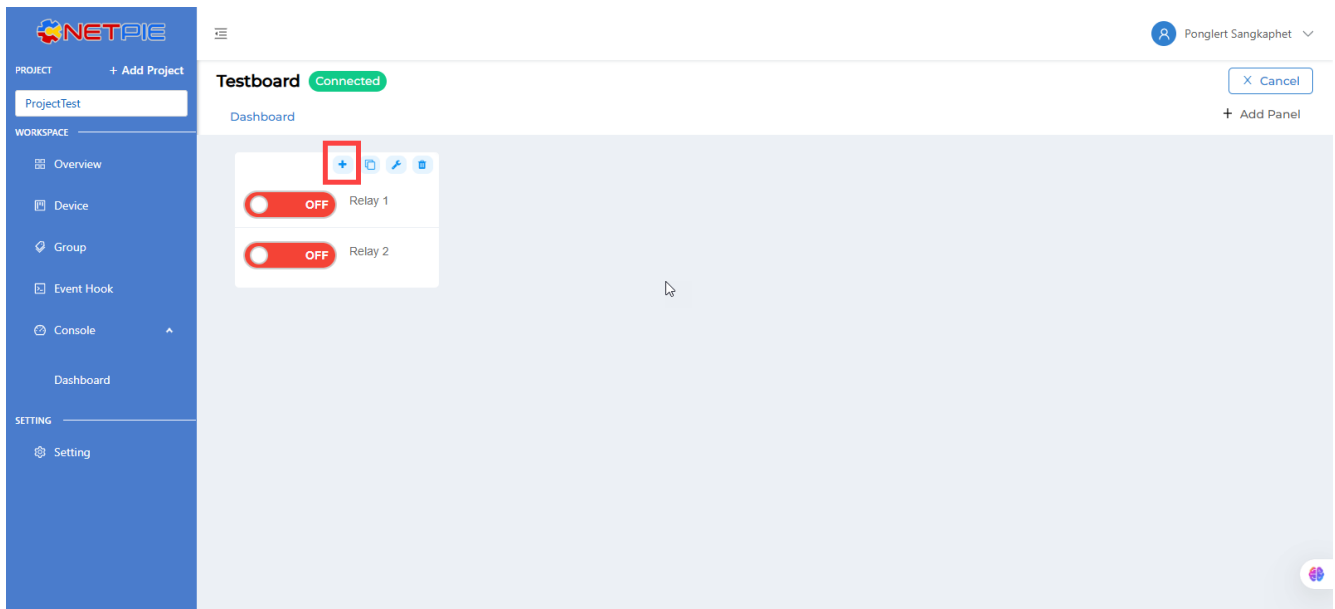
```

}else{
  if(digitalRead(relaypin2)){
    digitalWrite(relaypin2, LOW);
  }
}
if((millis() - previousTime) >= 2000){
  previousTime = millis();
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  if (isnan(h) || isnan(t)) {
    Serial.println(F("Failed to read from DHT sensor!"));
  }else{
    setMsgTemp(String(t));
    Serial.print(F("Humidity: "));
    Serial.print(h);
    Serial.print(F("% Temperature: "));
    Serial.print(t);
    Serial.println(F("°C "));
  }
}
}
client.loop();
}

```

6) ให้ทำการเพิ่ม gauge ใน Dashboard ดังรูป





Gauge

TITLE
Temperature

VALUE
#[\"TestEsp32\"][\"shadow\"][\"temp\"]
+ DEVICE + VARIABLE EDITOR

UNITS
C

MINIMUM
0

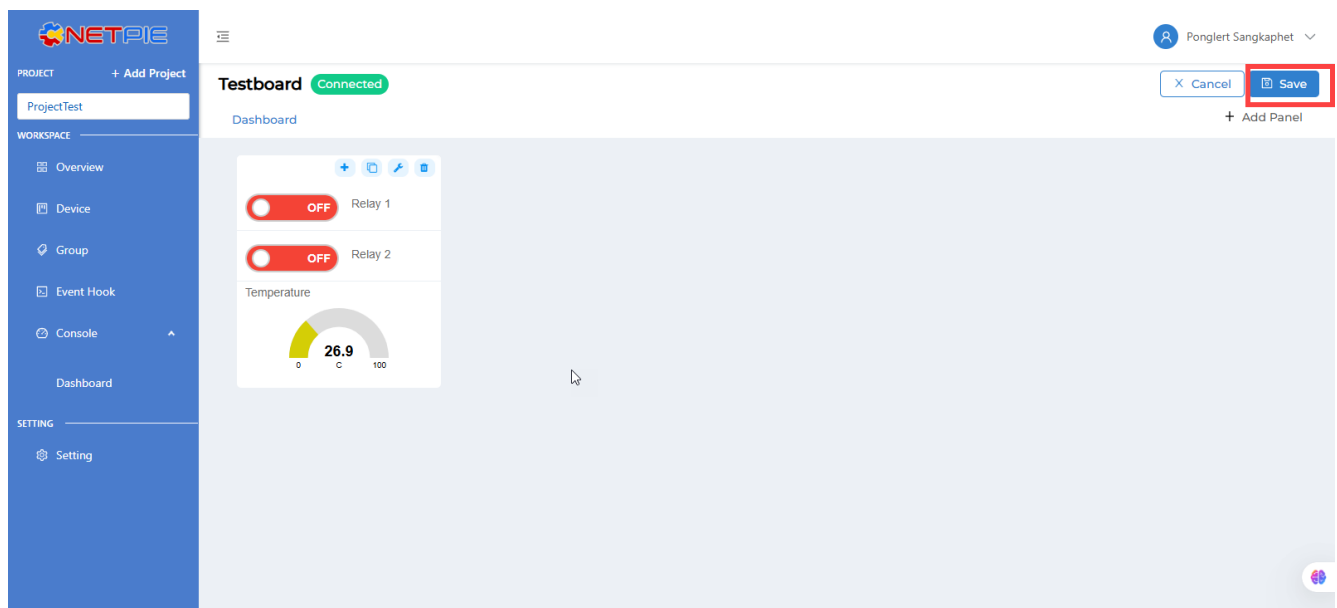
MAXIMUM
100

DECIMAL DIGIT
2

HUMAN FRIENDLY NUMBER
YES

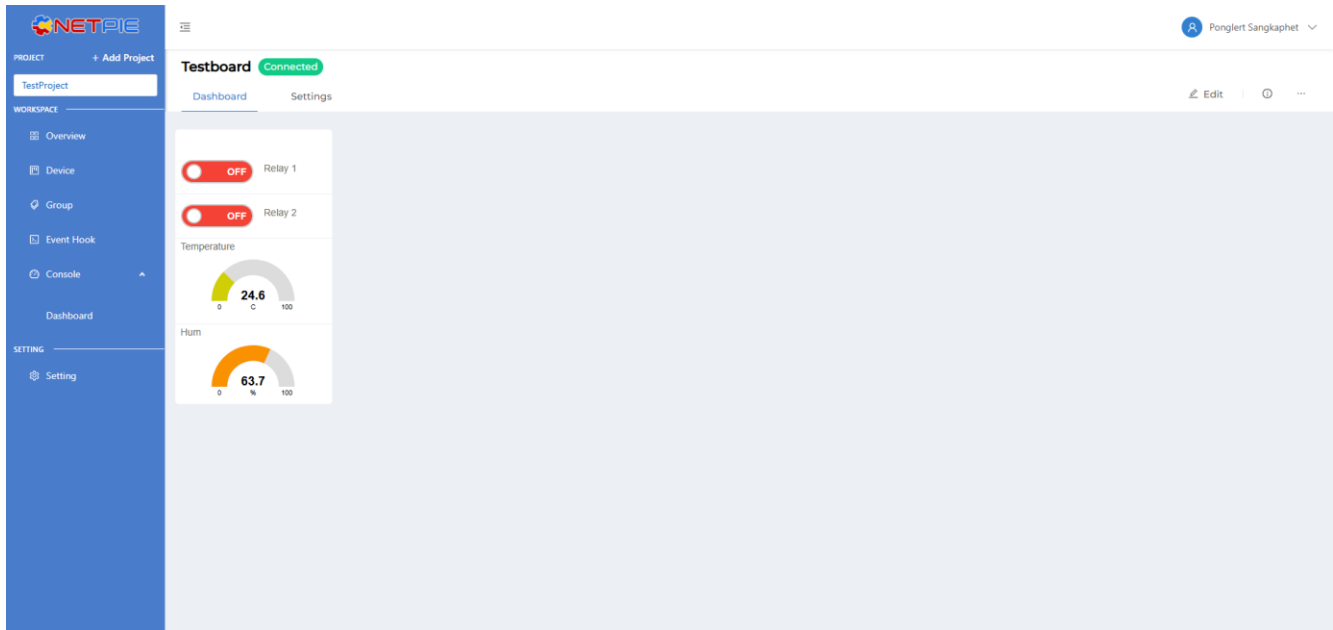
- TITLE = Temperature
- VALUE = #[\"TestEsp32\"][\"shadow\"][\"temp\"]
- UNITS = C
- MINIMUM = 0

แล้วคลิก Done



แบบฝึกหัด

ทำการแสดงค่าความชื้นในอากาศจากเซนเซอร์บน NETPIE แสดงที่เว็บ Dashboard โดยให้ทำการเขียนโปรแกรมเพิ่มจากโค้ดก่อนหน้านี้ และตั้งค่าต่าง ๆ เพิ่มในแพลตฟอร์ม NETPIE ให้ได้ผลลัพธ์ดังรูป



ตอนที่ 8

การพัฒนาแอปพลิเคชันบนมือถือควบคุมอุปกรณ์ IoT

React Native คือเทคโนโลยีสำหรับสร้างแอปมือถือ (ทั้ง Android และ iOS) โดยเขียนด้วยภาษา JavaScript ทำให้เขียนครั้งเดียวใช้ได้ทั้งสองระบบ

Expo คือชุดเครื่องมือที่ช่วยให้การเริ่มต้นเขียน React Native ง่ายขึ้นมาก ไม่ต้องตั้งค่ายุ่งยาก

Expo Snack คือเว็บไซต์ (snack.expo.dev) สำหรับเขียนโค้ดแอปได้จากในเบราว์เซอร์โดยตรง ไม่ต้องติดตั้งอะไรบนเครื่อง เหมาะกับการเรียนรู้และทดลอง

Expo Go คือแอปบนมือถือที่ใช้ "เปิดดู" โปรเจกต์ที่เราเขียนไว้บน Snack แบบเรียลไทม์ พอแก้ไขโค้ดบนเว็บ หน้าจอในมือถือก็อัปเดตตามทันที

ขั้นตอนที่ 1: สมัครใช้งาน Expo Snack

1. เปิดเว็บเบราว์เซอร์บนคอมพิวเตอร์ แล้วพิมพ์ที่อยู่ snack.expo.dev

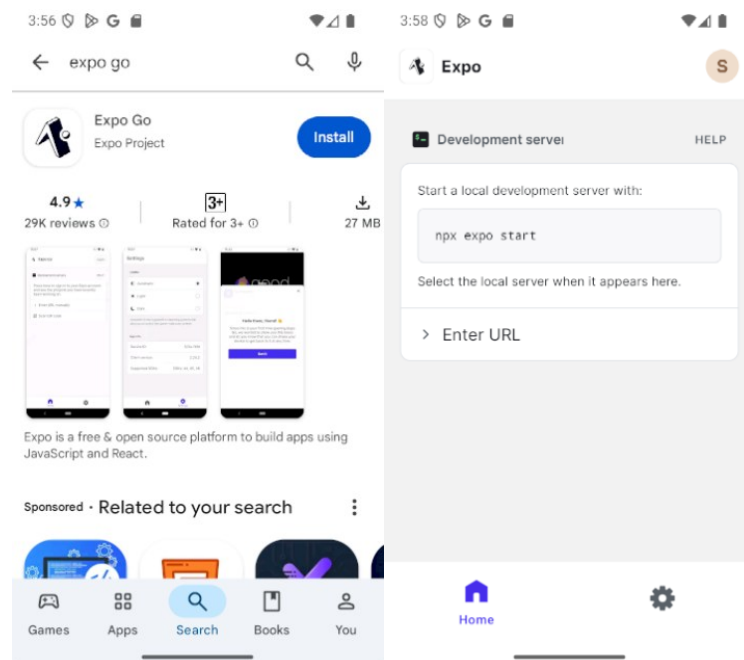


2. เมื่อหน้าเว็บโหลดเสร็จ ให้มองมุมขวาด้านบน จะเห็นปุ่ม "Sign Up" (สมัครสมาชิก) หรือ "Log In" คลิกที่ "Sign Up"
3. เลือกวิธีสมัครที่สะดวก: กรอกอีเมลและตั้งรหัสผ่านเอง หรือกดปุ่ม "Continue with Google" / "Continue with GitHub" เพื่อใช้บัญชีที่มีอยู่แล้ว
4. หากสมัครด้วยอีเมล ให้เปิดกล่องอีเมลของคุณ ค้นหาเมลยืนยันจาก Expo แล้วคลิกลิงก์ยืนยัน (บางครั้งอาจอยู่ในโฟลเดอร์ Junk/Spam)
5. เมื่อเข้าสู่ระบบสำเร็จ จะเห็นชื่อบัญชี/รูปโปรไฟล์ที่มุมขวาด้านบน ถือว่าสมัครและล็อกอินเรียบร้อยแล้ว



ขั้นตอนที่ 2: ติดตั้งแอป Expo Go บนมือถือ สำหรับ Android

1. เปิดแอป "Play Store" (Google Play) บนโทรศัพท์
2. แตะช่องค้นหาด้านบน พิมพ์คำว่า Expo Go
3. เลือกแอปชื่อ "Expo Go" (ผู้พัฒนาคือ Expo) แล้วกดปุ่ม "ติดตั้ง" (Install)
4. รอจนติดตั้งเสร็จ แล้วเปิดแอปขึ้นมา



สำหรับ iPhone (iOS)

1. เปิดแอป "App Store" บน iPhone

2. แตะแท็บ "ค้นหา" (Search) ด้านล่างขวา แล้วพิมพ์ Expo Go
3. เลือกแอป "Expo Go" แล้วกดปุ่มดาวน์โหลด (รูปเมฆ หรือ Get) อาจต้องยืนยันด้วย Face ID / Touch ID / รหัสผ่าน Apple ID
4. รอจนติดตั้งเสร็จ แล้วเปิดแอปขึ้นมา

ขั้นตอนที่ 3: สร้างโปรเจกต์และเขียนข้อความแรก

1. กลับมาที่เว็บ snack.expo.dev บนคอมพิวเตอร์ (ล็อกอินอยู่)
2. ระบบจะสร้างโปรเจกต์ตัวอย่างให้อัตโนมัติ มองด้านซ้ายจะเห็นรายการไฟล์ ให้คลิกไฟล์ชื่อ App.js
3. ในกรอบเขียนโค้ดตรงกลาง ให้ลบโค้ดเดิมทั้งหมดออก แล้วพิมพ์ (หรือคัดลอก) โค้ดด้านล่างนี้แทน:

```
import React from 'react';
import { Text, View, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>สวัสดี! นี่คือแอปแรกของฉัน 🍌</Text>
      <Text style={styles.subtitle}>Hello World from Expo</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#f0f6ff',
    padding: 24,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#1f4e79',
    textAlign: 'center',
  },
  subtitle: {
    fontSize: 16,
    color: '#555',
    marginTop: 12,
  },
});
```

*Snack จะบันทึกและประมวลผลโค้ดให้อัตโนมัติ ไม่ต้องกดปุ่ม Save เอง



ขั้นตอนที่ 4: แสดงผลงานโทรศัพท์ของคุณ

ตอนนี้ถึงเวลานำโค้ดขึ้นไปแสดงบนมือถือจริง ทำได้ด้วยการสแกน QR Code:

1. บนเว็บ Snack มองด้านขวา จะมีแท็บ/ปุ่มชื่อ "My Device" หรือ "Run" ให้คลิกเพื่อแสดง QR Code
2. เปิดแอป Expo Go บนมือถือ
3. Android: ในแอป Expo Go จะมีเมนู "Scan QR code" ให้แตะ แล้วเล็งกล้องไปที่ QR บนหน้าจอ

คอมพิวเตอร์

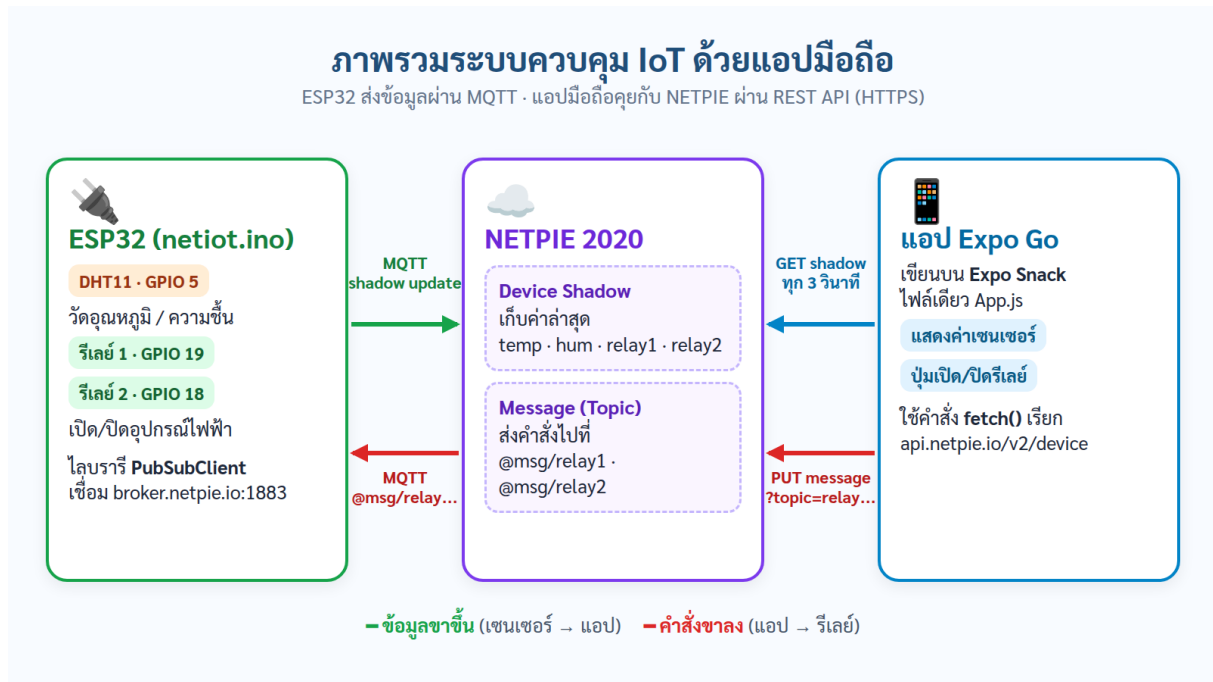
4. iPhone: เปิดแอปกล้อง (Camera) ปกติของเครื่อง เล็งไปที่ QR แล้วแตะแถบแจ้งเตือนที่ด้งขึ้นมาเพื่อเปิดใน Expo Go

5. รอสักครูให้โหลดโปรเจกต์ จากนั้นหน้าจอมือถือจะแสดงข้อความ "สวัสดี! นี่คือการแรกของฉัน" ยินดีด้วย — คุณแสดงผลข้อความแรกบนมือถือสำเร็จแล้ว!



ขั้นตอนที่ 5: ทำความเข้าใจภาพรวมระบบควบคุม IoT

เมื่อแสดงข้อความแรกบนมือถือได้แล้ว ขั้นตอนต่อไปคือสร้างแอป "Smart IoT Control" สำหรับควบคุมบอร์ด ESP32 จากโค้ด netiot ซึ่งต่อเซนเซอร์ DHT11 (วัดอุณหภูมิและความชื้น) และรีเลย์ 2 ช่อง โดยใช้แพลตฟอร์ม NETPIE 2020 เป็นตัวกลางรับส่งข้อมูล



ระบบแบ่งเป็น 3 ส่วน ทำงานร่วมกันดังนี้

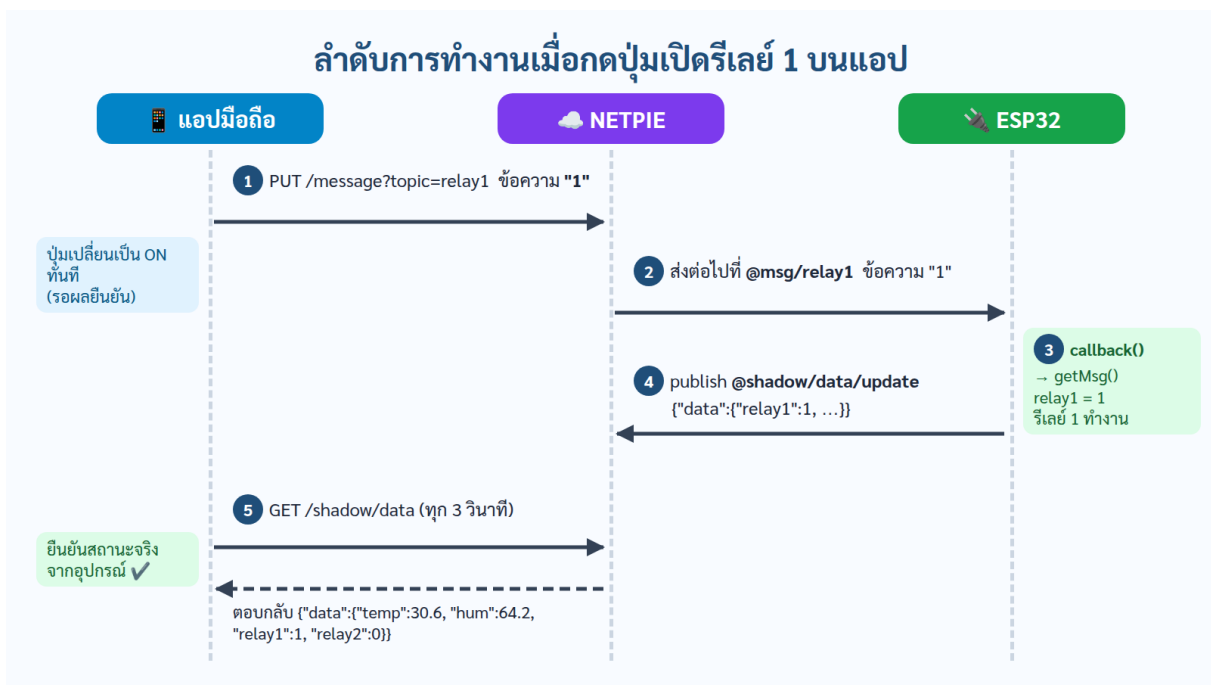
1. **ESP32** อ่านค่าเซนเซอร์ทุก 2 วินาที แล้วส่งขึ้น Device Shadow ของ NETPIE ผ่าน MQTT พร้อมรอรับคำสั่งเปิด/ปิดรีเลย์จาก Topic @msg/relay1 และ @msg/relay2
2. **NETPIE** เก็บค่าล่าสุดไว้ใน Shadow และส่งต่อข้อความคำสั่งไปยังอุปกรณ์
3. **แอปมือถือ** ใช้คำสั่ง fetch() เรียก REST API ของ NETPIE ผ่าน HTTPS เพื่ออ่านค่า Shadow และส่งคำสั่งข้อดีคือไม่ต้องติดตั้งไลบรารีเพิ่มใน Expo Snack

ตารางช่องทางการรับส่งข้อมูลในระบบ

ทิศทาง	วิธี	ปลายทาง (Topic / URL)	ข้อมูลที่ส่ง
ESP32 → NETPIE	MQTT publish	@shadow/data/update	{"data":{"temp":30.6, "hum":64.2, "relay1":0, "relay2":1}}
NETPIE → ESP32	MQTT subscribe	@msg/relay1, @msg/relay2	"1" = เปิด, "0" = ปิด

ทิศทาง	วิธี	ปลายทาง (Topic / URL)	ข้อมูลที่ส่ง
แอป → NETPIE (อ่านค่า)	HTTP GET	api.netpie.io/v2/device/shadow /data	ได้ JSON ค่าล่าสุดกลับมา
แอป → NETPIE (สั่งงาน)	HTTP PUT	api.netpie.io/v2/device/messag e?topic=relay1	ข้อความ "1" หรือ "0"

ทุกครั้งที่แอปเรียก API ต้องแนบส่วนหัว (Header) ชื่อ Authorization ในรูปแบบ Device ClientID:Token โดยใช้ Client ID และ Token ของ Device เดียวกับที่ใส่ในโค้ด ESP32 สังเกตว่าเมื่อส่งผ่าน REST ไม่ต้องใส่ @msg/ นำหน้าชื่อ Topic



เมื่อผู้ใช้กดปุ่ม แอปจะเปลี่ยนสถานะปุ่มบนจอทันที แล้วส่งคำสั่งไปที่ NETPIE (1) NETPIE ส่งต่อให้ ESP32 (2) ESP32 สั่งรีเลย์ทำงาน (3) และรายงานสถานะใหม่กลับขึ้น Shadow (4) เมื่อแอปอ่าน Shadow ในรอบถัดไปก็จะได้รับการยืนยันสถานะจริงจากอุปกรณ์ (5)

ขั้นตอนที่ 6: สร้างแอปควบคุม IoT บน Expo Snack

1. เข้าเว็บ snack.expo.dev แล้วสร้าง Snack ใหม่ ตั้งชื่อโปรเจกต์ว่า Smart IoT Control
2. คลิกไฟล์ App.js ลบโค้ดเดิมทั้งหมด แล้ววางโค้ดด้านล่างนี้แทน
3. แก้บรรทัด CLIENT_ID และ TOKEN ให้เป็นค่าของ Device เดียวกับที่ใส่ใน ESP32 (ใช้ Token ไม่ใช่

Secret)

```

import React, { useState, useEffect, useRef, useCallback } from 'react';
import {
  View,
  Text,
  StyleSheet,
  TouchableOpacity,
  ScrollView,
  RefreshControl,
  ActivityIndicator,
  StatusBar,
} from 'react-native';

// ----- ส่วนที่ 1: ตั้งค่าการเชื่อมต่อ NETPIE -----
// ใช้ Client ID และ Token ของ Device เดียวกับที่ใส่ในโค้ด ESP32
const CLIENT_ID = 'ใส่ Client ID ของอุปกรณ์';
const TOKEN = 'ใส่ Token ของอุปกรณ์';

const API_URL = 'https://api.netpie.io/v2/device';
const AUTH = 'Device ' + CLIENT_ID + ':' + TOKEN;
const POLL_MS = 3000; // ดึงข้อมูลใหม่ทุก 3 วินาที

// รายการรีเลย์ที่จะแสดงบนหน้าจอ (key ต้องตรงกับชื่อใน Shadow และชื่อ Topic)
const RELAYS = [
  { key: 'relay1', name: 'รีเลย์ 1', pin: 'GPIO 19', icon: '💡' },
  { key: 'relay2', name: 'รีเลย์ 2', pin: 'GPIO 18', icon: '🌀' },
];

// ----- ส่วนที่ 2: ฟังก์ชันเรียก NETPIE REST API -----
// อ่านข้อมูลล่าสุดจาก Device Shadow
async function getShadow() {
  const res = await fetch(API_URL + '/shadow/data', {
    method: 'GET',
    headers: { Authorization: AUTH },
  });
  if (!res.ok) throw new Error('อ่านข้อมูลไม่สำเร็จ (HTTP ' + res.status + ')');
  const json = await res.json();
  return json.data || {};
}

// ส่งข้อความไปที่ Topic (REST ไม่ต้องใส่ @msg/ หน้าหน้า)
// เช่น sendMessage('relay1', '1') ESP32 จะได้รับที่ @msg/relay1
async function sendMessage(topic, message) {
  const res = await fetch(API_URL + '/message?topic=' + topic, {
    method: 'PUT',
    headers: { Authorization: AUTH, 'Content-Type': 'text/plain' },
    body: message,
  });
  if (!res.ok) throw new Error('ส่งคำสั่งไม่สำเร็จ (HTTP ' + res.status + ')');
}

```

```
// ----- ส่วนที่ 3: คอมโพเนนต์หลักของแอป -----
export default function App() {
  const [temp, setTemp] = useState(null);
  const [hum, setHum] = useState(null);
  const [relays, setRelays] = useState({ relay1: 0, relay2: 0 });
  const [busy, setBusy] = useState({}); // รีเลย์ที่กำลังส่งคำสั่ง
  const [connected, setConnected] = useState(false);
  const [lastUpdate, setLastUpdate] = useState('-');
  const [error, setError] = useState('');
  const [refreshing, setRefreshing] = useState(false);

  // จำคำสั่งที่เพิกัด เพื่อไม่ให้ค่าเก่าจาก Shadow มาเปลี่ยนสวิตช์กลับ
  const pending = useRef({});

  // ----- ส่วนที่ 4: ดึงข้อมูลจาก NETPIE -----
  const loadData = useCallback(async () => {
    try {
      const data = await getShadow();
      if (data.temp !== undefined) setTemp(Number(data.temp));
      if (data.hum !== undefined) setHum(Number(data.hum));

      setRelays((old) => {
        const next = { ...old };
        RELAYS.forEach(({ key }) => {
          if (data[key] === undefined) return;
          const p = pending.current[key];
          const waiting =
            p && Date.now() < p.until && Number(data[key]) !== p.value;
          if (!waiting) {
            next[key] = Number(data[key]);
            delete pending.current[key];
          }
        });
      });
      return next;
    } catch (e) {
      setConnected(false);
      setError(e.message);
    }
  }, []);

  setConnected(true);
  setError('');
  setLastUpdate(new Date().toLocaleTimeString('th-TH'));
} catch (e) {
  setConnected(false);
  setError(e.message);
}

// เรียก loadData ครั้งแรก และตั้งเวลาเรียกซ้ำทุก POLL_MS
useEffect(() => {
  loadData();
  const timer = setInterval(loadData, POLL_MS);

```

```

    return () => clearInterval(timer); // ล้าง timer เมื่อปิดหน้าจอ
  }, [loadData]);

// ----- ส่วนที่ 5: กดปุ่มเพื่อสั่งเปิด/ปิดรีเลย์ -----
const toggleRelay = async (key) => {
  const newValue = relays[key] ? 0 : 1;
  pending.current[key] = { value: newValue, until: Date.now() + 6000 };
  setRelays((old) => ({ ...old, [key]: newValue })); // เปลี่ยนบนจอทันที
  setBusy((b) => ({ ...b, [key]: true }));
  try {
    await sendMessage(key, String(newValue));
    setError('');
  } catch (e) {
    delete pending.current[key];
    // ส่งไม่สำเร็จ คืนค่าเดิม
    setRelays((old) => ({ ...old, [key]: newValue ? 0 : 1 }));
    setError(e.message);
  } finally {
    setBusy((b) => ({ ...b, [key]: false }));
  }
};

const onRefresh = async () => {
  setRefreshing(true);
  await loadData();
  setRefreshing(false);
};

const show = (v) => (v === null || isNaN(v) ? '--' : v.toFixed(1));

// ----- ส่วนที่ 6: ส่วนแสดงผล (UI) -----
return (
  <View style={styles.screen}>
    <StatusBar barStyle="dark-content" />
    <ScrollView
      contentContainerStyle={styles.container}
      refreshControl={
        <RefreshControl refreshing={refreshing} onRefresh={onRefresh} />
      }>
      {/* หัวแอป */}
      <Text style={styles.title}>Smart IoT Control</Text>
      <Text style={styles.subtitle}>ESP32 + NETPIE 2020</Text>

      <View style={[styles.badge, connected ? styles.bgOn : styles.bgOff]}>
        <View style={[styles.dot, connected ? styles.dotOn : styles.dotOff]} />
        <Text style={styles.badgeText}>
          {connected ? 'เชื่อมต่อ NETPIE แล้ว' : 'ยังไม่เชื่อมต่อ'}
        </Text>
      </View>
      <Text style={styles.updated}>อัปเดตล่าสุด {lastUpdate}</Text>
    </ScrollView>
  </View>
);

```

```

{error !== '' && <Text style={styles.error}>⚠ {error}</Text>}

{/* การ์ดเซนเซอร์ */}
<Text style={styles.section}>ค่าจากเซนเซอร์ DHT11</Text>
<View style={styles.row}>
  <View style={[styles.card, styles.sensorCard, styles.tempCard]}>
    <Text style={styles.sensorIcon}>🌡</Text>
    <Text style={styles.sensorLabel}>อุณหภูมิ</Text>
    <Text style={[styles.sensorValue, styles.tempText]}>{show(temp)}</Text>
    <Text style={styles.unit}>°C</Text>
  </View>
  <View style={[styles.card, styles.sensorCard, styles.humCard]}>
    <Text style={styles.sensorIcon}>💧</Text>
    <Text style={styles.sensorLabel}>ความชื้น</Text>
    <Text style={[styles.sensorValue, styles.humText]}>{show(hum)}</Text>
    <Text style={styles.unit}>%RH</Text>
  </View>
</View>

{/* การ์ดรีเลย์ */}
<Text style={styles.section}>ควบคุมอุปกรณ์</Text>
{RELAYS.map((r) => {
  const on = relays[r.key] === 1;
  return (
    <View key={r.key} style={[styles.card, styles.relayCard]}>
      <Text style={styles.relayIcon}>{r.icon}</Text>
      <View style={{ flex: 1 }}>
        <Text style={styles.relayName}>{r.name}</Text>
        <Text style={styles.relayPin}>{r.pin}</Text>
        <Text style={[styles.relayState, on ? styles.textOn : styles.textOff]}>
          {on ? 'กำลังทำงาน' : 'ปิดอยู่'}
        </Text>
      </View>
      <TouchableOpacity
        style={[styles.button, on ? styles.buttonOn : styles.buttonOff]}
        onPress={() => toggleRelay(r.key)}
        disabled={busy[r.key]}>
        {busy[r.key] ? (
          <ActivityIndicator color="#fff" />
        ) : (
          <Text style={styles.buttonText}>{on ? 'ON' : 'OFF'}</Text>
        )}
      </TouchableOpacity>
    </View>
  );
})}

<Text style={styles.hint}>dingหน้าจอเพื่อโหลดข้อมูลใหม่</Text>
</ScrollView>

```

```

    </View>
  );
}

// ----- ส่วนที่ 7: ตกแต่งหน้าจอ -----
const styles = StyleSheet.create({
  screen: { flex: 1, backgroundColor: '#f0f6ff' },
  container: { padding: 20, paddingTop: 56 },
  title: { fontSize: 28, fontWeight: 'bold', color: '#1f4e79' },
  subtitle: { fontSize: 16, color: '#64748b', marginBottom: 12 },
  badge: {
    flexDirection: 'row',
    alignItems: 'center',
    alignSelf: 'flex-start',
    paddingVertical: 6,
    paddingHorizontal: 12,
    borderRadius: 20,
  },
  bgOn: { backgroundColor: '#dcfce7' },
  bgOff: { backgroundColor: '#fee2e2' },
  dot: { width: 10, height: 10, borderRadius: 5, marginRight: 8 },
  dotOn: { backgroundColor: '#16a34a' },
  dotOff: { backgroundColor: '#dc2626' },
  badgeText: { fontSize: 15, fontWeight: '600', color: '#1e293b' },
  updated: { fontSize: 13, color: '#64748b', marginTop: 6 },
  error: {
    marginTop: 12,
    padding: 10,
    borderRadius: 10,
    backgroundColor: '#fef2f2',
    color: '#b91c1c',
    fontSize: 14,
  },
  section: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#1f4e79',
    marginTop: 24,
    marginBottom: 10,
  },
  row: { flexDirection: 'row', gap: 12 },
  card: {
    backgroundColor: 'ffffff',
    borderRadius: 16,
    padding: 16,
    shadowColor: 'black',
    shadowOpacity: 0.08,
    shadowRadius: 8,
    shadowOffset: { width: 0, height: 2 },
    elevation: 3,
  },
});

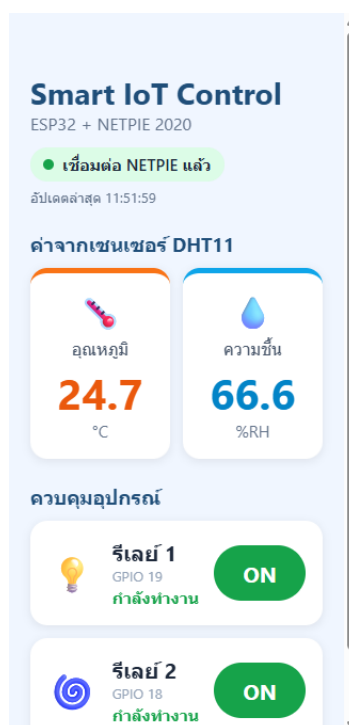
```

```

sensorCard: { flex: 1, alignItems: 'center', borderTopWidth: 5 },
tempCard: { borderTopColor: '#f97316' },
humCard: { borderTopColor: '#0ea5e9' },
sensorIcon: { fontSize: 30 },
sensorLabel: { fontSize: 16, color: '#475569', marginTop: 4 },
sensorValue: { fontSize: 40, fontWeight: 'bold', marginTop: 4 },
tempText: { color: '#ea580c' },
humText: { color: '#0284c7' },
unit: { fontSize: 15, color: '#64748b' },
relayCard: { flexDirection: 'row', alignItems: 'center', marginBottom: 12 },
relayIcon: { fontSize: 34, marginRight: 14 },
relayName: { fontSize: 20, fontWeight: 'bold', color: '#1e293b' },
relayPin: { fontSize: 13, color: '#94a3b8' },
relayState: { fontSize: 15, fontWeight: '600', marginTop: 2 },
textOn: { color: '#16a34a' },
textOff: { color: '#64748b' },
button: {
  width: 86,
  height: 52,
  borderRadius: 26,
  justifyContent: 'center',
  alignItems: 'center',
},
buttonOn: { backgroundColor: '#16a34a' },
buttonOff: { backgroundColor: '#94a3b8' },
buttonText: { color: 'fff', fontSize: 20, fontWeight: 'bold' },
hint: { textAlign: 'center', color: '#94a3b8', marginTop: 12, fontSize: 13 },
});

```

4. ทดสอบการทำงานบนมือถือ



ขั้นตอนที่ 8: ทำความเข้าใจโค้ดทีละส่วน

โค้ดแบ่งเป็น 7 ส่วนตามหมายเหตุ (comment) ในโค้ด หน้าที่ของแต่ละส่วนสรุปได้ดังตาราง

ตารางหน้าที่ของโค้ดแต่ละส่วนใน App.js

ส่วนที่	หน้าที่	คำสั่ง / แนวคิดสำคัญ
1	ตั้งค่าการเชื่อมต่อ NETPIE และรายการรีเลย์	const, อาร์เรย์ RELAYS
2	ฟังก์ชันอ่าน Shadow และส่งข้อความคำสั่ง	async / await, fetch()
3	สร้างตัวแปรสถานะของหน้าจอ	useState, useRef
4	ดึงข้อมูลอัตโนมัติทุก 3 วินาที	useEffect, setInterval, clearInterval
5	กดปุ่มเพื่อเปิด/ปิดรีเลย์	try / catch / finally
6	ส่วนแสดงผลหน้าจอ	JSX, .map() สร้างการ์ดรีเลย์
7	ตกแต่งสีและขนาด	StyleSheet.create

แนวคิดสำคัญที่ควรเข้าใจมีดังนี้

- useState เก็บค่าที่แสดงบนจอ เช่น temp, hum, relays เมื่อเรียก setTemp() หน้าจอจะวาดใหม่อัตโนมัติ
- useEffect + setInterval สั่งให้ loadData() ทำงานทันทีเมื่อเปิดแอป และทำซ้ำทุก POLL_MS มิลลิวินาที ส่วนคำสั่ง return () => clearInterval(timer) ใช้หยุดการทำงานเมื่อปิดหน้าจอ
- fetch() กับ async/await ใช้เรียก API แล้วรอผลลัพธ์ ถ้า res.ok เป็นเท็จ (เช่น HTTP 401 รหัสไม่ถูกต้อง) จะโยน Error ไปแสดงบนจอ
- เปลี่ยนปุ่มทันที (Optimistic Update) เมื่อกดปุ่ม แอปเปลี่ยนสถานะบนจอก่อน แล้วจำค่าไว้ใน pending ประมาณ 6 วินาที ระหว่างนั้นถ้า Shadow ยังเป็นค่าเก่า แอปจะไม่เปลี่ยนปุ่มกลับ ทำให้ปุ่มไม่กระพริบ และถ้าส่งคำสั่งไม่สำเร็จ แอปจะคืนค่าเดิมพร้อมแสดงข้อผิดพลาด
- RELAYS.map() สร้างการ์ดรีเลย์จากอาร์เรย์ หากต้องการเพิ่มรีเลย์ช่องที่ 3 เพียงเพิ่มข้อมูลในอาร์เรย์ RELAYS หนึ่งบรรทัด